CSCI 566: Deep Learning and its Applications

Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California
Credits: prior USC CSCI 566 materials; Stanford CS231n.

USC Viterbi School of Engineering

Quizzes: Dynamic Scheduling

No fixed quiz dates

A quiz may be given if in-person attendance falls below 50%.

If a quiz is not administered:

Its weight moves to the project final report and presentation.

Policy clarification: Piazza @101.

Project Proposal Feedback

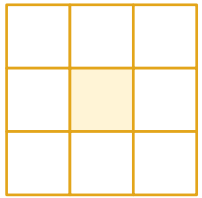
IN PROGRESS

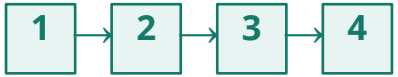
Graders are reviewing your proposals and providing feedback.

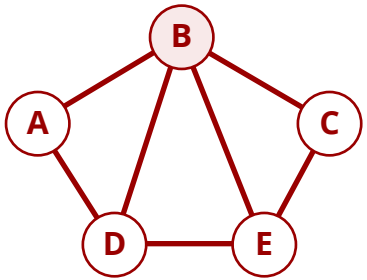
All feedback will be completed by

Sunday, October 4

From Grids to Sequences and Graphs







CNN: spatial offsets **RNN: previous state** **GNN: graph neighbors**

Shared parameters → structured computation → task-specific outputs

Keep parameter sharing; change which information can interact.

Course schematic.

Recurrent Neural Networks (RNN)

Motivation

What about speech recognition?



Slide credit: Dhruv Batra
Image credit: Alex Graves and Kevin Gimpel

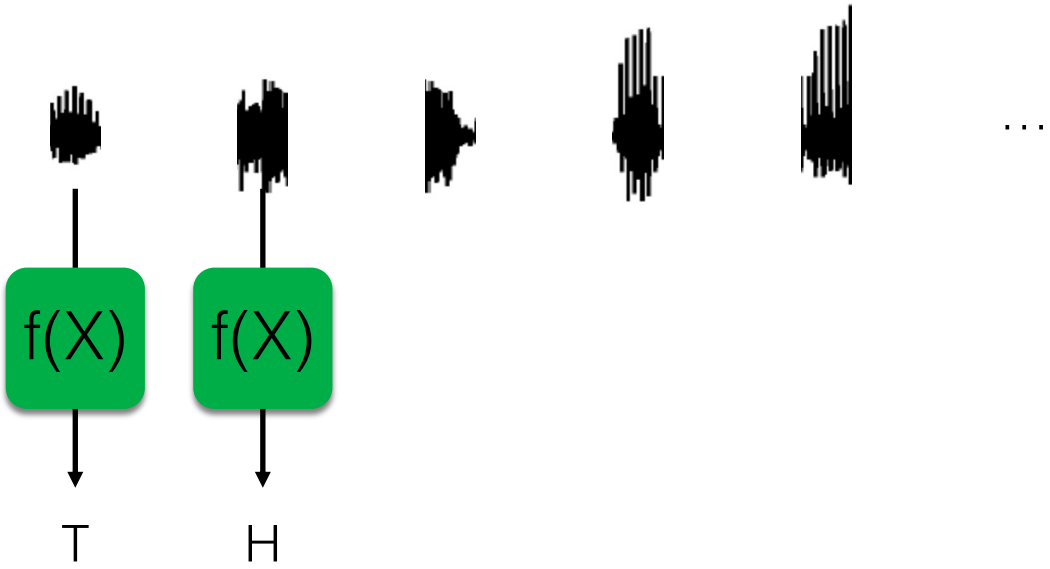
What about speech recognition?



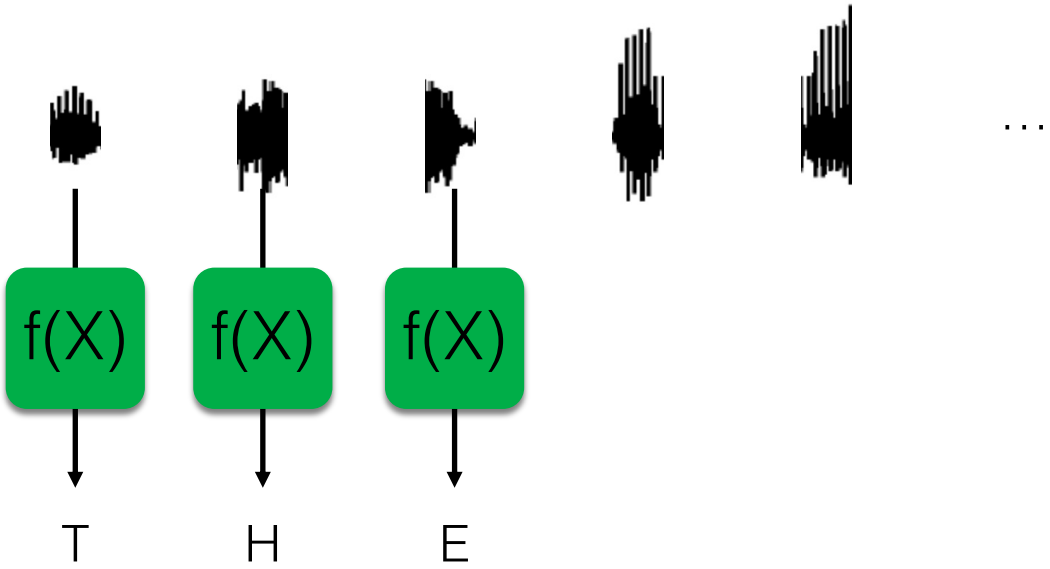
What about speech recognition?



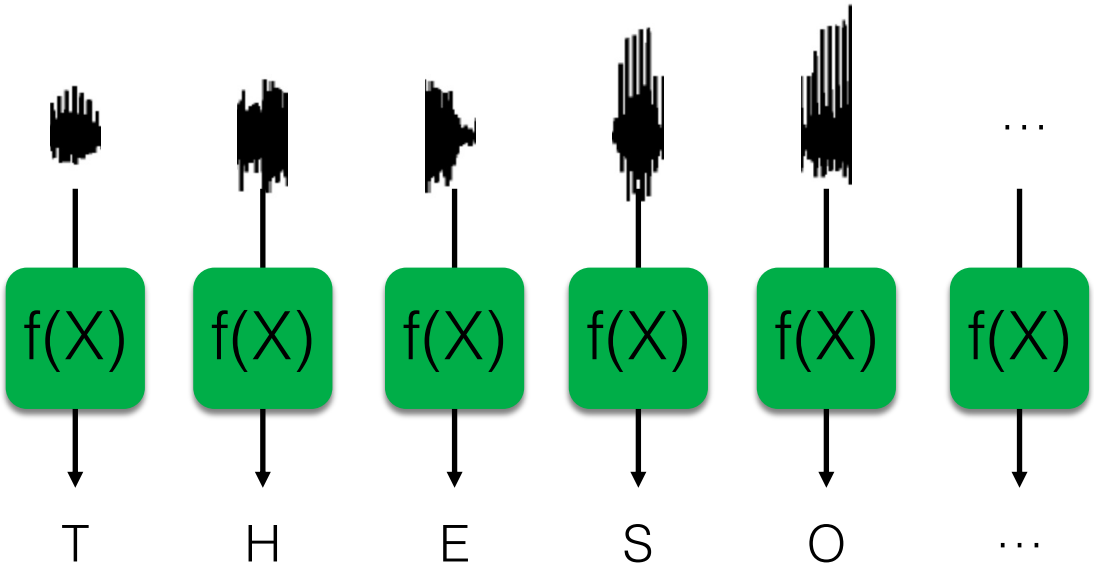
What about speech recognition?



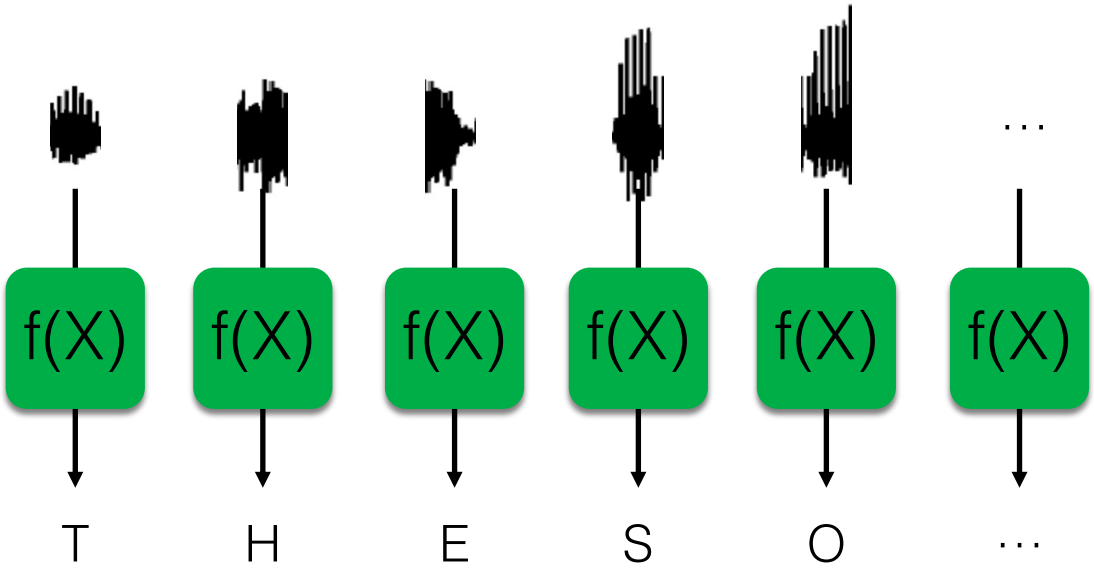
What about speech recognition?



What about speech recognition?

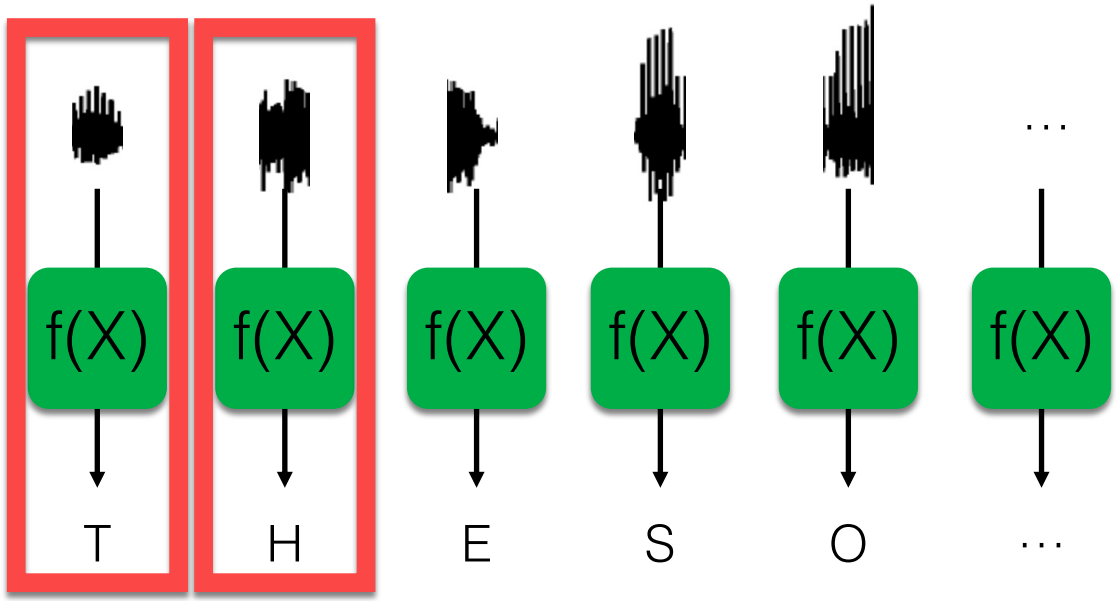


What about speech recognition?

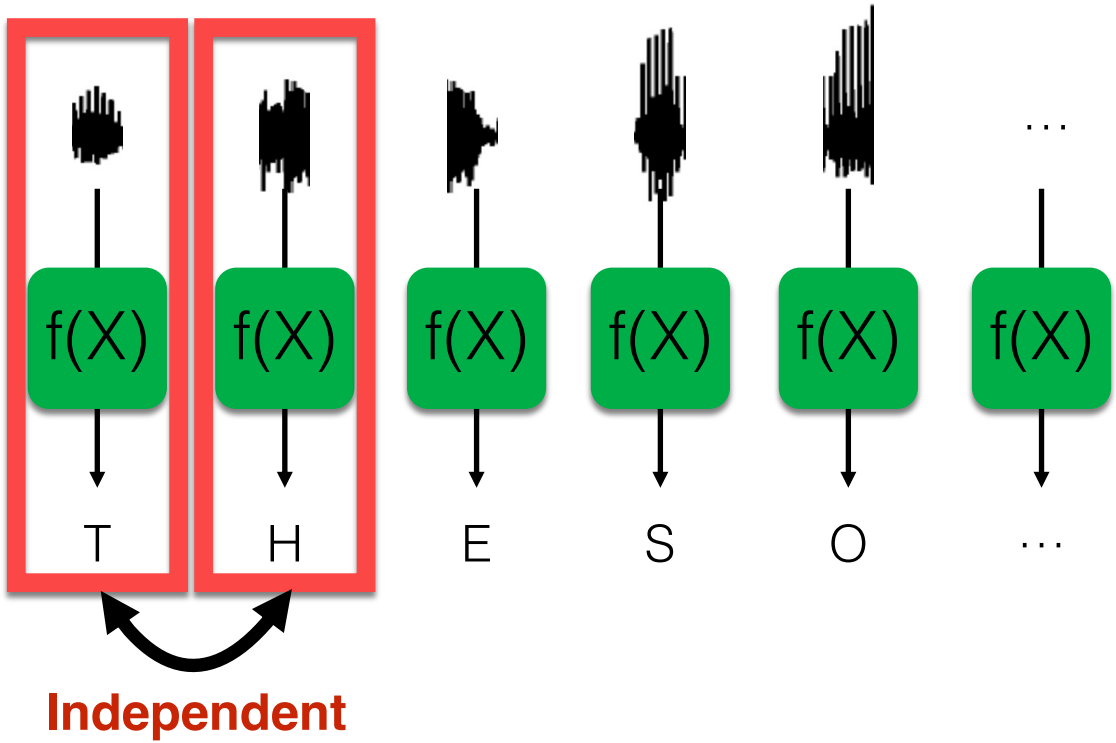


Any limitation on this approach?

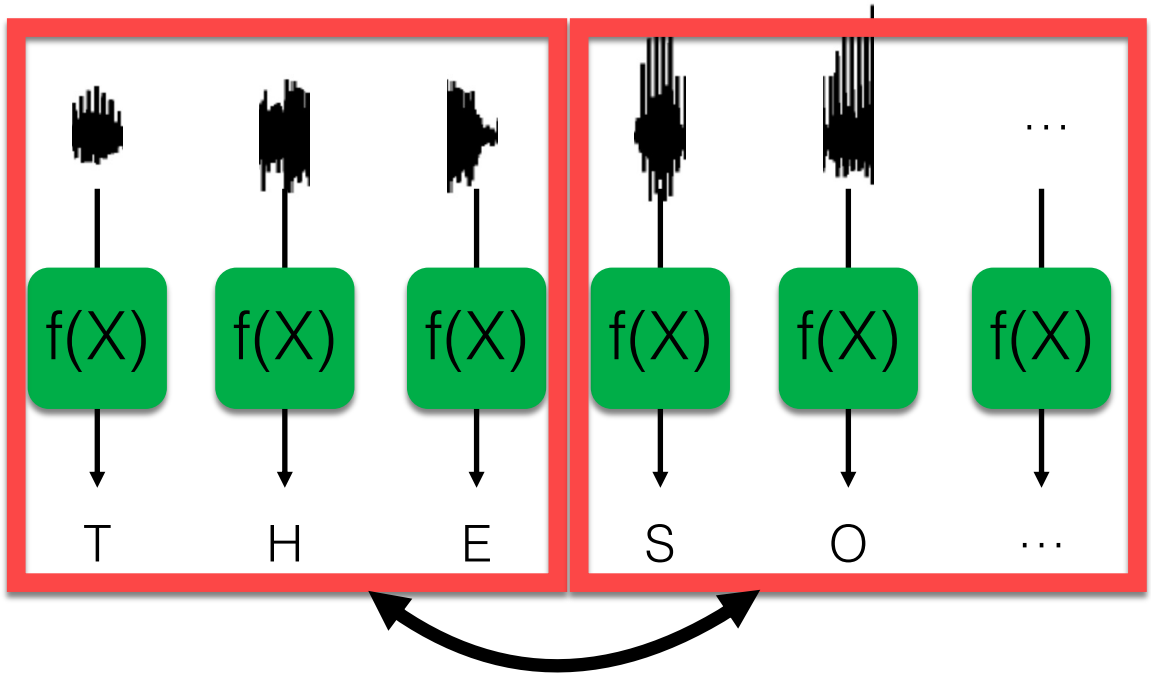
Independent Windows Lose Context



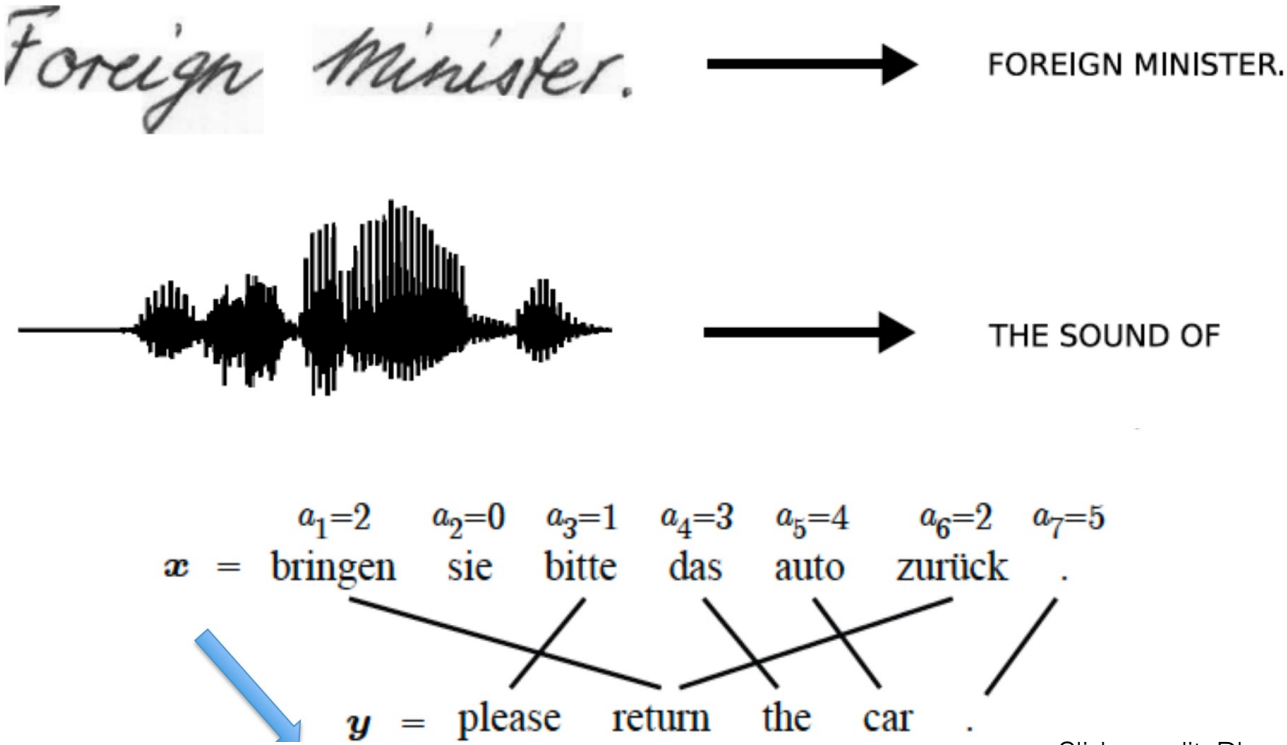
Independent Windows Lose Context



Independent Windows Lose Context

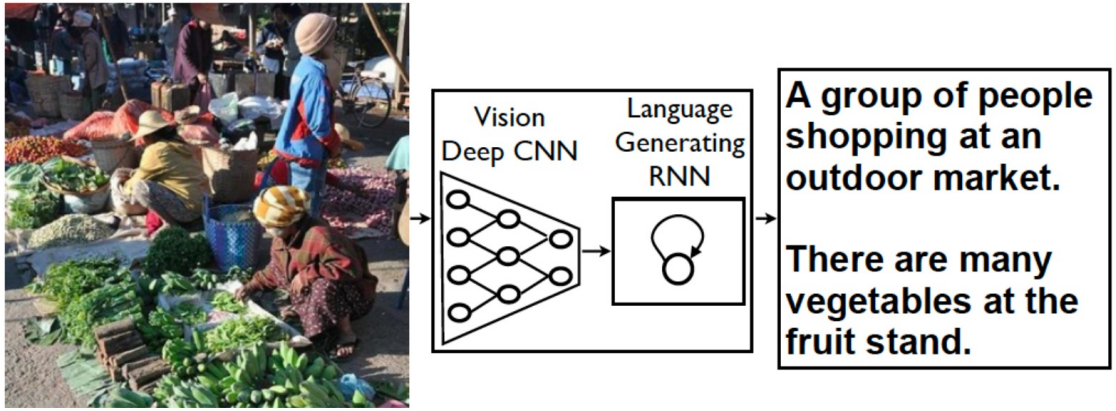


Sequential data is everywhere

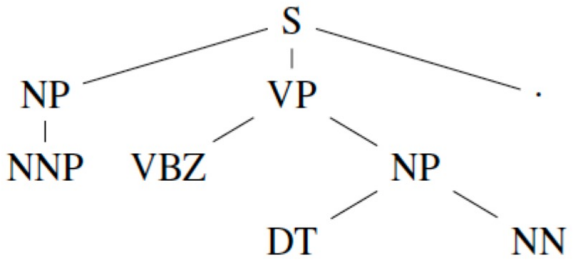


Slide credit: Dhruv Batra
Image credit: Alex Graves and Kevin Gimpel

Sequential data is everywhere



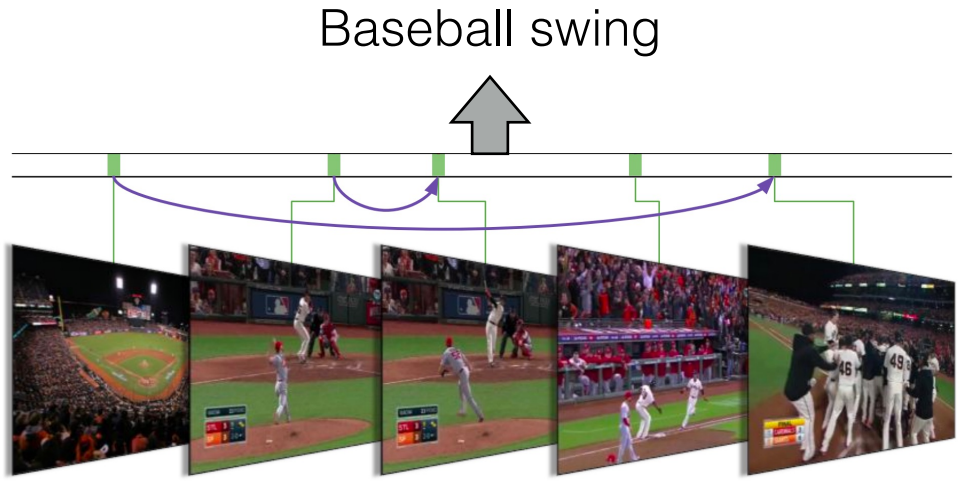
John has a dog . →



John has a dog . → (S (NP NNP)$_{NP}$ (VP VBZ (NP DT NN)$_{NP}$)$_{VP}$.)$_{S}$

Image credit: Vinyals et. al.

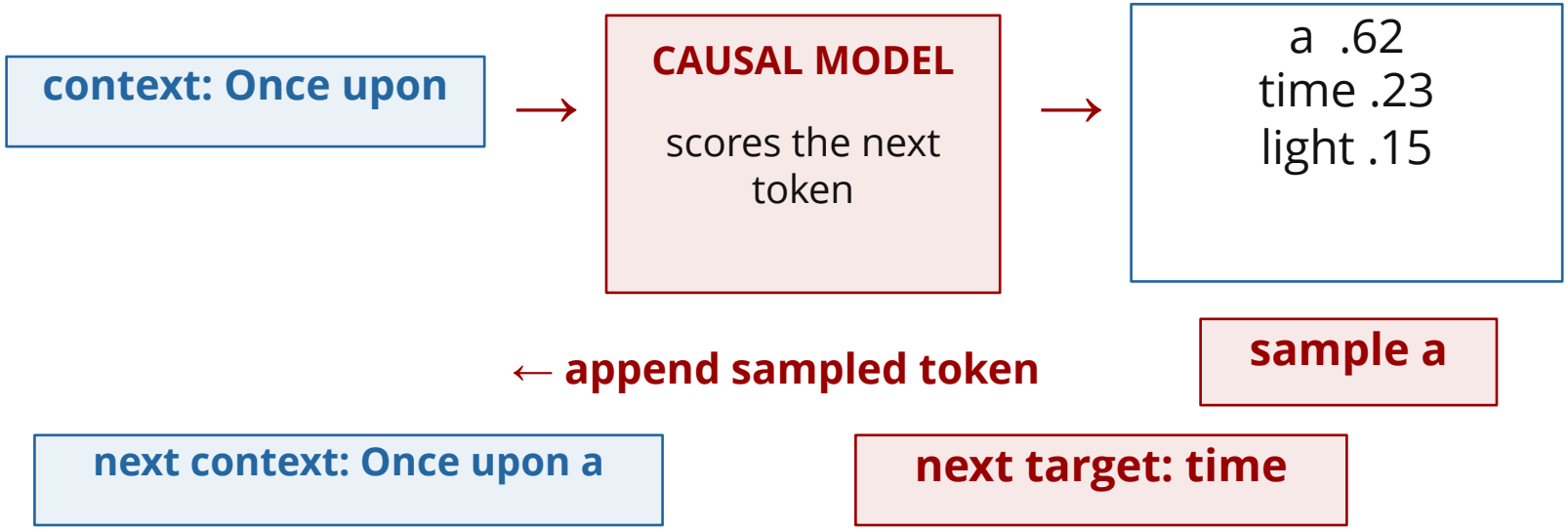
Sequential data is everywhere



Yeung, et. al. End-to-end Learning of Action Detection from Frame Glimpses in Videos. CVPR 2017.

Autoregression Is an Objective

1 / 2



Autoregressive = factorization and target, not one model family.

USC Viterbi School of Engineering

What Must a Sequence Model Handle?

a	big	house		
a	car	on	a	road

Different lengths. Earlier words can affect later predictions.

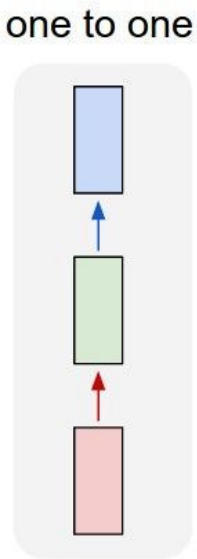
We need a computation that can continue as the sequence grows.

Course schematic.

Recurrent Neural Networks (RNN)

Basics

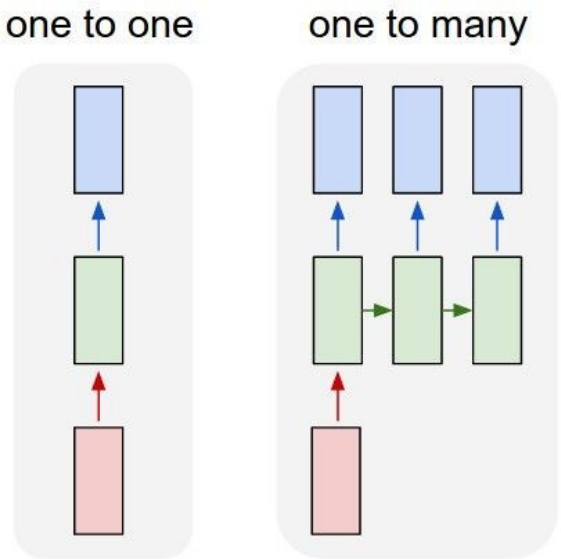
“Vanilla” Neural Network



Vanilla Neural Networks

Credit to Stanford cs231n lecture slides

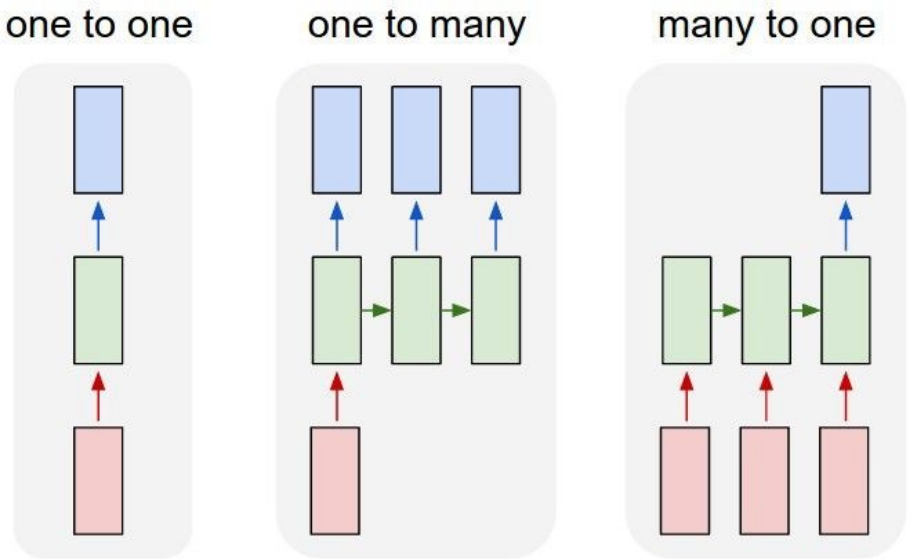
Recurrent Neural Networks: Process Sequences



e.g. **Image Captioning**
image -> sequence of words

Credit to Stanford cs231n lecture slides

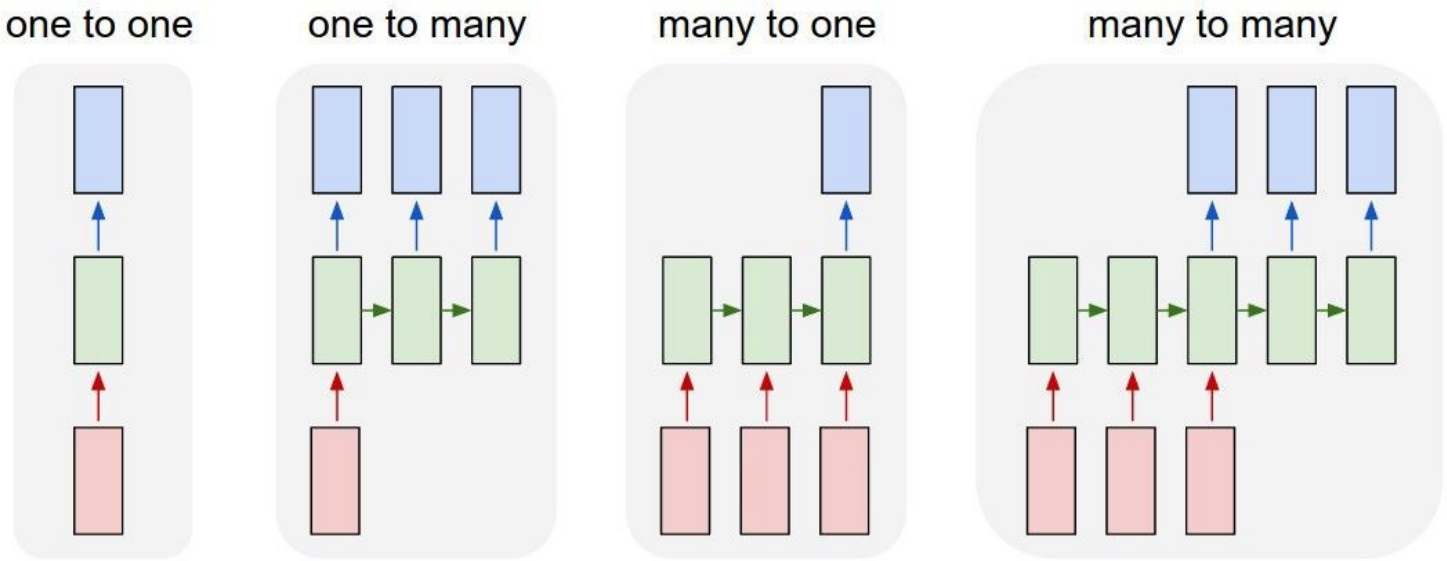
Recurrent Neural Networks: Process Sequences



e.g. **action prediction**
sequence of video frames -> action class

Credit to Stanford cs231n lecture slides

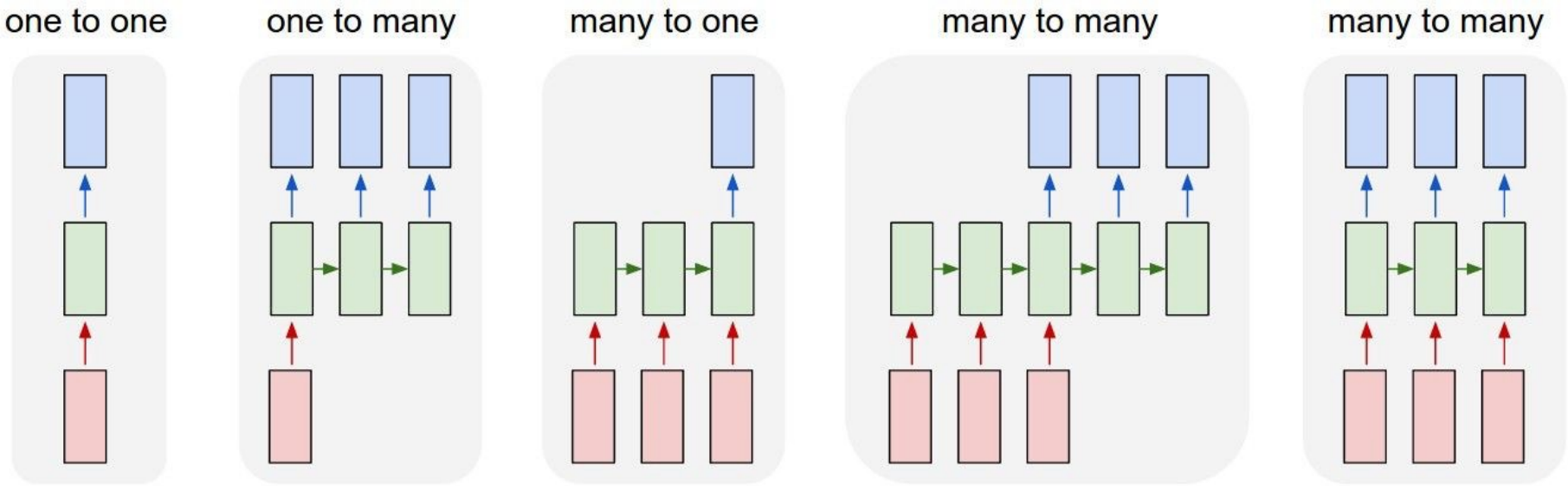
Recurrent Neural Networks: Process Sequences



E.g. **Video Captioning**
Sequence of video frames -> caption

USC Viterbi School of Engineering

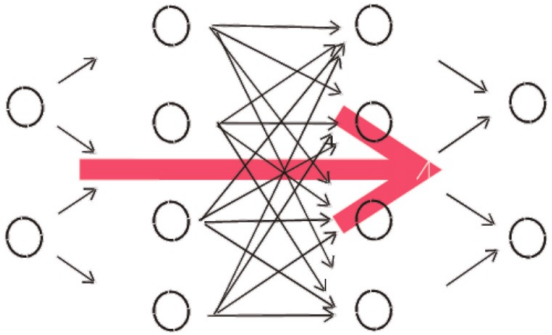
Recurrent Neural Networks: Process Sequences



e.g. **Video classification on frame level**

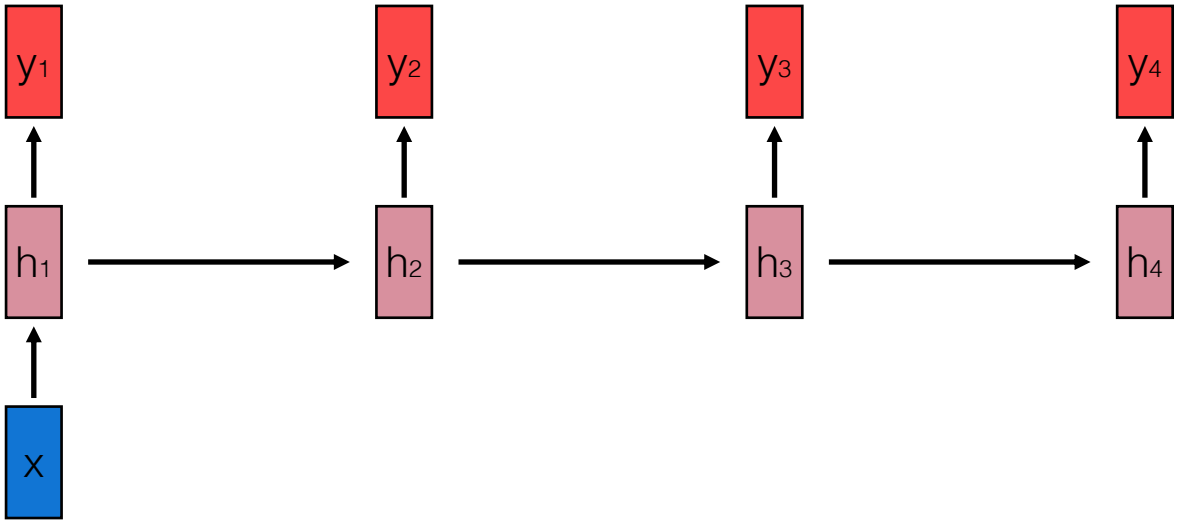
Credit to Stanford cs231n lecture slides

It can get tricky…

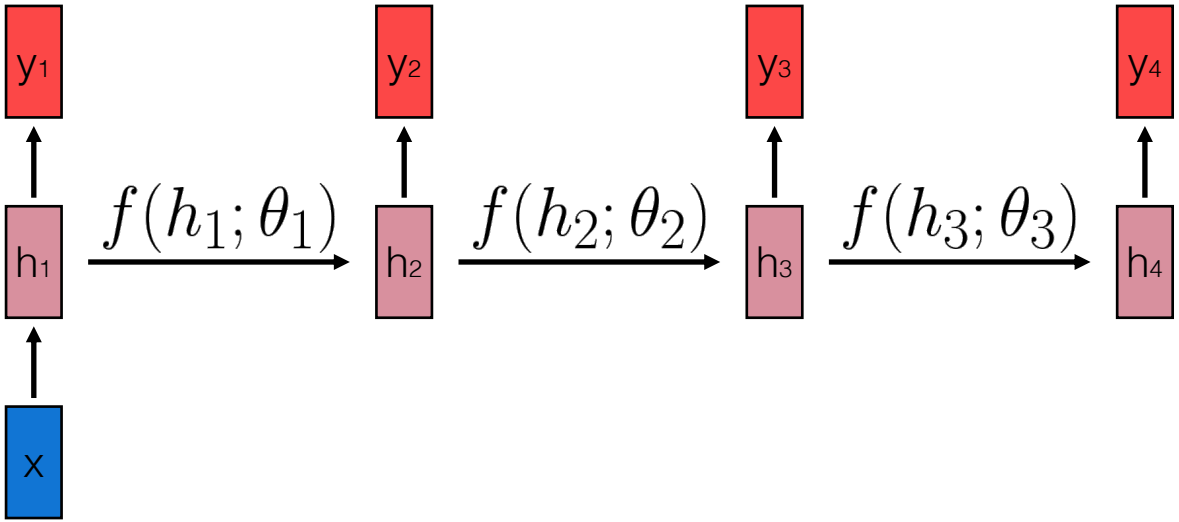




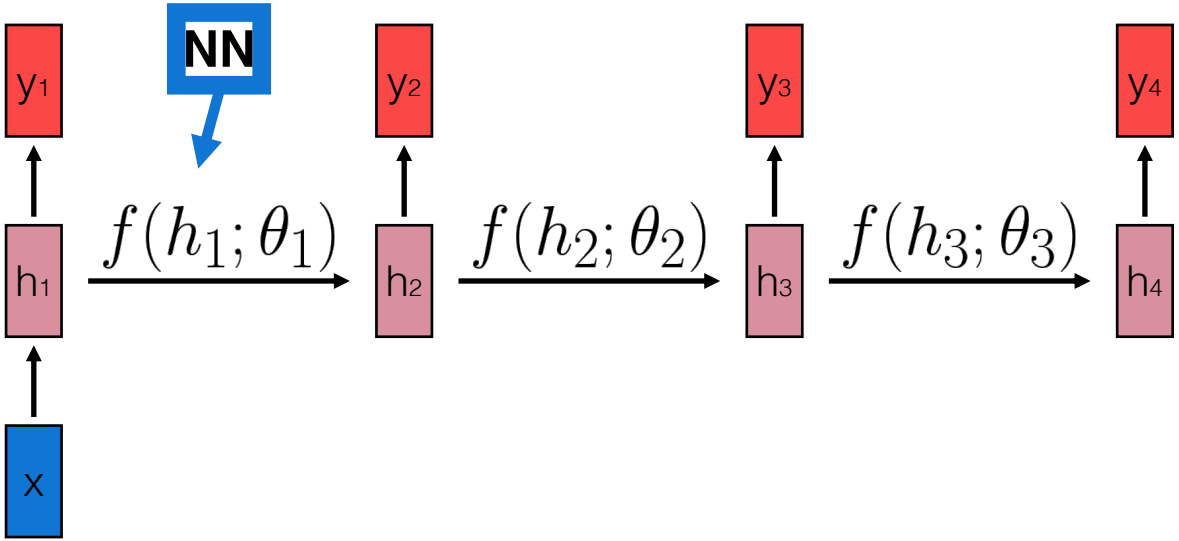
Let's zoom in!



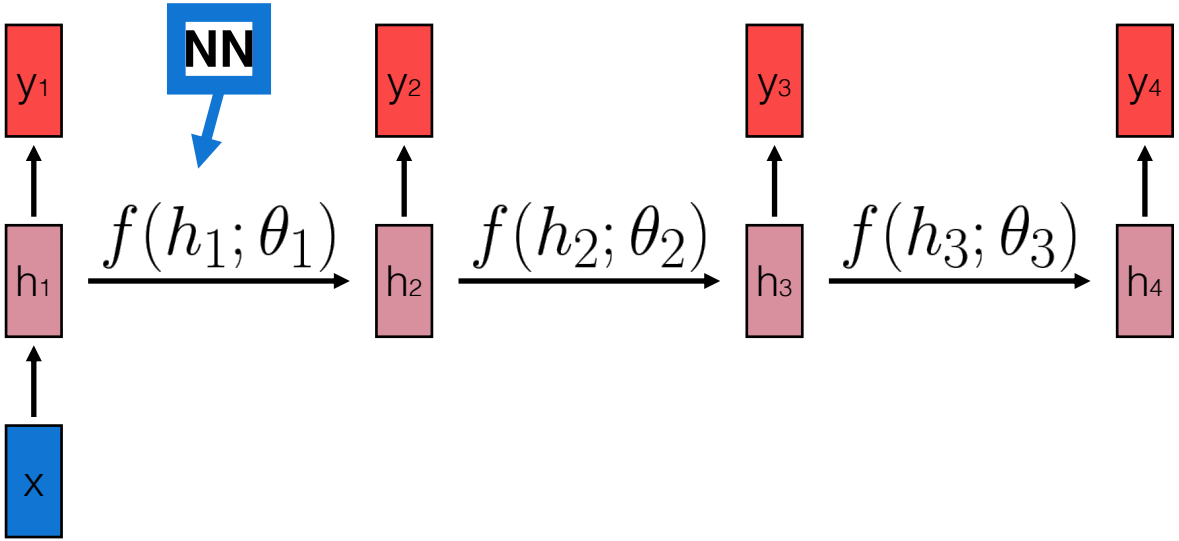
Let's zoom in!



Let's zoom in!



Let's zoom in!



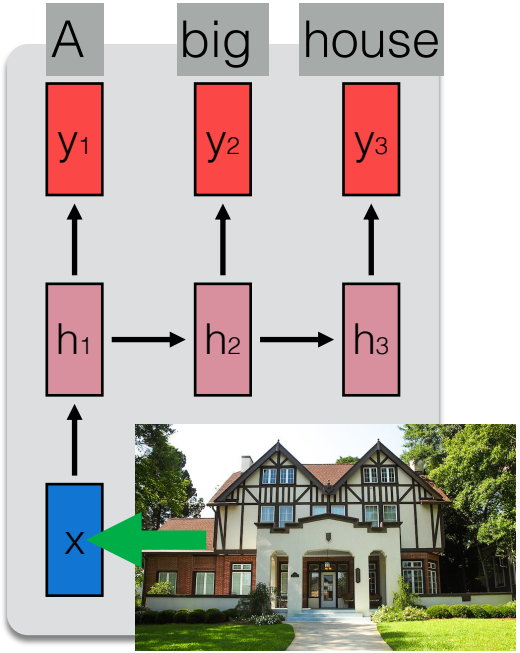
Any issue with this?

How do we model sequences?



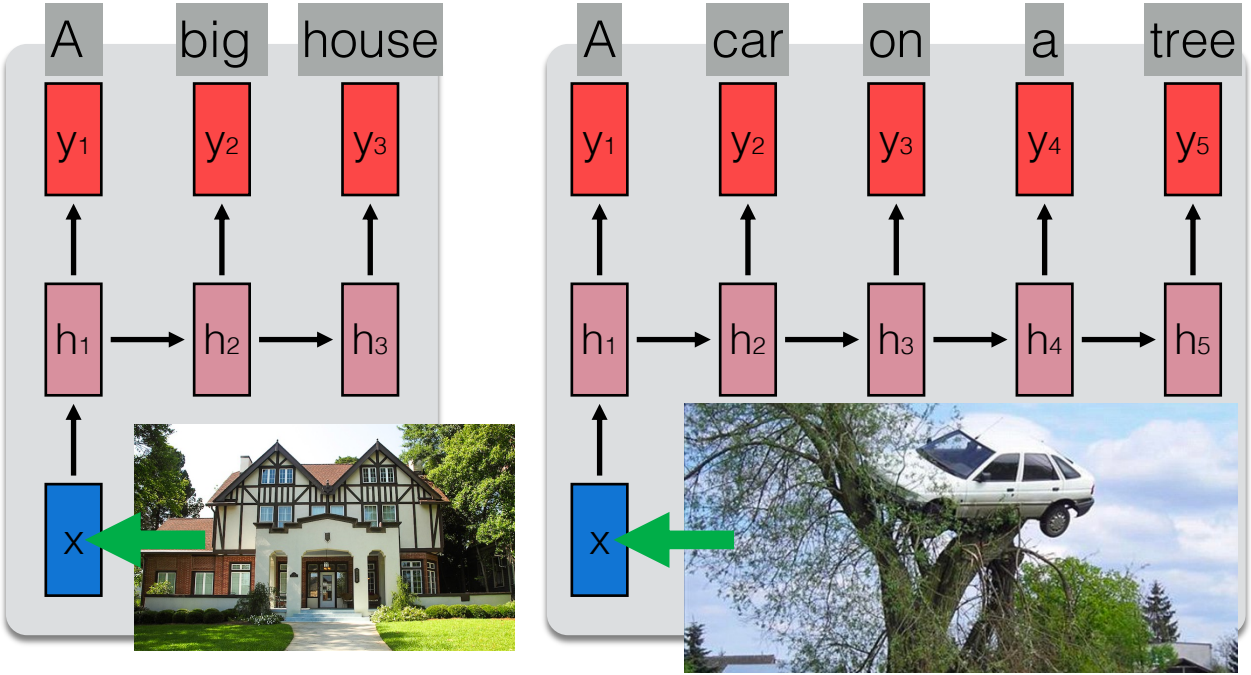


How do we model sequences?

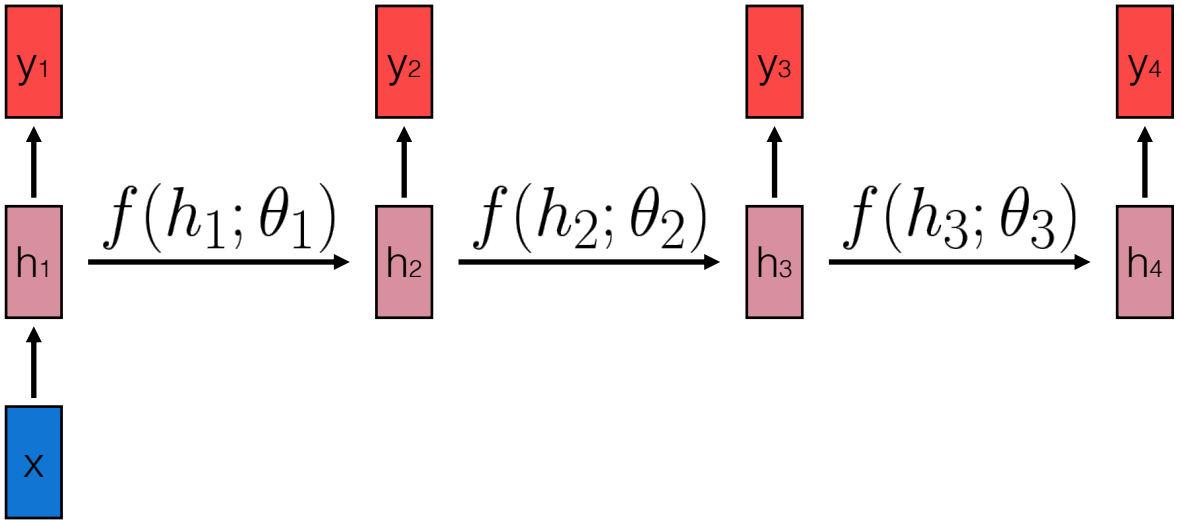




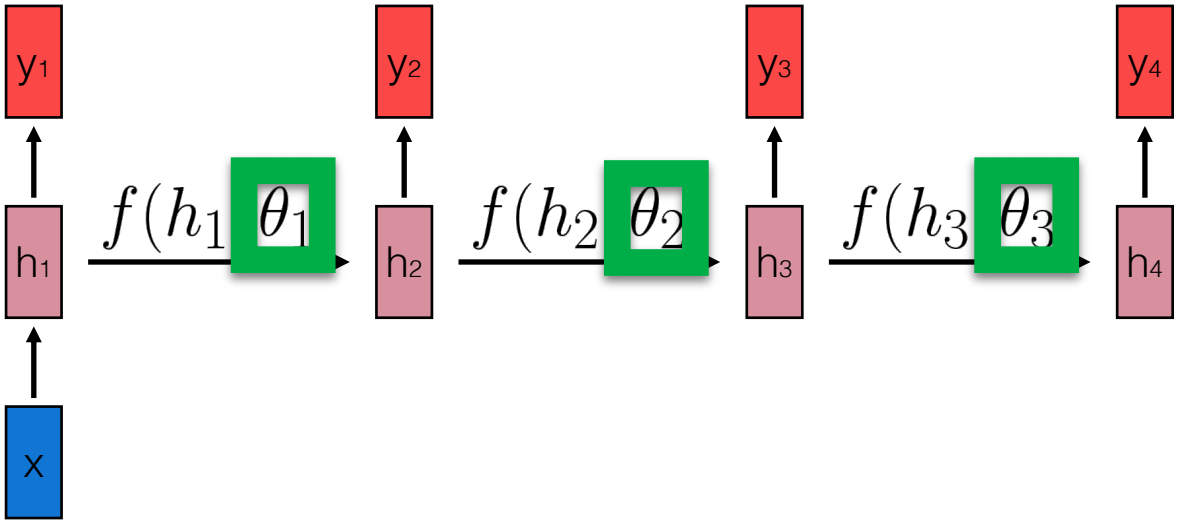
How do we model sequences?



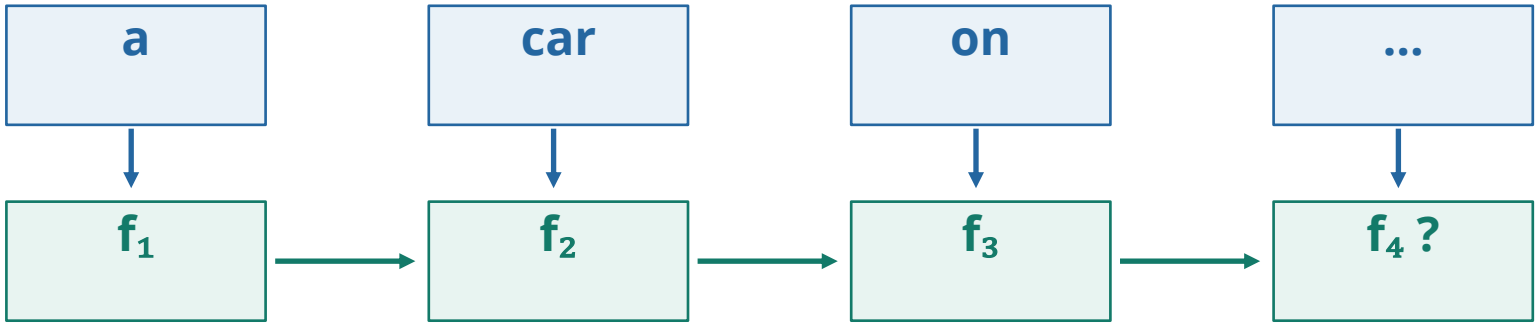
Recurrent Neural Network



Recurrent Neural Network



Why Reuse the Same Update?

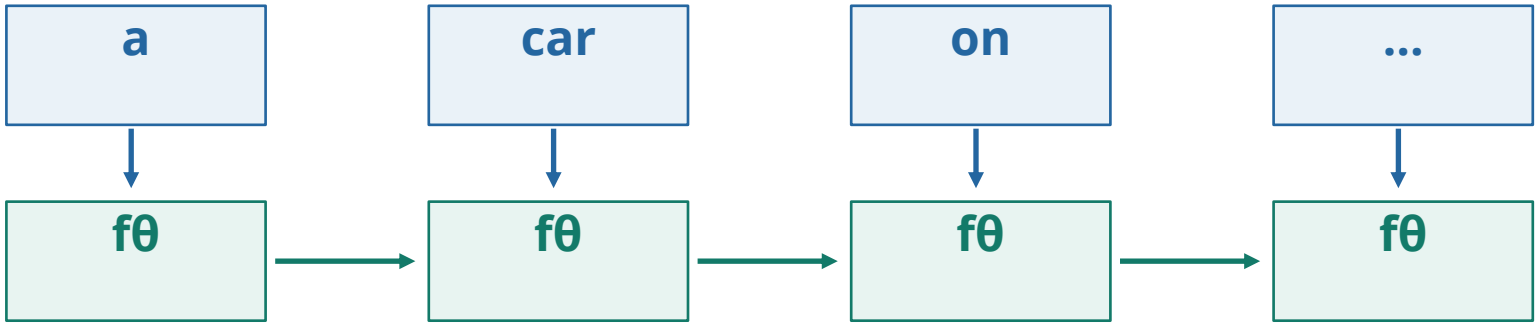


A new position would need another trained network.

Position-specific networks need more parameters as length grows.

RNN parameter sharing; Sutskever (2013). Teaching schematic.

One Update Rule, Any Number of Steps



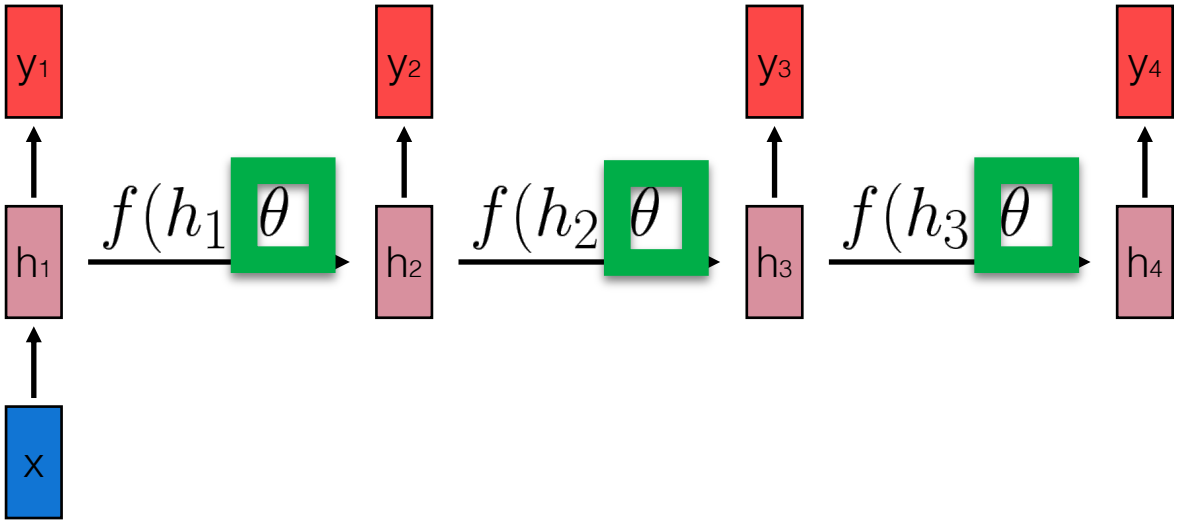
Same weights θ; changing input and previous state.

An RNN reuses a learned transition across time.

RNN parameter sharing; Sutskever (2013). Teaching schematic.

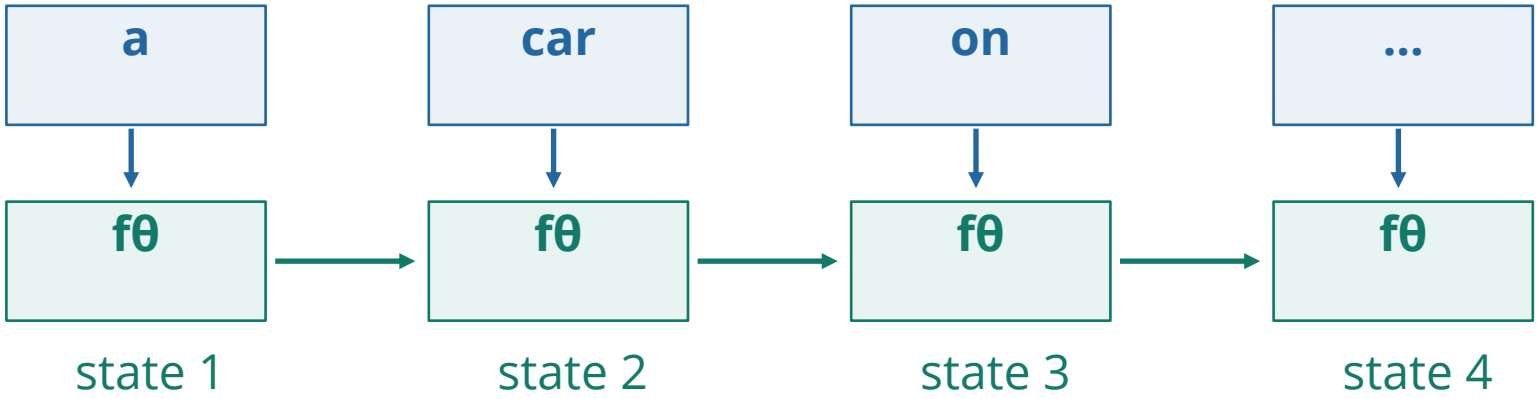
USC Viterbi School of Engineering

Recurrent Neural Network



Shared Weights, Different States

3 / 3



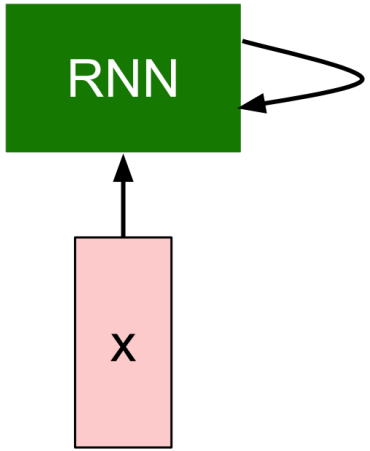
CNN: reuse a filter across space. RNN: reuse an update across time.

Sharing θ does not make the hidden states or predictions identical.

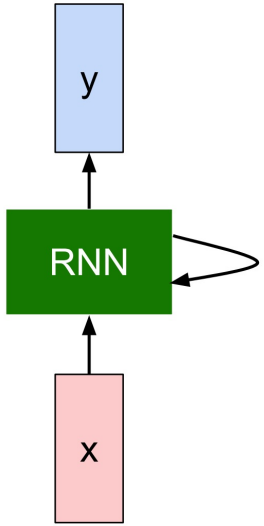
RNN parameter sharing; Sutskever (2013). Teaching schematic.

USC Viterbi School of Engineering

Recurrent Neural Network



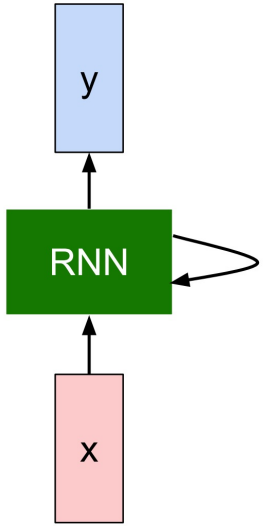
Recurrent Neural Network



Slide credit: Stanford CS231n

Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

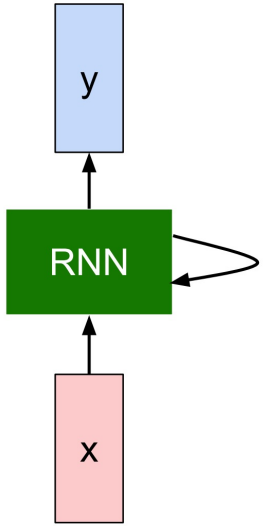


Slide credit: Stanford CS231n

Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$



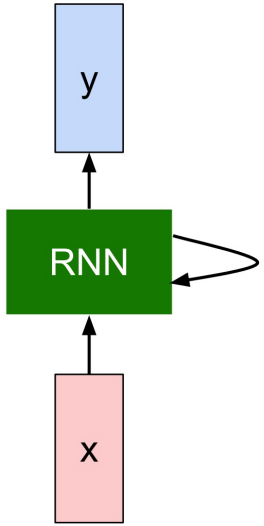
Slide credit: Stanford CS231n

Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

old state



Slide credit: Stanford CS231n

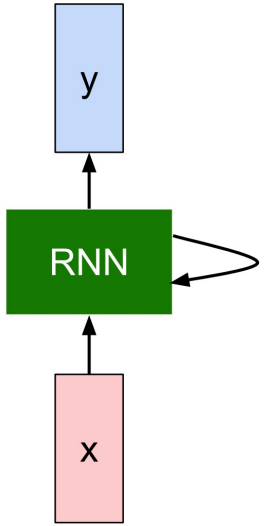
Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

old state

input vector at some time step



Slide credit: Stanford CS231n

Recurrent Neural Network

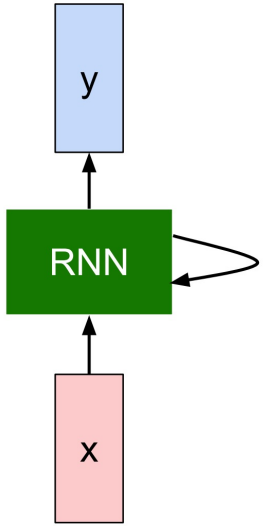
We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

old state

input vector at some time step

some function with parameters W



Slide credit: Stanford CS231n

Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

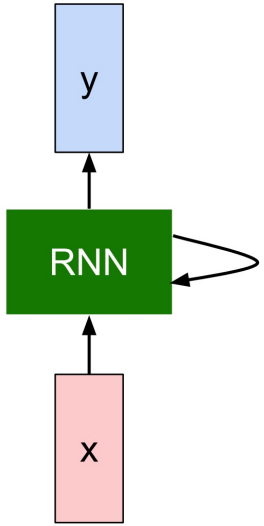
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

input vector at some time step



Recurrent Neural Network

We can process a sequence of vectors **x** by applying a **recurrence formula** at every time step:

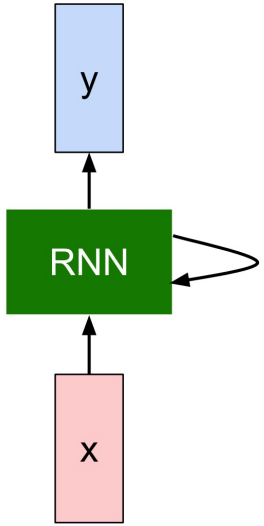
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

old state

input vector at some time step

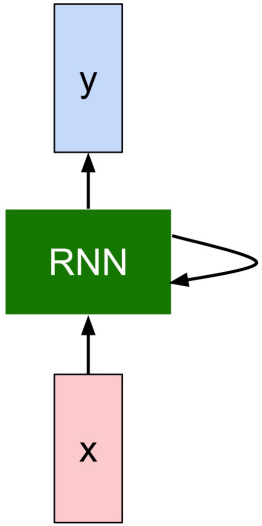


Note: the same function with the same parameters

Slide credit: Stanford CS231n

Recurrent Neural Network

$$h_t = f_W(h_{t-1}, x_t)$$



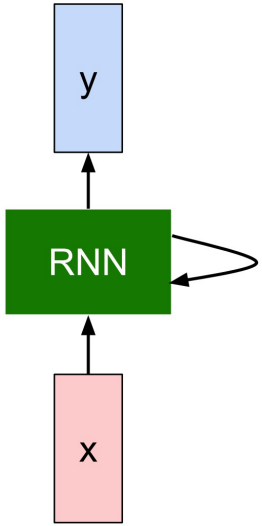
Slide credit: Stanford CS231n

(Vanilla) Recurrent Neural Network

$$h_t = f_W(h_{t-1}, x_t)$$

$$\downarrow$$

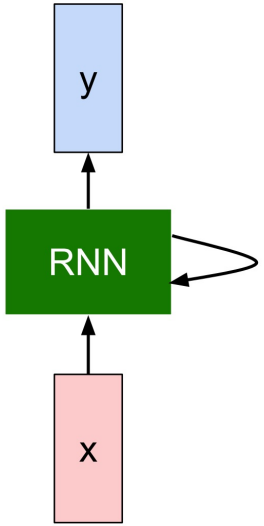
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



(Vanilla) Recurrent Neural Network

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



The state has a single “hidden” vector **h**

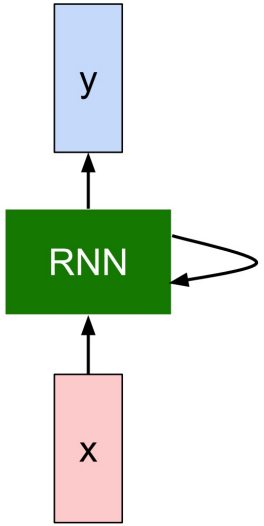
Slide credit: Stanford CS231n

(Vanilla) Recurrent Neural Network

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

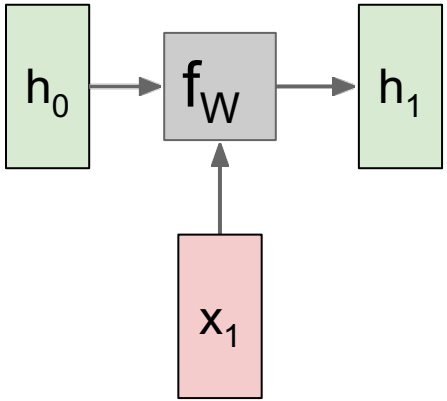
$$y_t = W_{hy}h_t$$



The state has a single “hidden” vector **h**

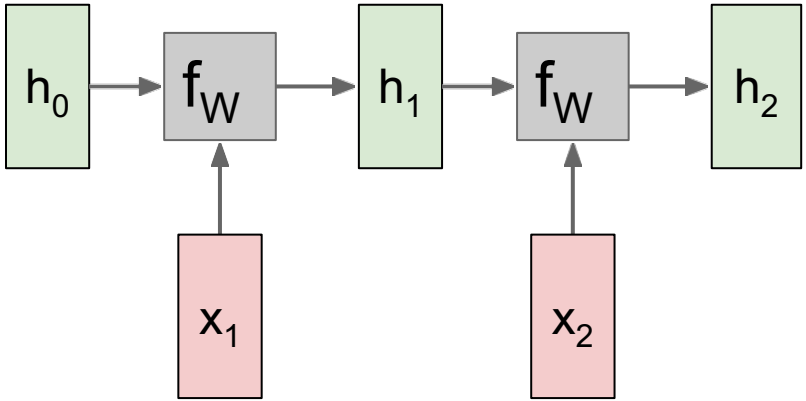
Slide credit: Stanford CS231n

RNN: Computational Graph



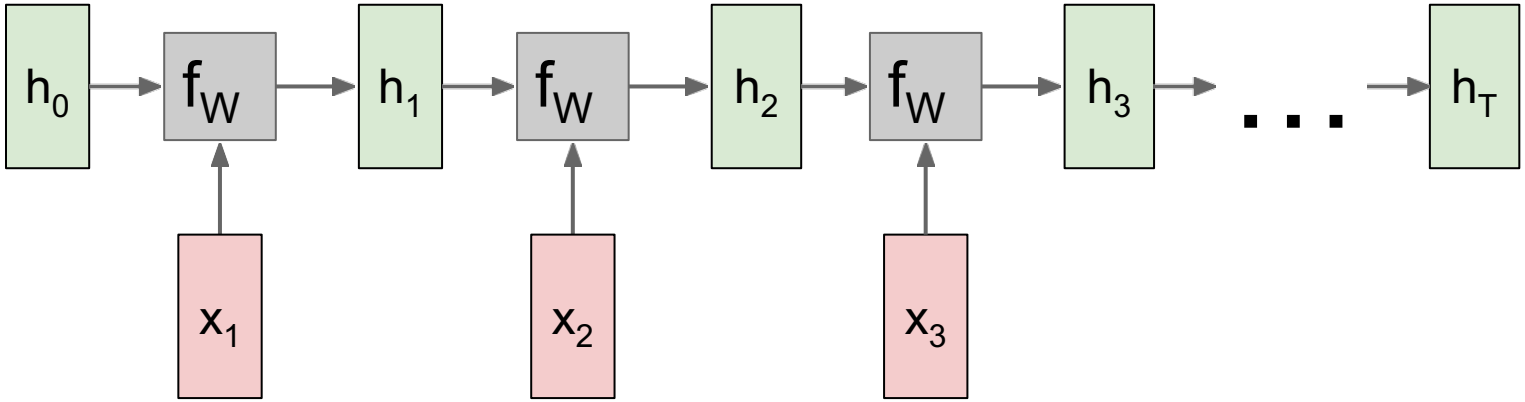
Credit to Stanford cs231n lecture slides

RNN: Computational Graph



Credit to Stanford cs231n lecture slides

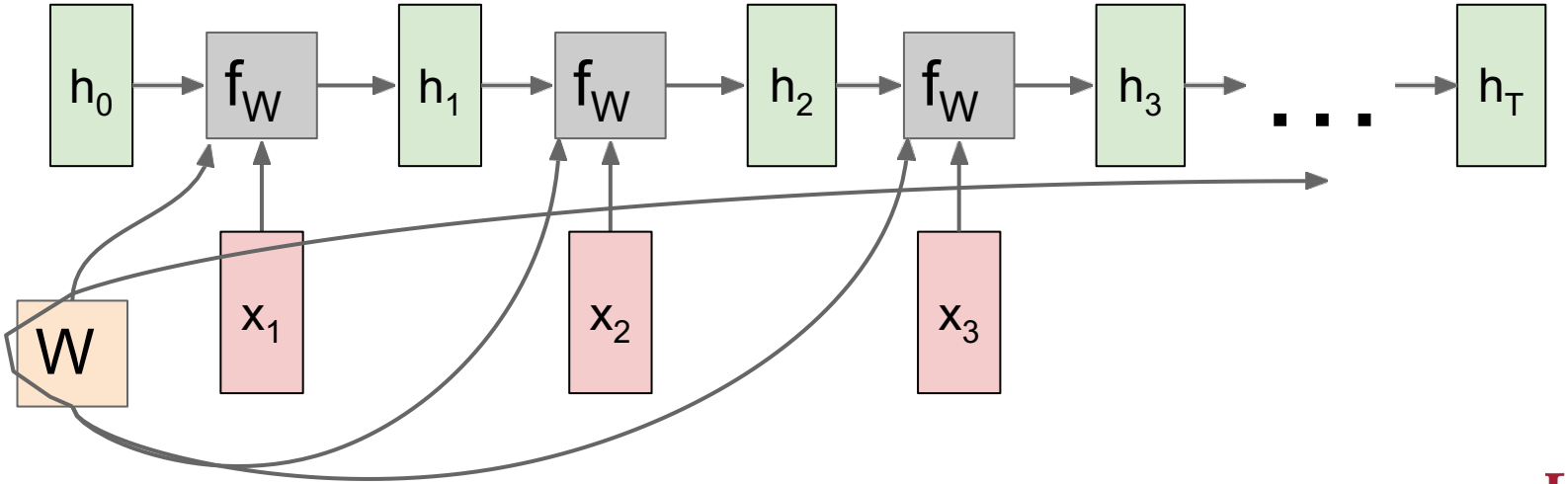
RNN: Computational Graph



Credit to Stanford cs231n lecture slides

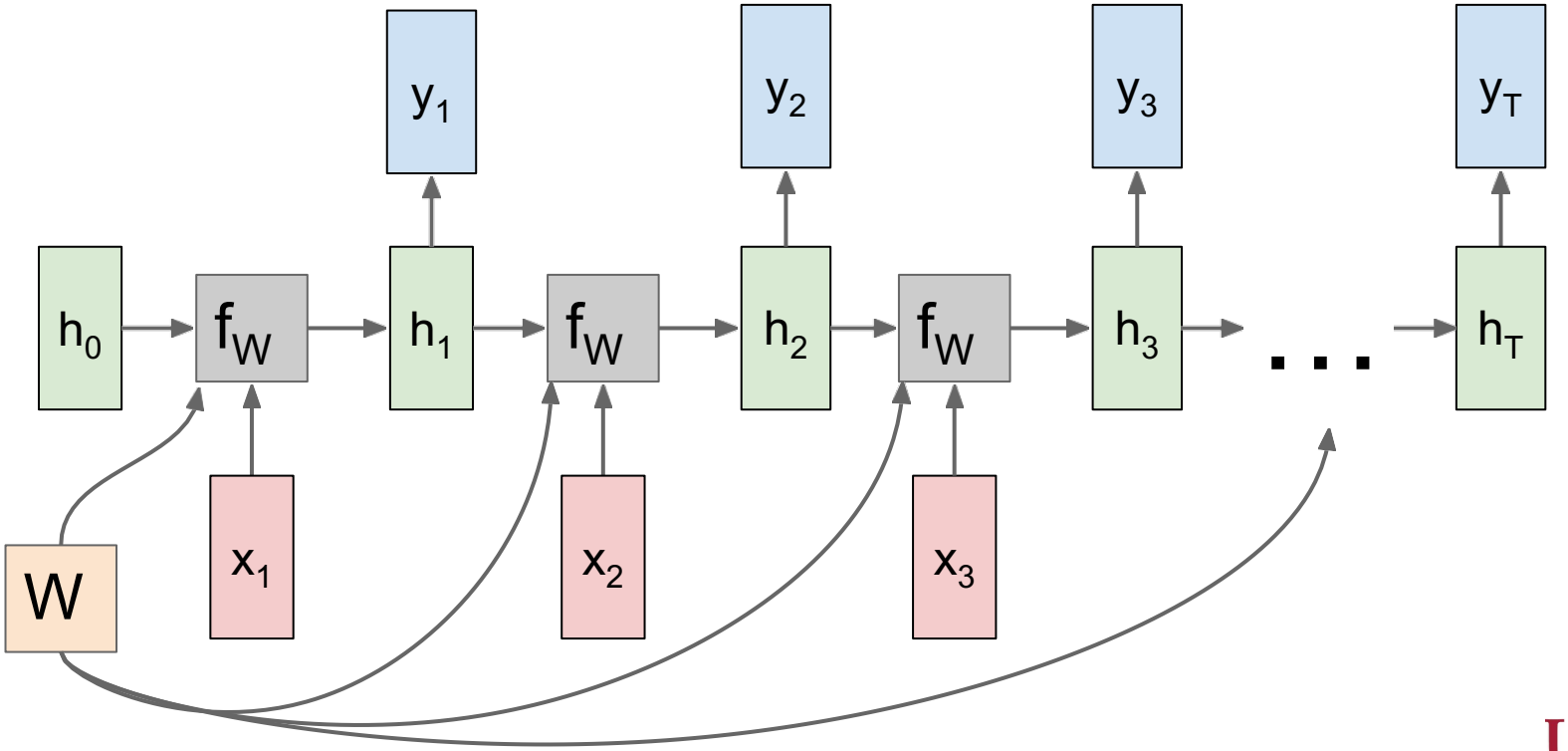
RNN: Computational Graph

Re-use the **same** weight matrix at every time-step



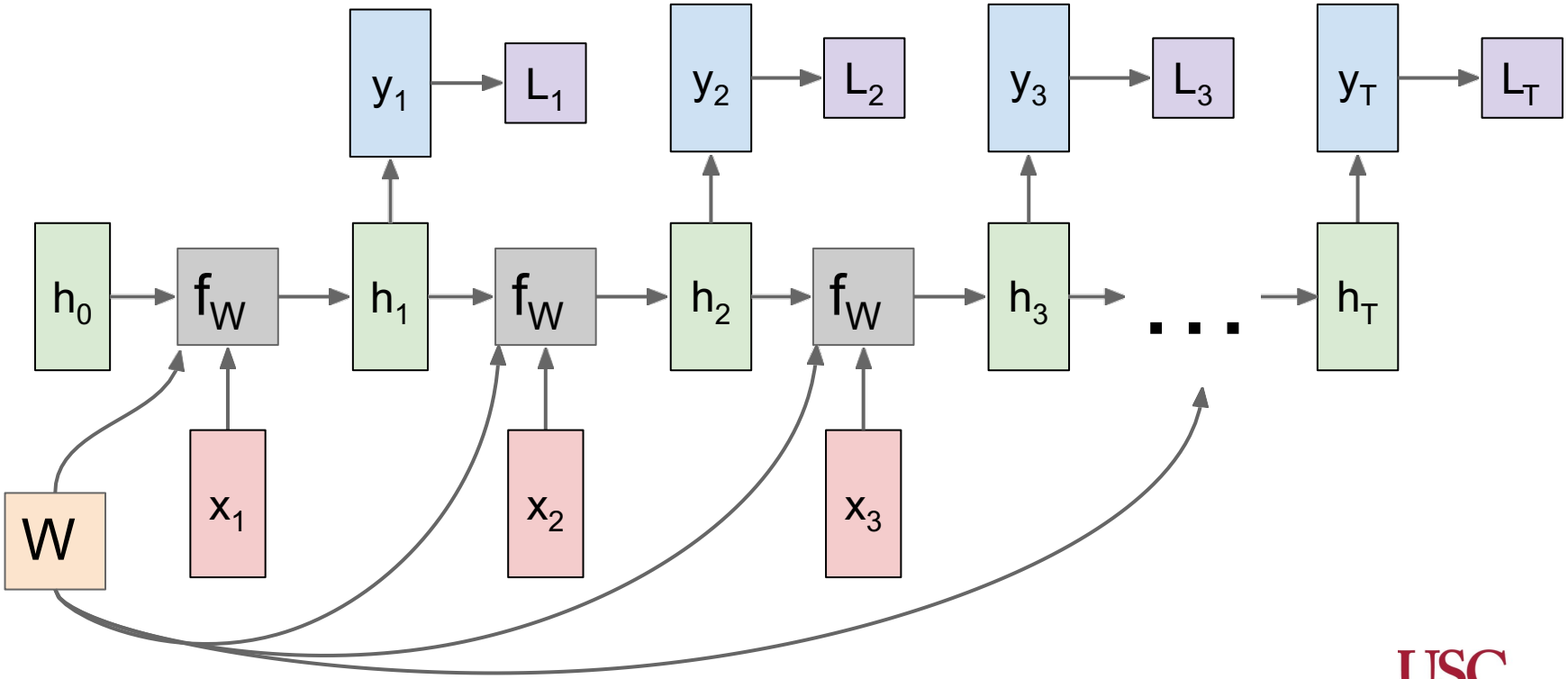
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: Many to Many



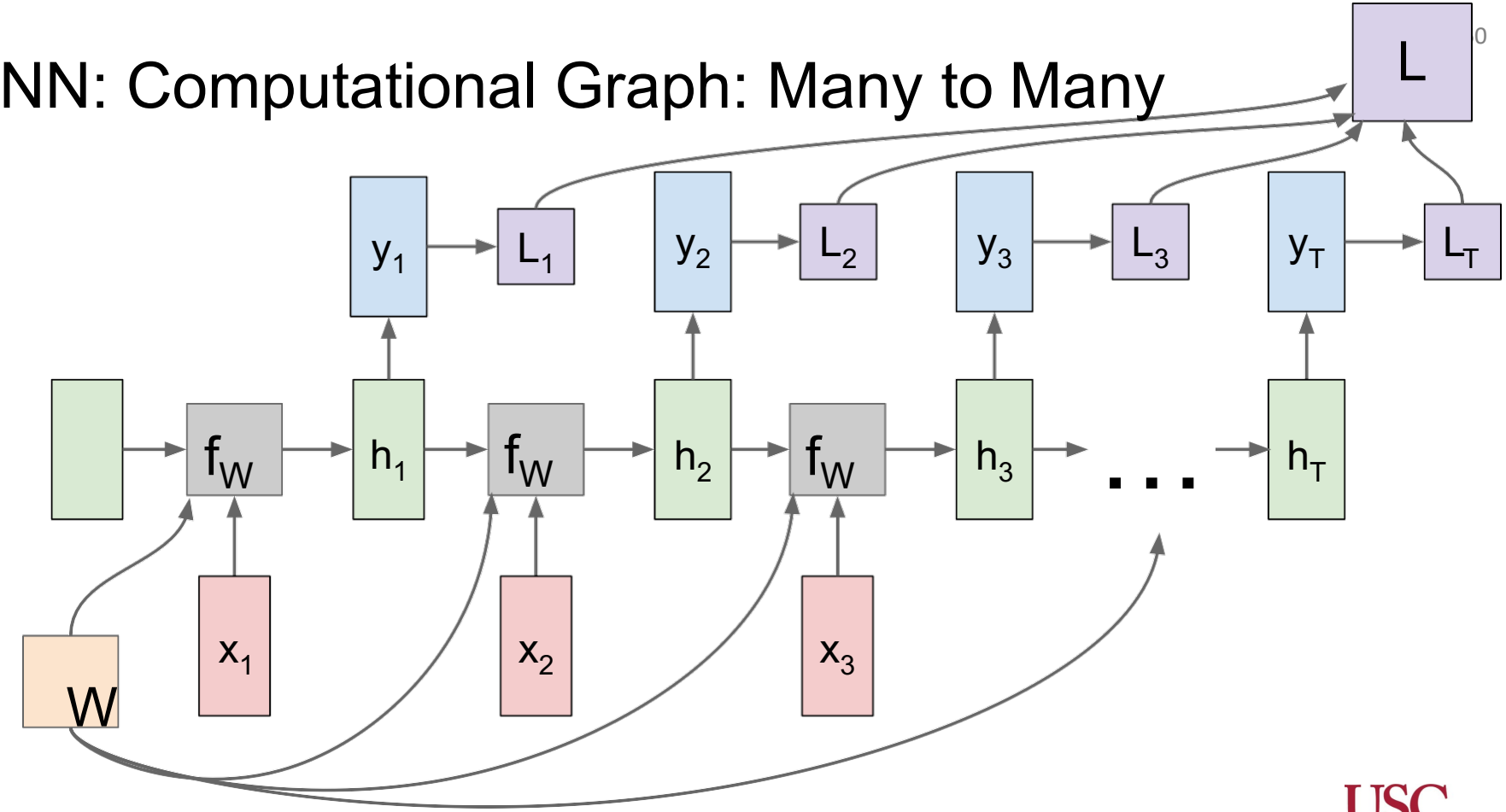
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: Many to Many



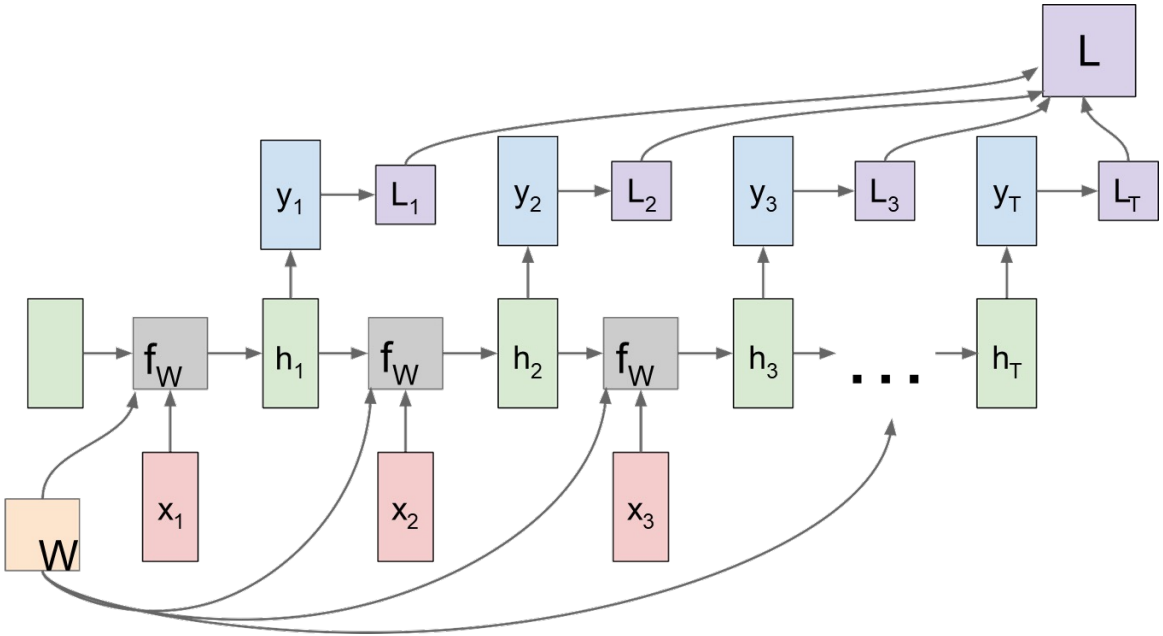
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: Many to Many



Credit to Stanford cs231n lecture slides

USC Viterbi School of Engineering



f_W: This is the function that calculates the new hidden state

$y_1, y_2, y_3, \ldots y_t$: These are the outputs at each time step,

$L_1, L_2, L_3, \ldots L_t$: These are the loss values at each time step, calculated by comparing the output at each time step with the true value

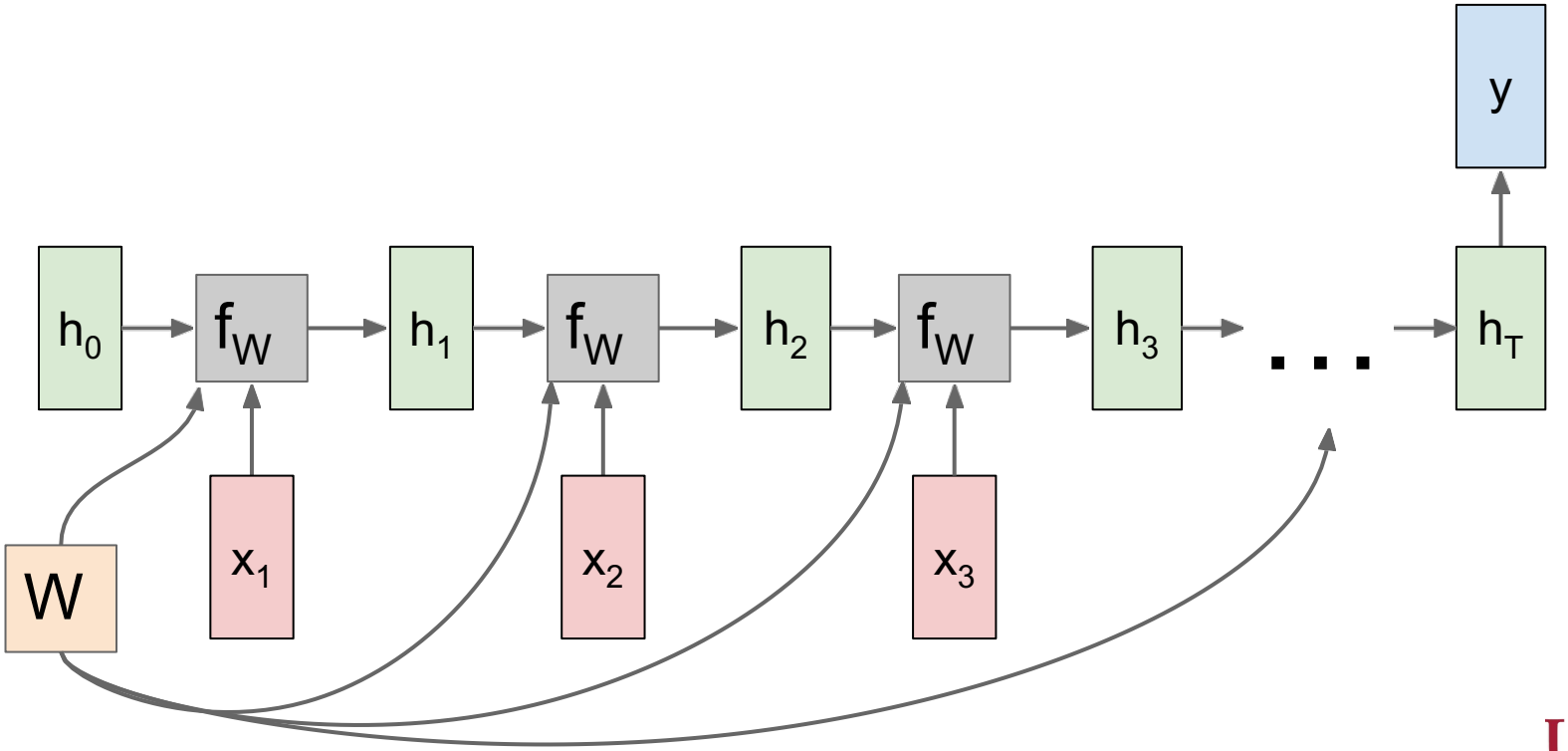
$x_1, x_2, x_3, \ldots x_t$: These are the input data points at each time step

$h_1, h_2, h_3, \ldots h_t$: These are the hidden states at each time step

W: This represents the **shared** weights that are learned during training and are applied at each time step to compute the hidden state

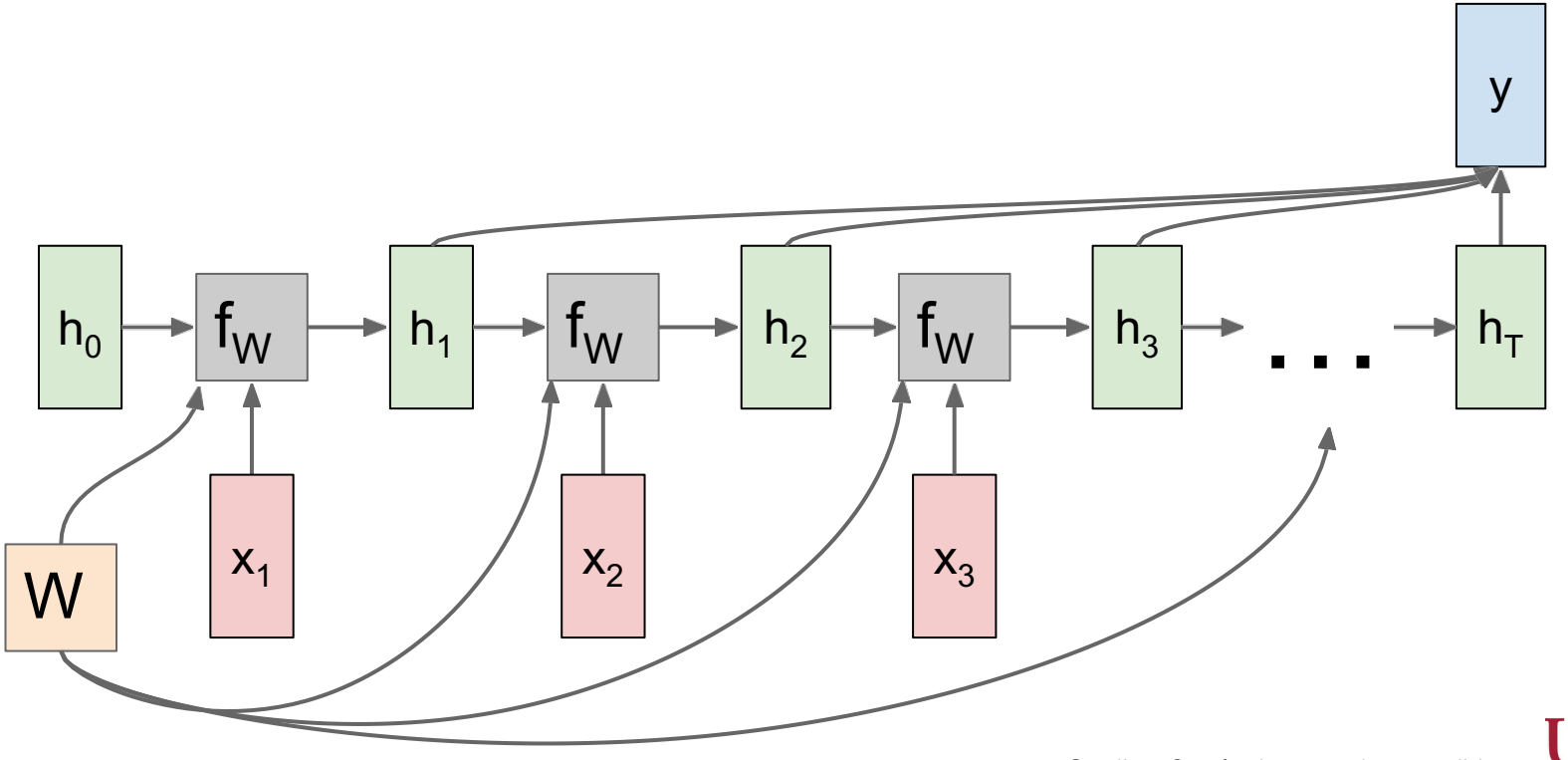
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: Many to One



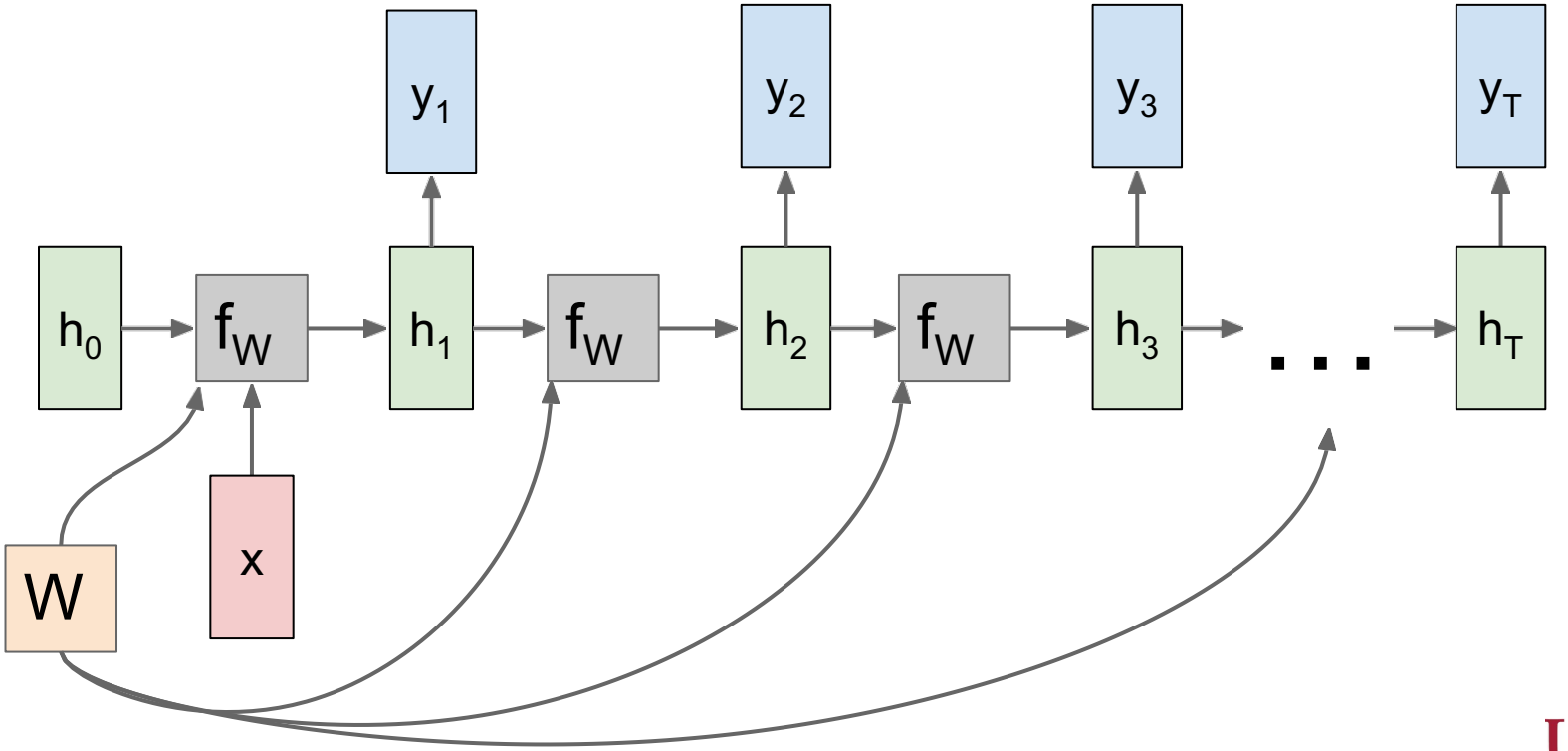
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: Many to One



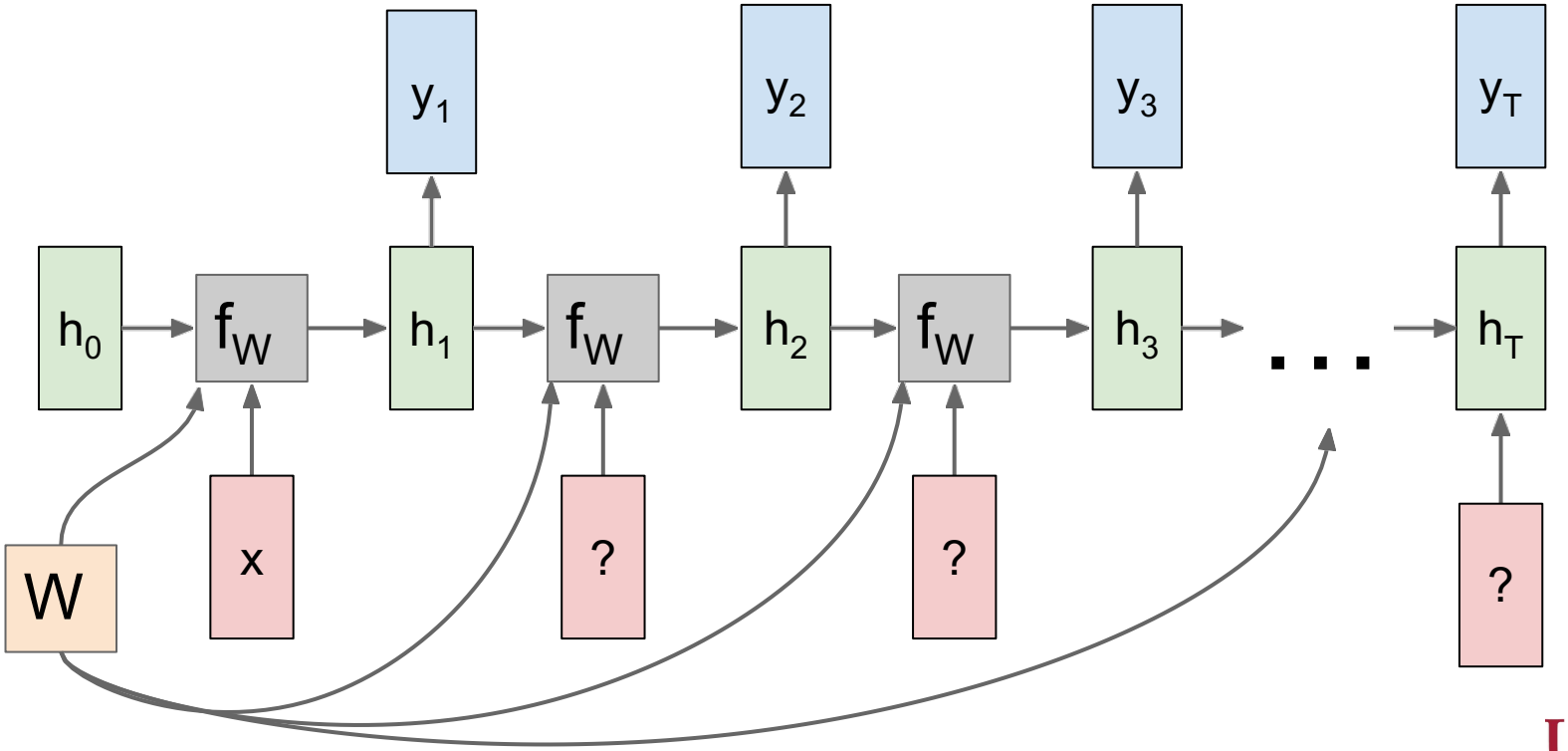
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: One to Many



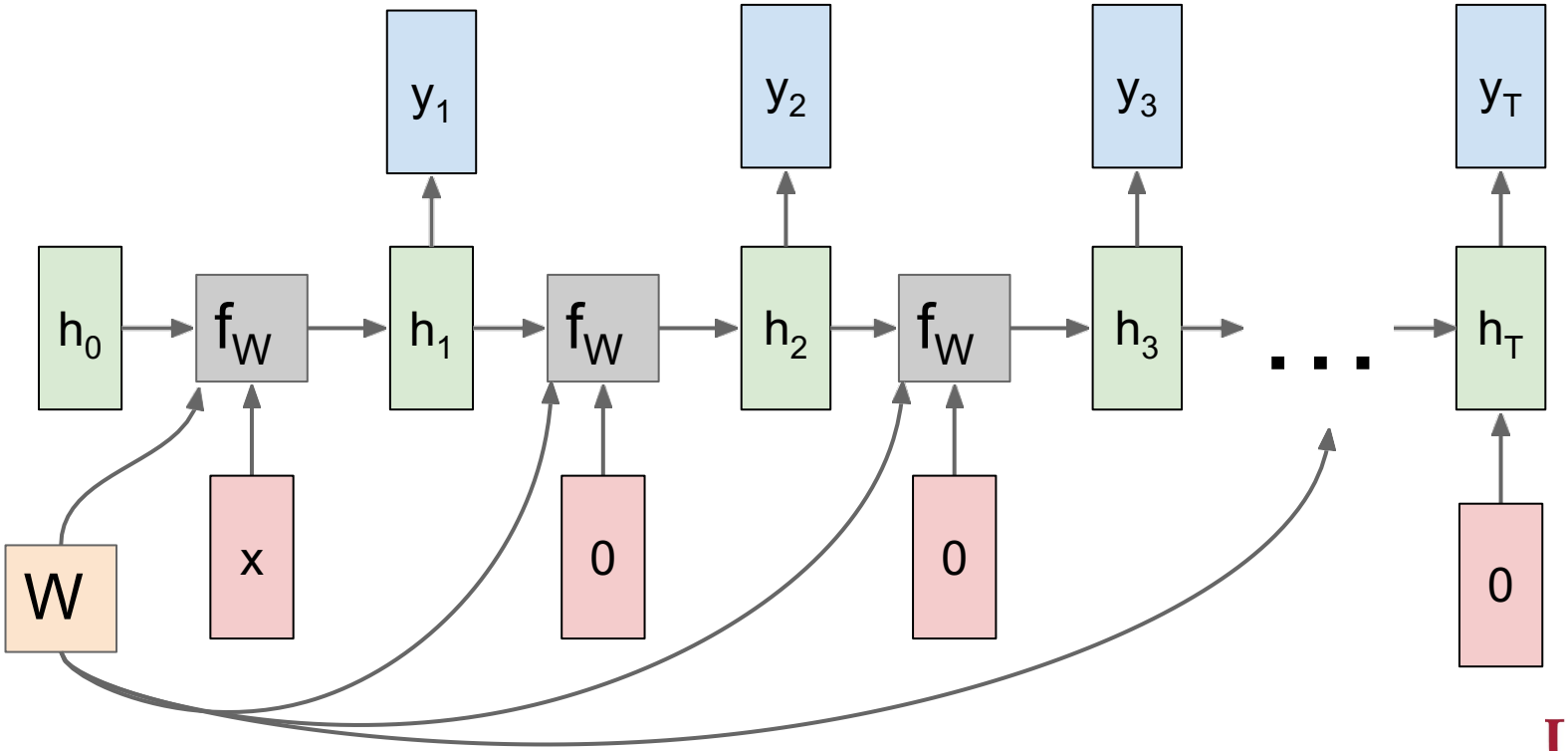
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: One to Many



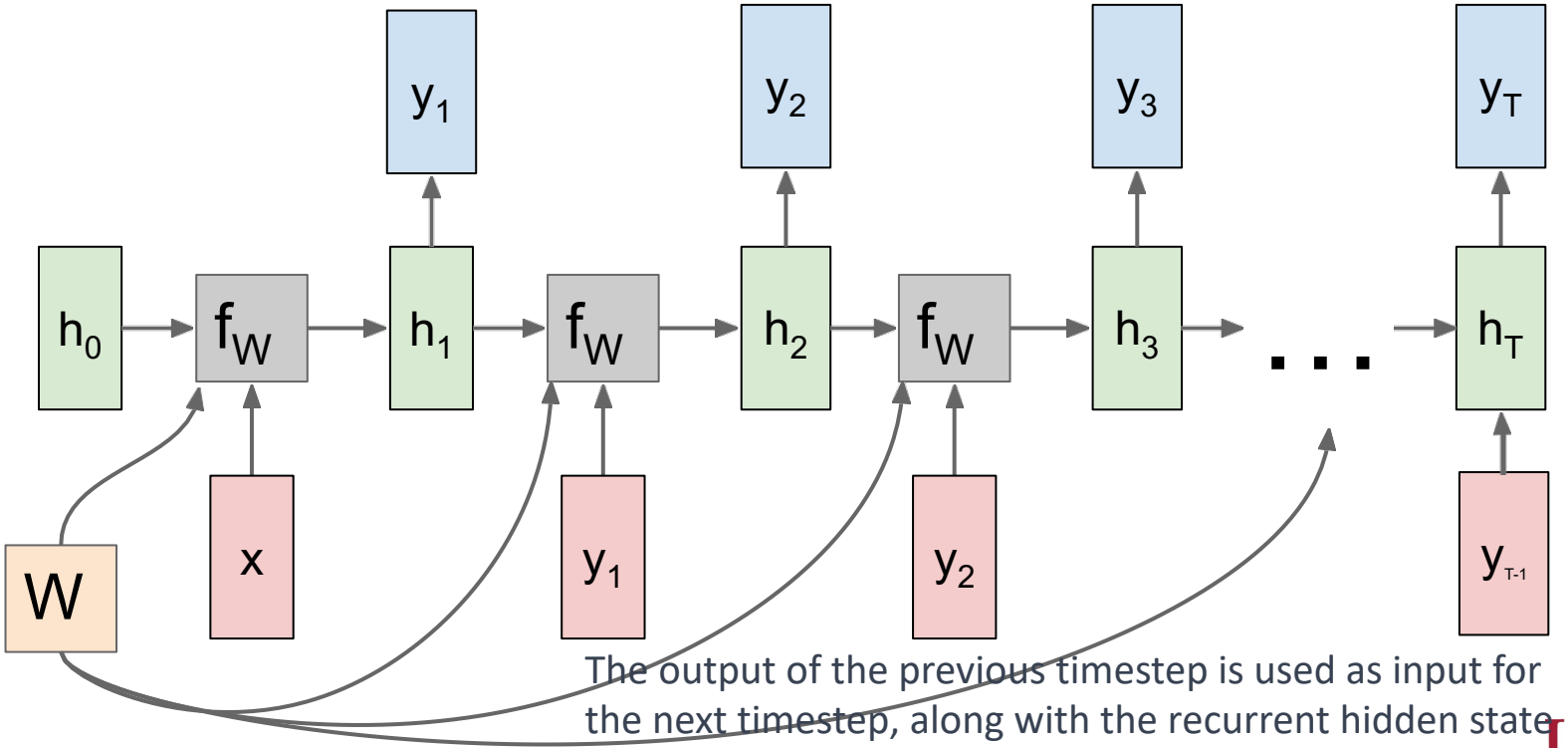
Credit to Stanford cs231n lecture slides

RNN: Computational Graph: One to Many



Credit to Stanford cs231n lecture slides

RNN: Computational Graph: One to Many

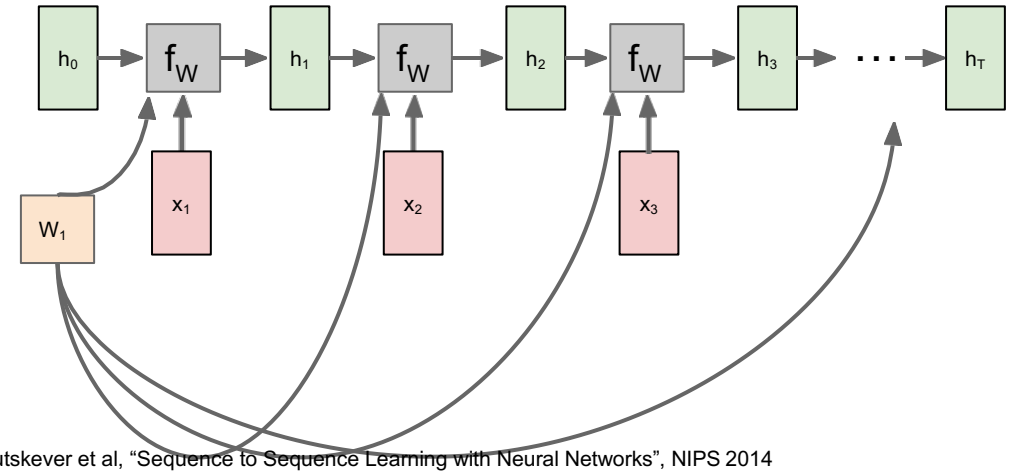


The output of the previous timestep is used as input for the next timestep, along with the recurrent hidden state

Credit to Stanford cs231n lecture slides

Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector



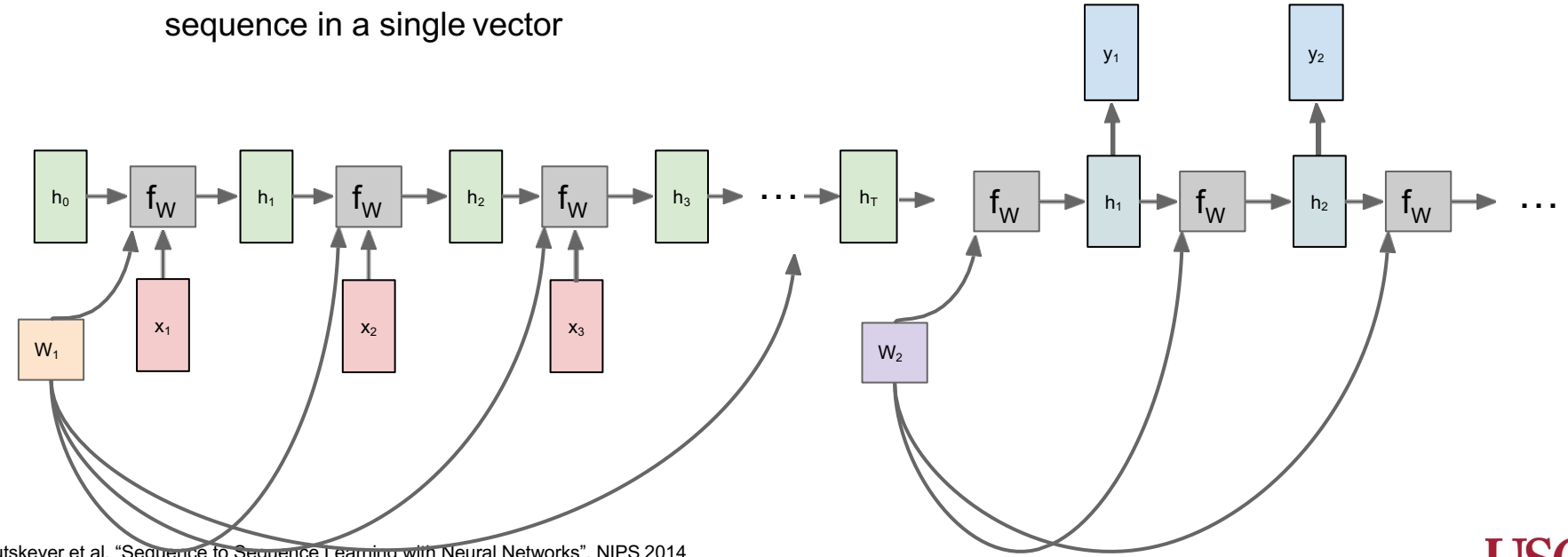
Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Credit to Stanford cs231n lecture slides

Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

One to many: Produce output sequence from single input vector



Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Credit to Stanford cs231n lecture slides

The Task: Predict the Next Character

Training text: hello

Prefix read so far	Current input	Next-character target
h	h	e
he	e	l
hel	**l**	**l**
hell	**l**	**o**

Same input l, different history: the hidden state carries context.

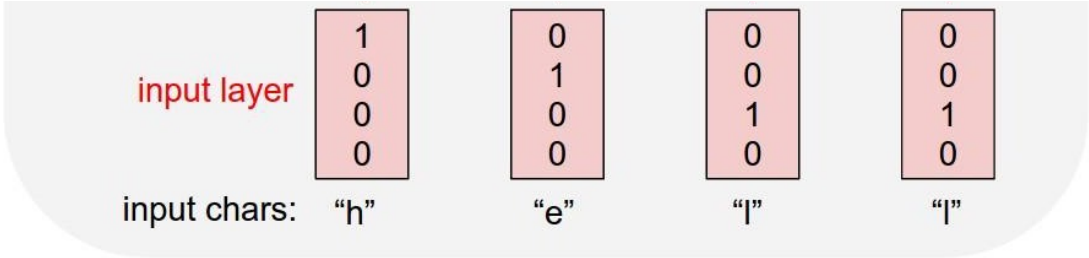
Teaching example: hello. Detailed diagram follows (Stanford CS231n).

USC Viterbi School of Engineering

Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training sequence:
“hello”

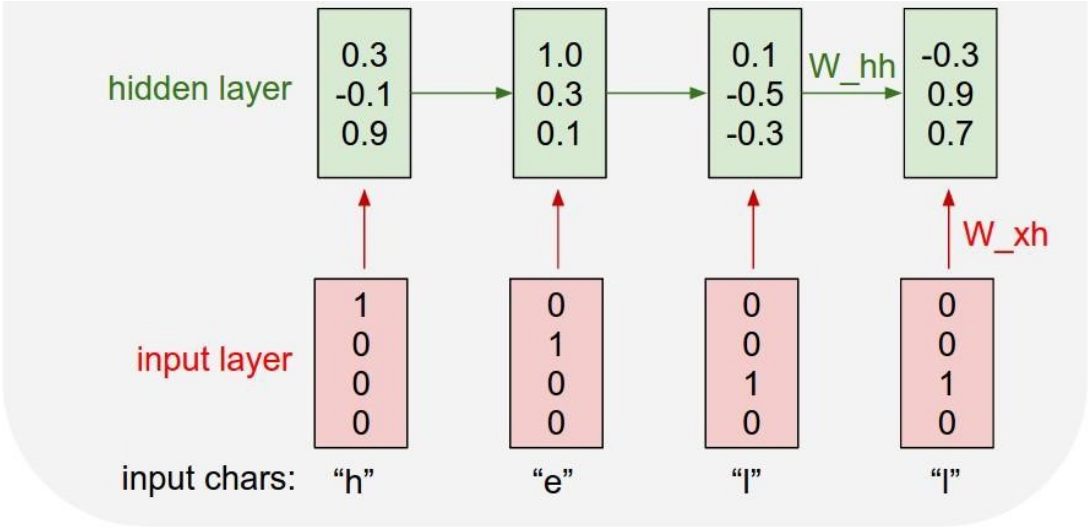


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training sequence:
“hello”

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

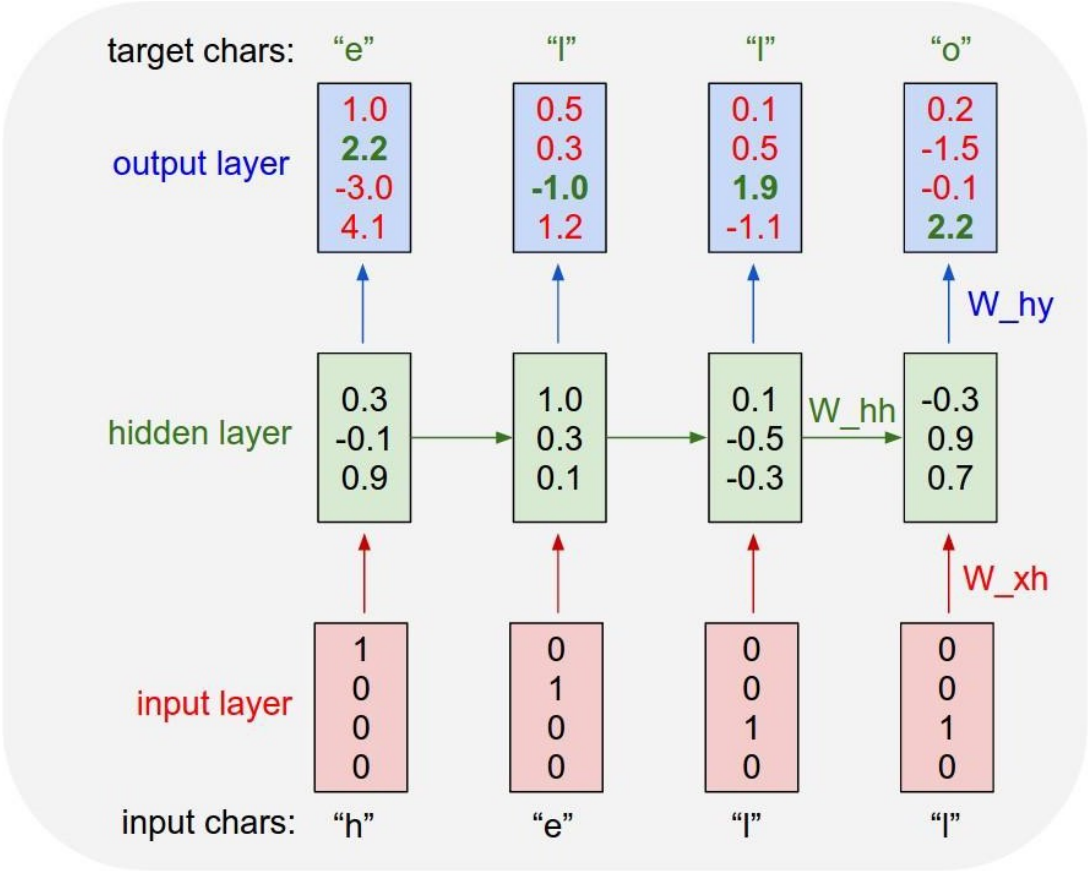


Credit to Stanford cs231n lecture slides

Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training sequence:
“hello”

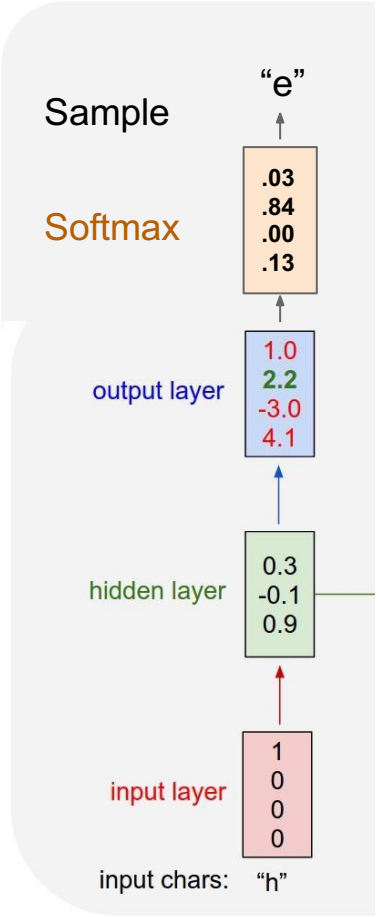


Credit to Stanford cs231n lecture slides

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

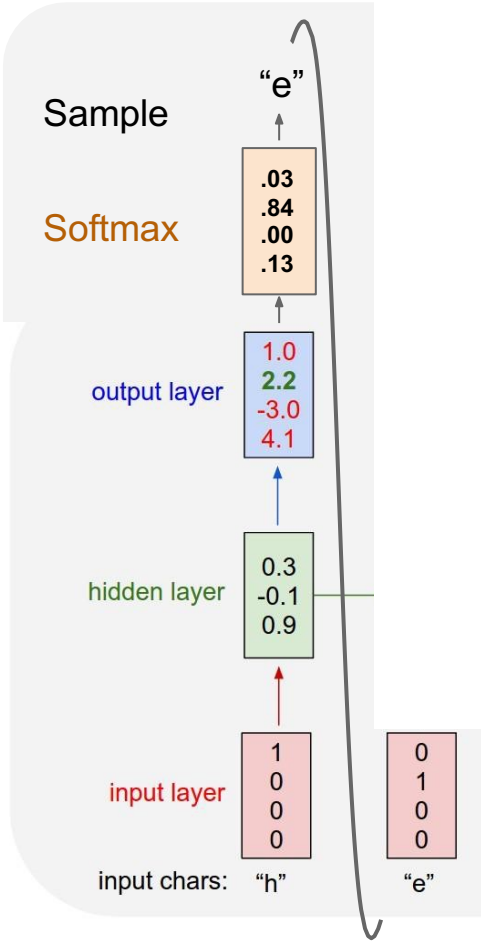
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

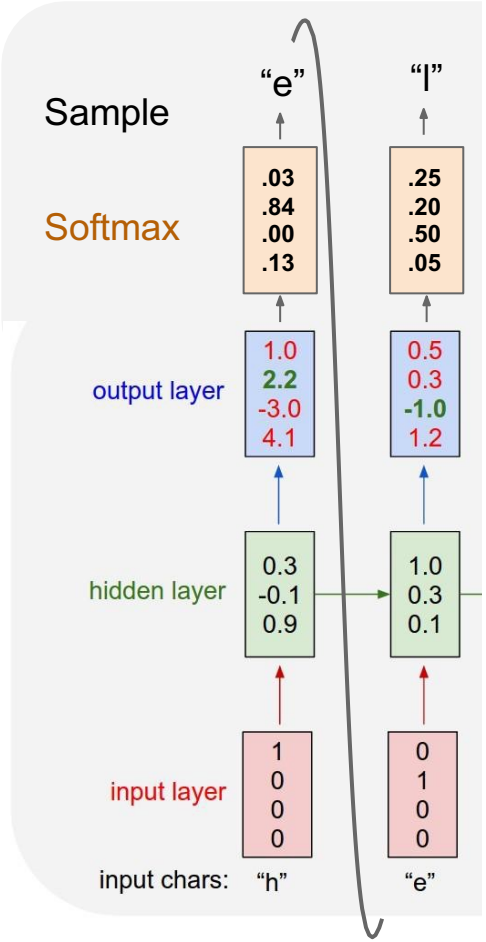
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

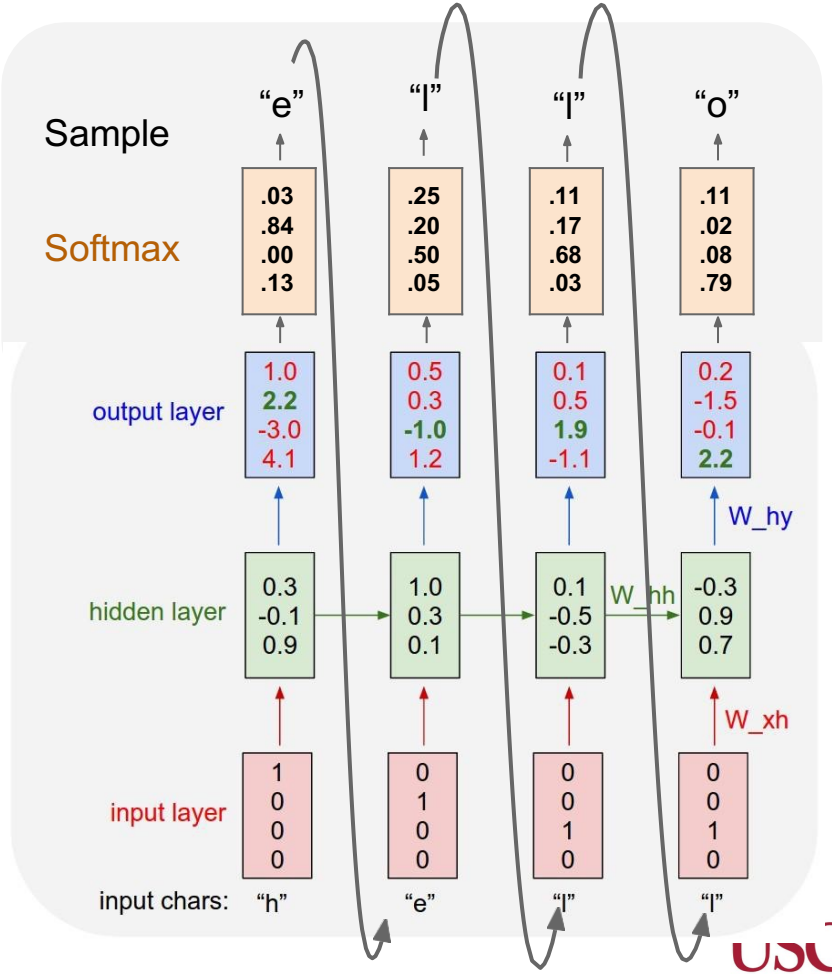
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

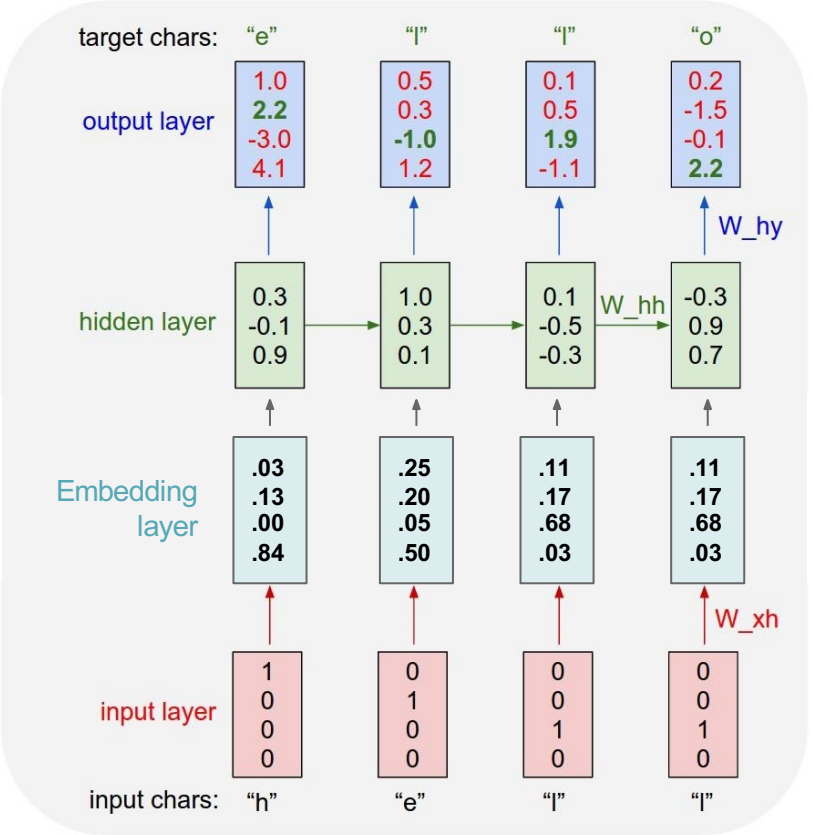
At test-time sample characters one at a time, feed back to model



Example: Character-level Language Model Sampling

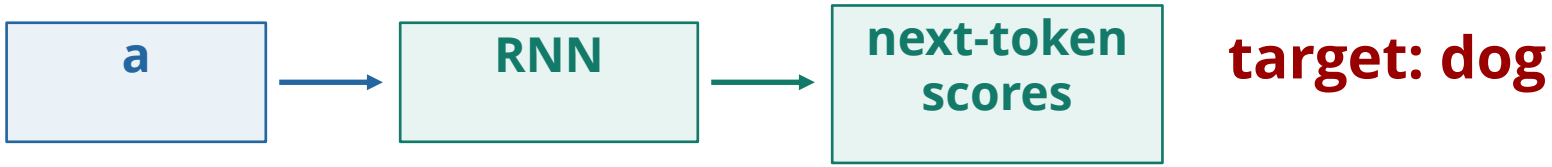
$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{14} \\ w_{31} & w_{32} & w_{33} & w_{14} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} w_{11} \\ w_{21} \\ w_{31} \end{bmatrix}$$

Matrix multiply with a one-hot vector just extracts a column from the weight matrix. We often put a separate **embedding** layer between input and hidden layers.

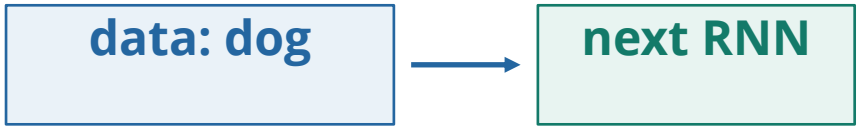


Training Supplies the Next Input

TRAINING EXAMPLE: a dog runs



Next input comes from:

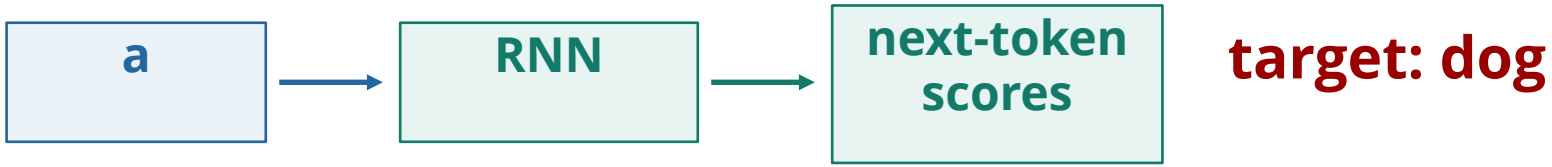


Training: score against dog, then read the observed dog.

Course schematic.

A Prediction Does Not Replace Training Data

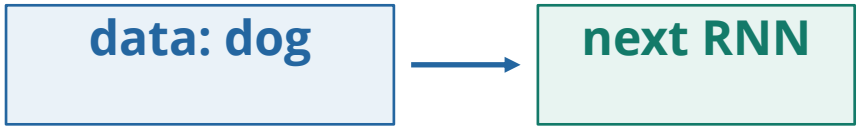
TRAINING EXAMPLE: a dog runs



target: dog

Suppose top prediction = cat

Next input comes from:



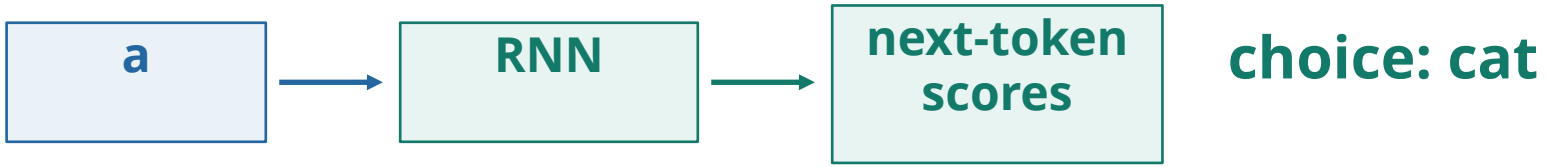
Loss uses the target dog; the next input is still dog.

Course schematic.

USC Viterbi School of Engineering

Generation Feeds Its Choice Back In

GENERATION: prompt a; the model chooses cat



Next input comes from:



Generation feeds back its choice, even when that choice is wrong.

Course schematic.

Fluent Text Still Has to Fit the Task

Free writing: continue "a ..."

"a dog runs" and "a cat runs" can both be valid.

Describe a photo that shows a dog

Calling it a cat is wrong, even in a fluent sentence.

Feedback into the model explains generation, not correctness.

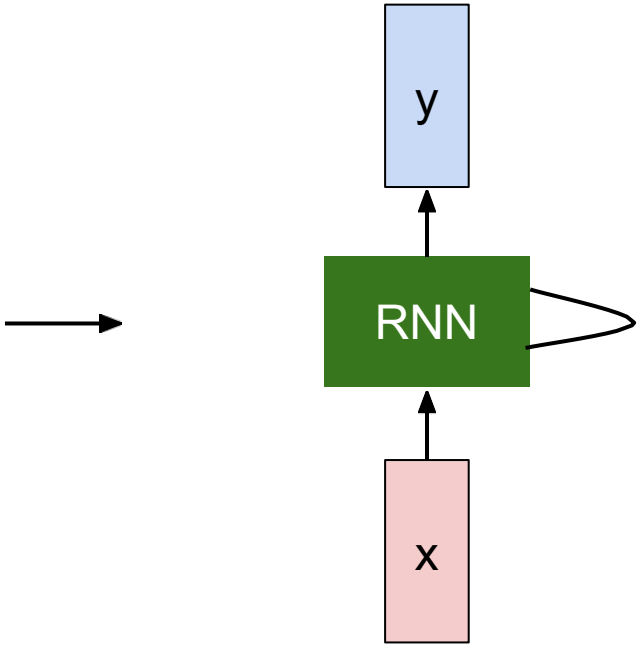
Course schematic.

THE SONNETS

by William Shakespeare

From fairest creatures we desire increase,
That thereby beauty's rose might never die,
But as the riper should by time decease,
His tender heir might bear his memory:
But thou, contracted to thine own bright eyes,
Feed'st thy light's flame with self-substantial fuel,
Making a famine where abundance lies,
Thyself thy foe, to thy sweet self too cruel:
Thou that art now the world's fresh ornament,
And only herald to the gaudy spring,
Within thine own bud buriest thy content,
And tender churl mak'st waste in niggarding:
    Pity the world, or else this glutton be,
    To eat the world's due, by the grave and thee.

When forty winters shall besiege thy brow,
And dig deep trenches in thy beauty's field,
Thy youth's proud livery so gazed on now,
Will be a tatter'd weed of small worth held:
Then being asked, where all thy beauty lies,
Where all the treasure of thy lusty days;
To say, within thine own deep sunken eyes,
Were an all-eating shame, and thriftless praise.
How much more praise deserv'd thy beauty's use,
If thou couldst answer 'This fair child of mine
Shall sum my count, and make my old excuse,'
Proving his beauty by succession thine!
    This were to be new made when thou art old,
    And see thy blood warm when thou feel'st it cold.



at first:

```
tyntd-iafhatawiaoihrdemot  lytdws  e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgd t o idoe ns,smtt   h ne etie h,hregtrs nigtike,aoaenns lng
```

train more

```
"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```

train more

```
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.
```

train more

```
"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.
```

```
PANDARUS:
Alas, I think he shall be come approached and the day
When little srain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:
They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:
Well, your wit is in the care of side and that.

Second Lord:
They would be ruled after this chamber, and
my fair nues begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Clown:
Come, sir, I will make did behold your worship.

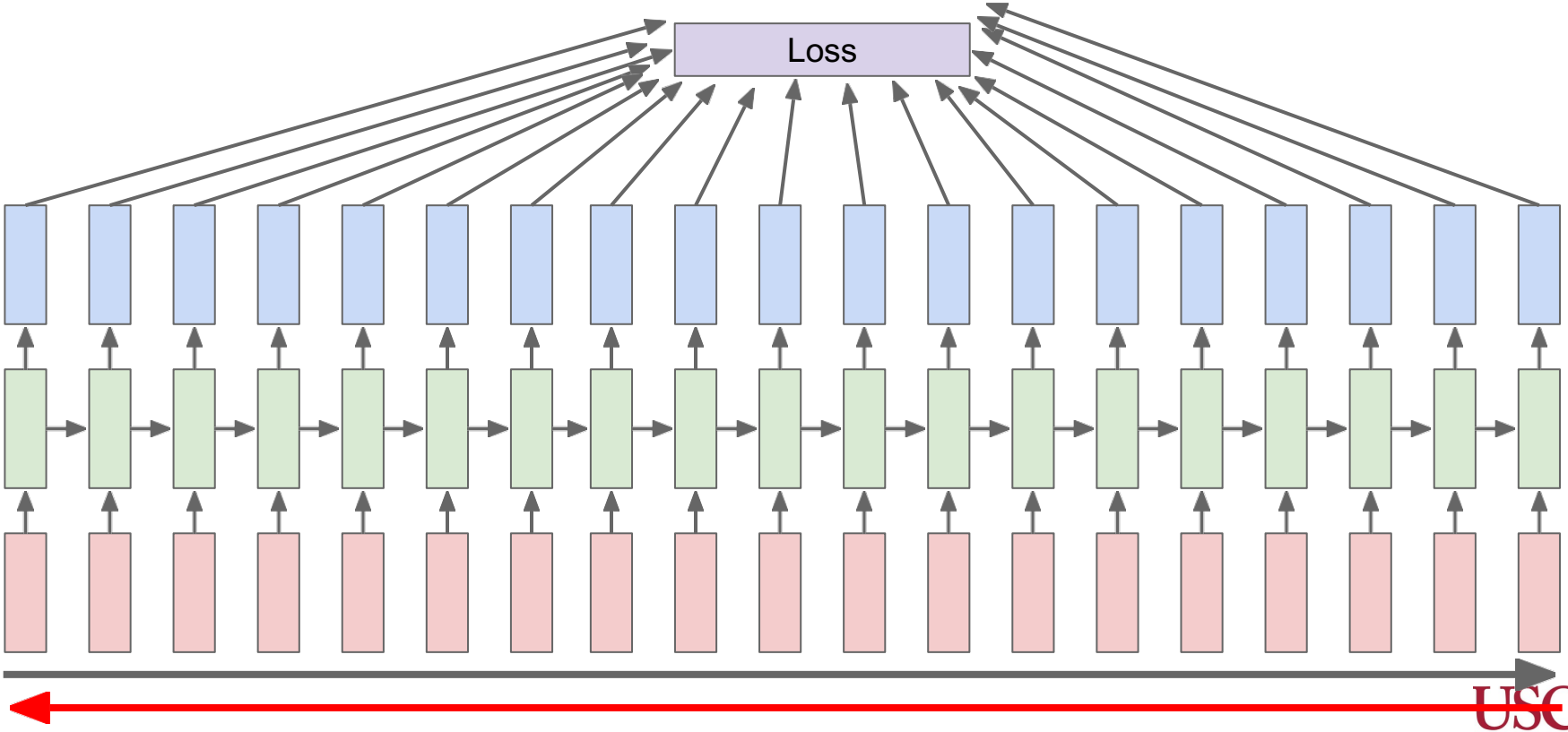
VIOLA:
I'll drink it.
```

```
VIOLA:
Why, Salisbury must find his flesh and thought
That which I am not aps, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

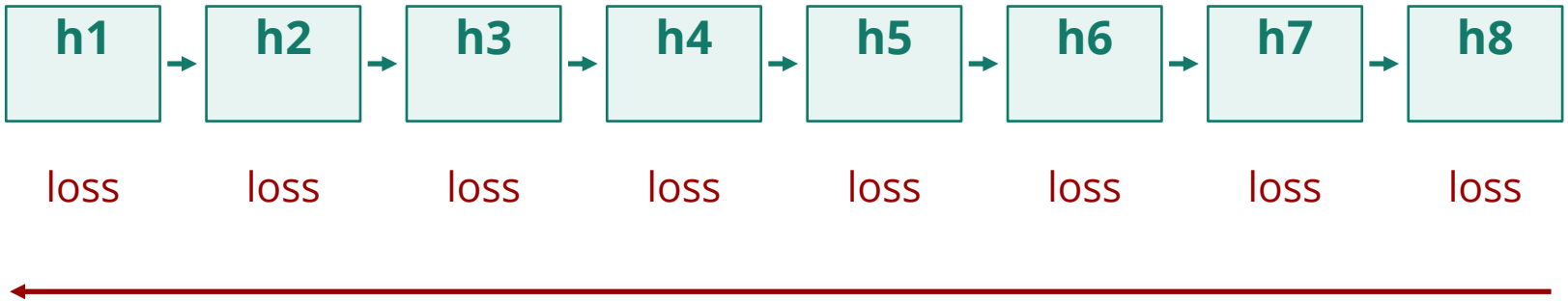
KING LEAR:
O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.
```

Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Long Sequences Make Full BPTT Expensive

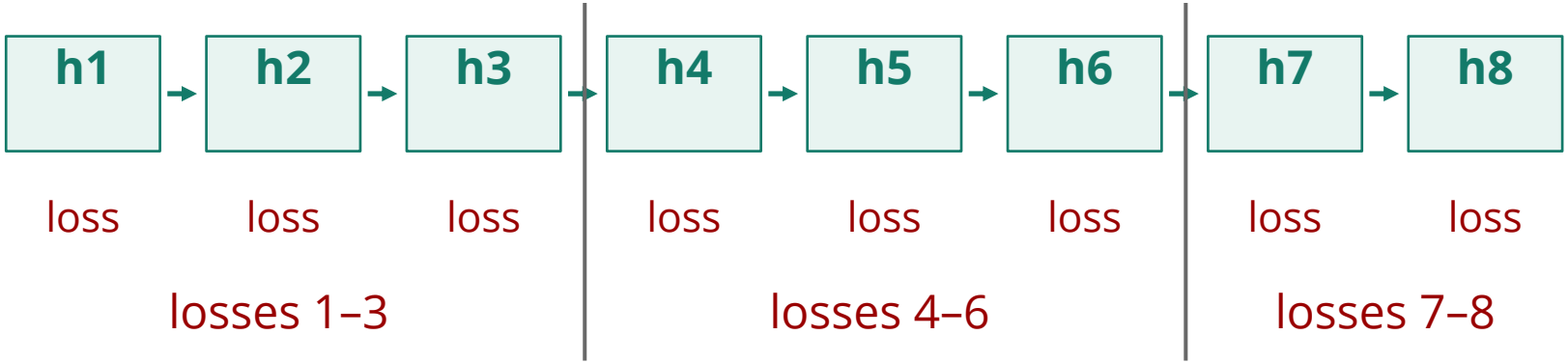


Save intermediate activations across the whole sequence.

A longer backward path needs more memory before an update.

Truncated BPTT; Sutskever (2013). Non-overlapping chunks shown.

Train on Shorter Spans, Keep the Token Targets

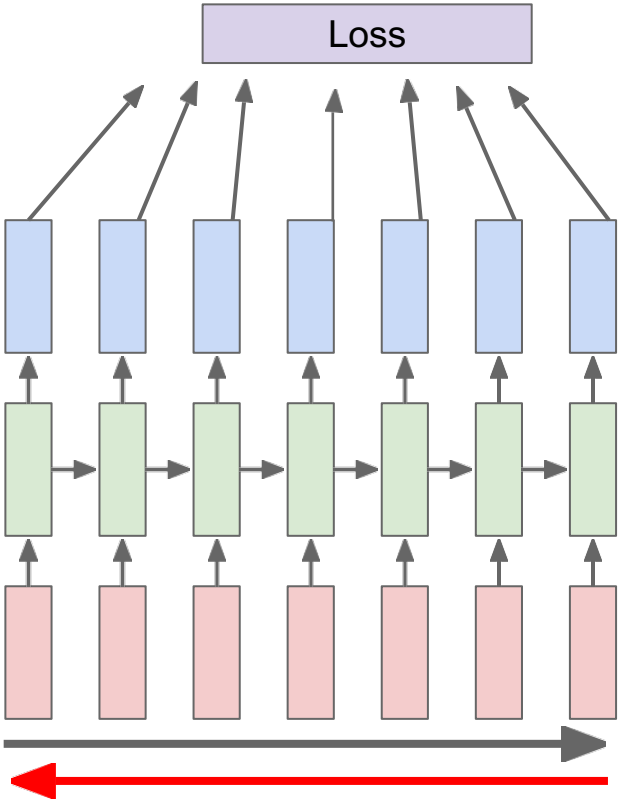


For each chunk: sum/average its losses, backpropagate, update θ.

The targets stay the same; the gradient history is truncated.

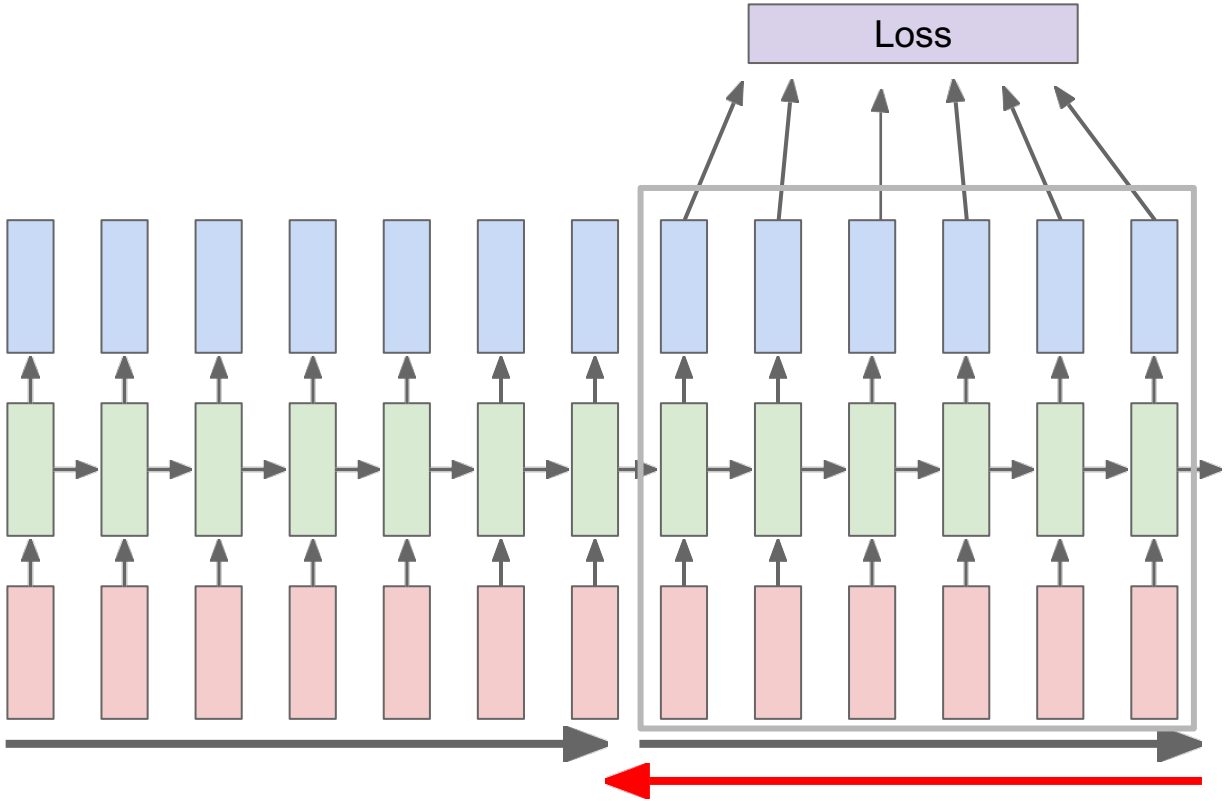
Truncated BPTT; Sutskever (2013). Non-overlapping chunks shown.

Truncated Backpropagation through time



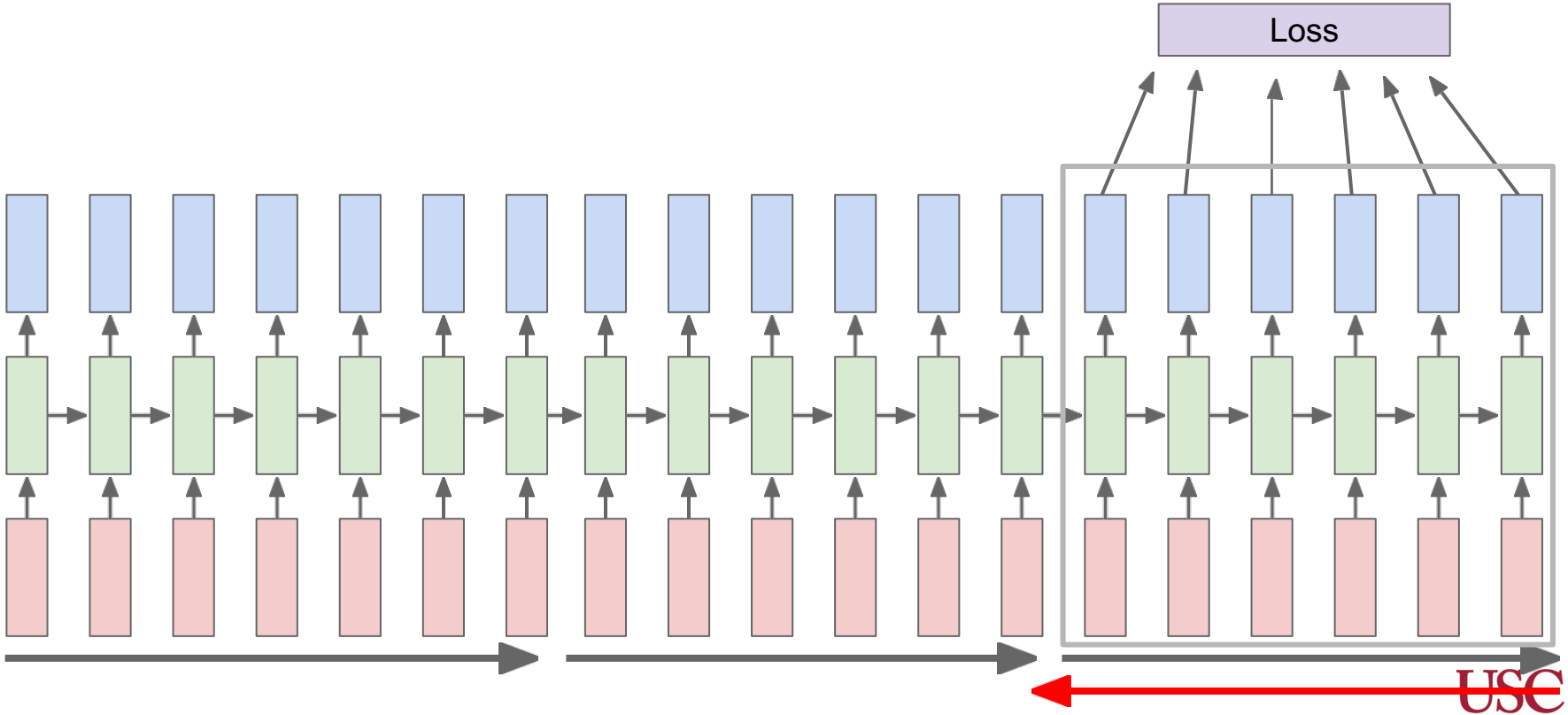
Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation through time

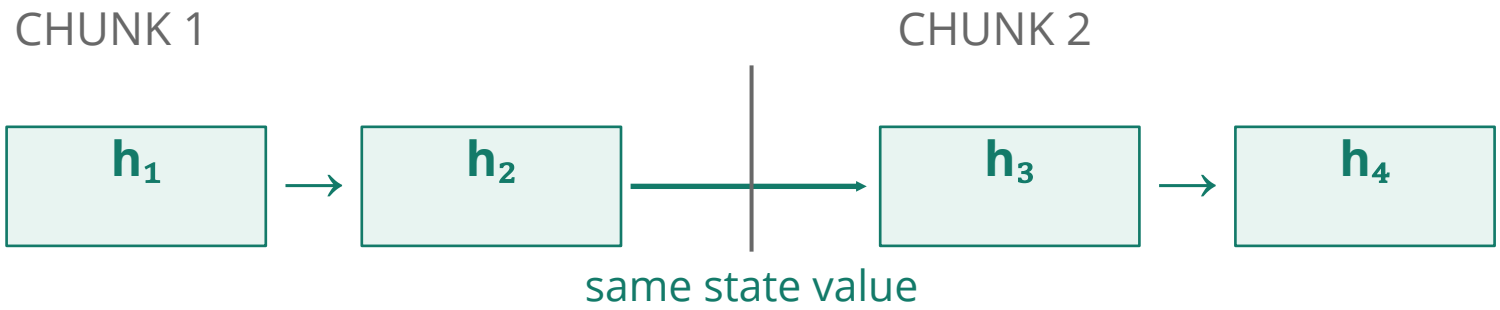


Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Truncated Backpropagation through time



Carry the State Across Chunks



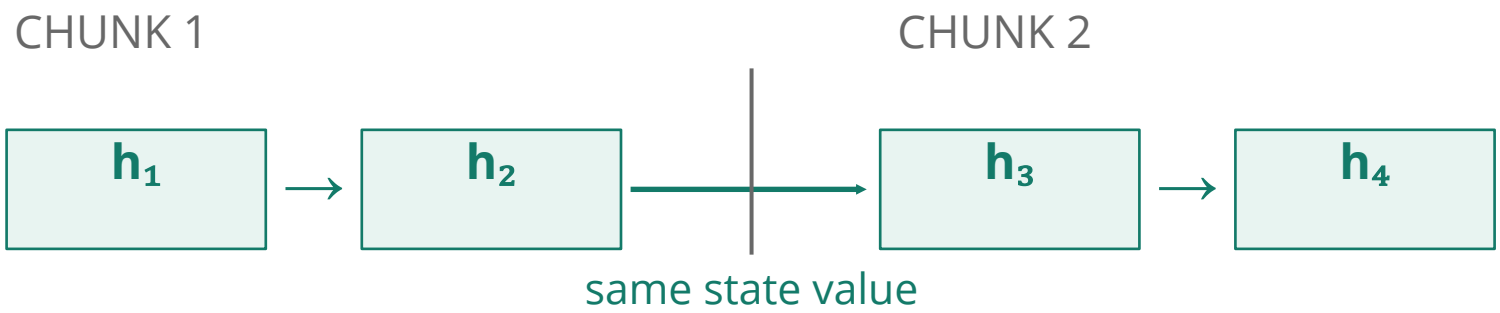
The next chunk starts with information from the previous chunk.

Forward memory can cross the boundary.

Course schematic.

Stop Gradients at the Boundary

2 / 2



STOP ← gradient from chunk-2 loss

detach(h_2)

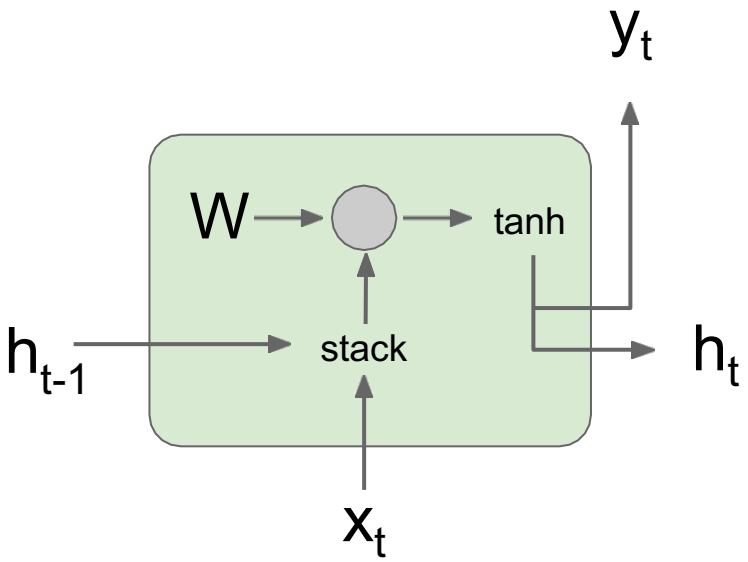
The backpropagation horizon is shorter than the forward history.

Course schematic.

USC Viterbi School of Engineering

Vanilla RNN Gradient Flow

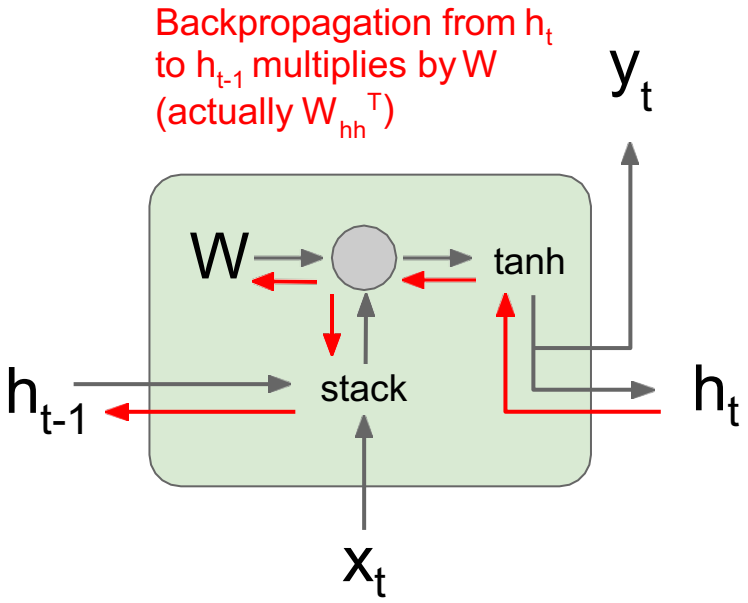
Bengio et al, “Learning long-term dependencies with gradient descent is difficult”, IEEE Transactions on Neural Networks, 1994
Pascanu et al, “On the difficulty of training recurrent neural networks”, ICML 2013



$$
\begin{aligned}
h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\
&= \tanh\left(\begin{pmatrix} W_{hh} & W_{hx} \end{pmatrix} \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right) \\
&= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)
\end{aligned}
$$

Vanilla RNN Gradient Flow

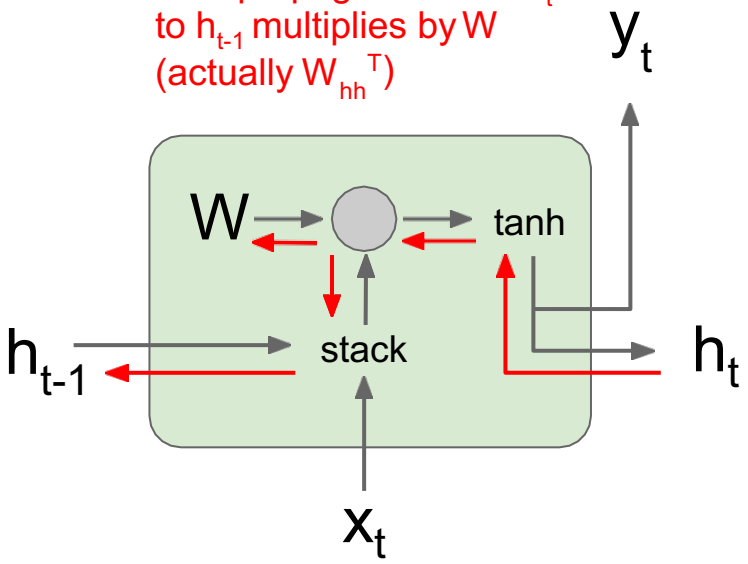
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



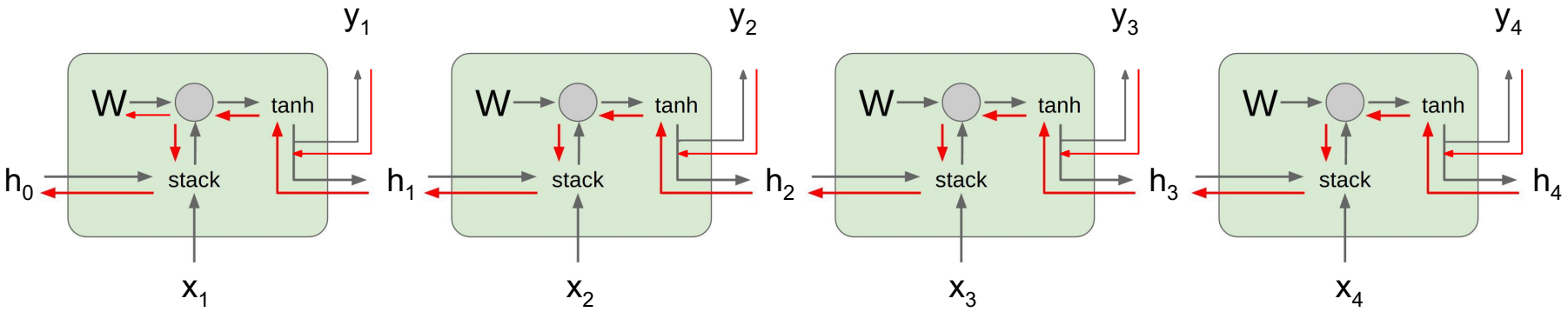
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$
$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$
$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

USC Viterbi School of Engineering

Vanilla RNN Gradient Flow

Bengio et al, “Learning long-term dependencies with gradient descent is difficult”, IEEE Transactions on Neural Networks, 1994
Pascanu et al, “On the difficulty of training recurrent neural networks”, ICML 2013

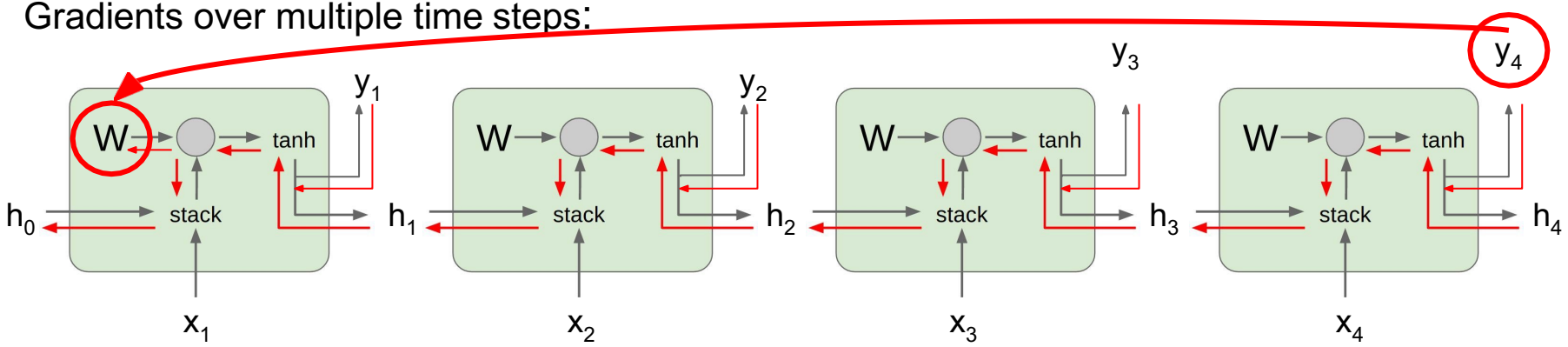


$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



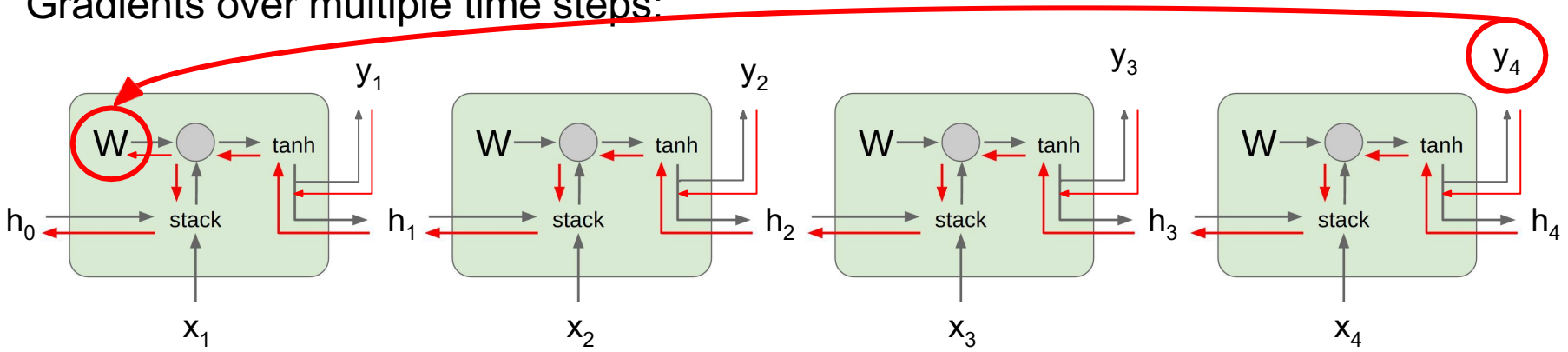
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



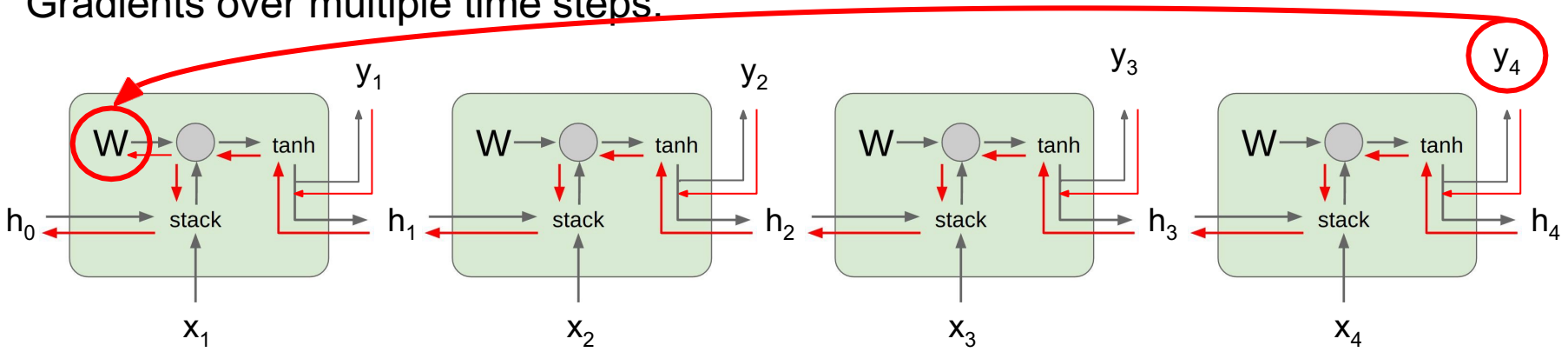
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

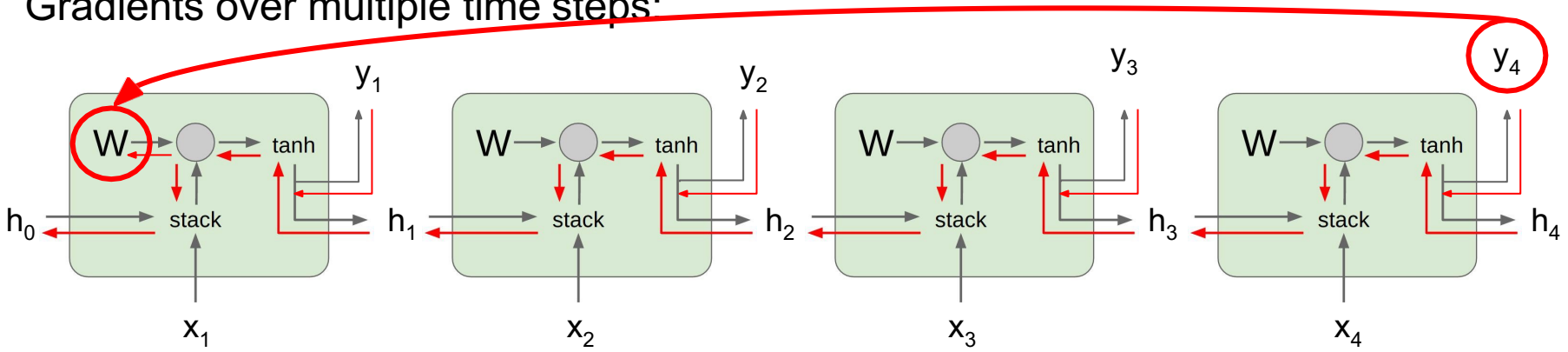
$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

tanh′ ≤ 1 at every step

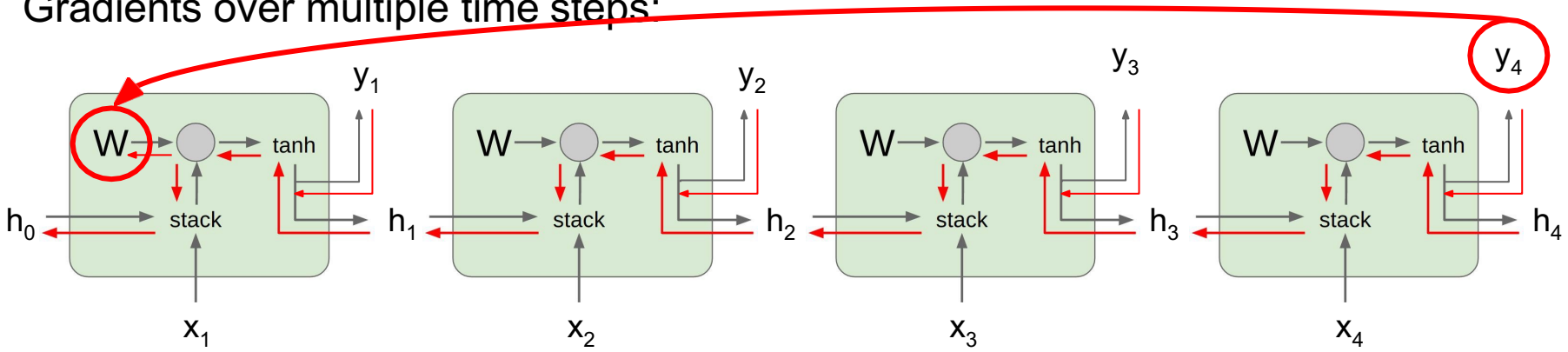
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} tanh'\left(W_{hh} h_{t-1} + W_{xh} x_t\right) \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



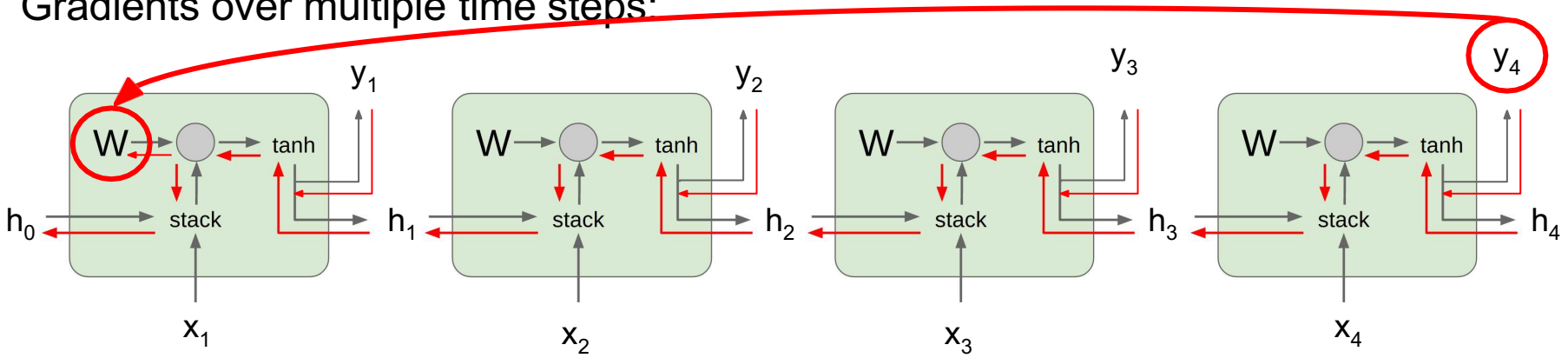
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

What if we assumed no non-linearity?

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:

Gradients can explode

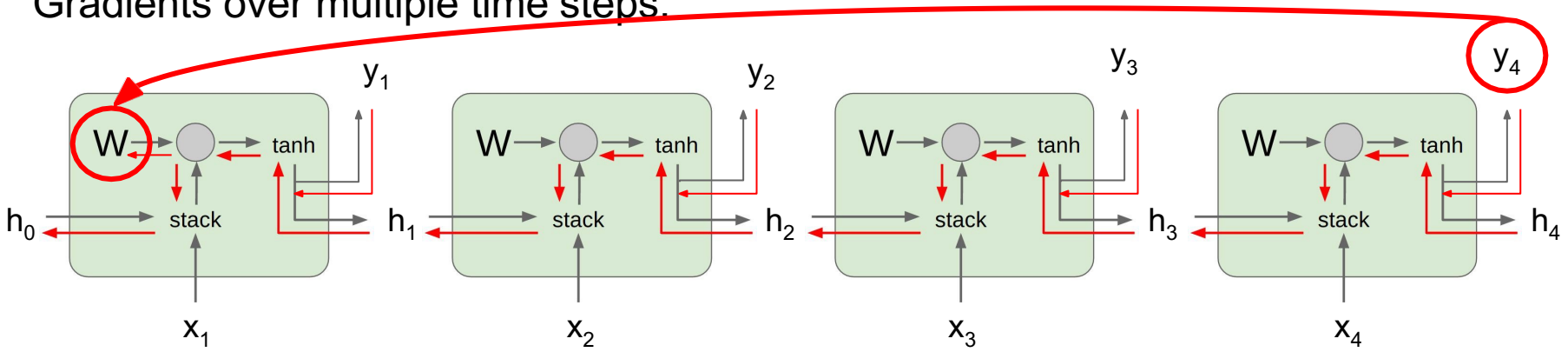
Largest singular value < 1:

Vanishing gradients

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:
Gradients can explode

→ **Gradient clipping**: Scale gradient if its norm is too big

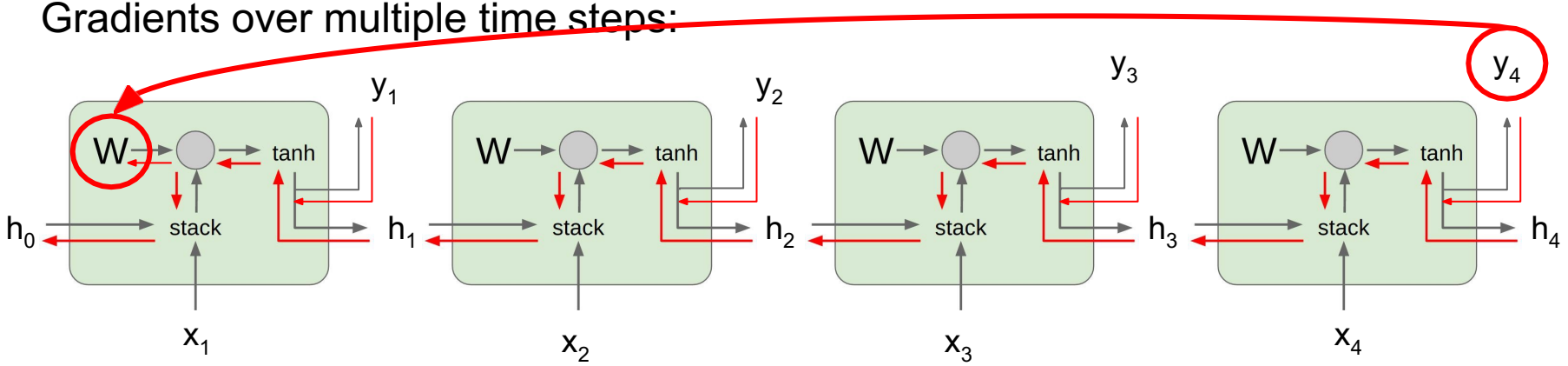
Largest singular value < 1:
Vanishing gradients

$n = \|g\|_2$
If $n > \tau$: $g \leftarrow (\tau / n)\, g$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

Gradients over multiple time steps:



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:

Gradients can explode

Largest singular value < 1:
Vanishing gradients

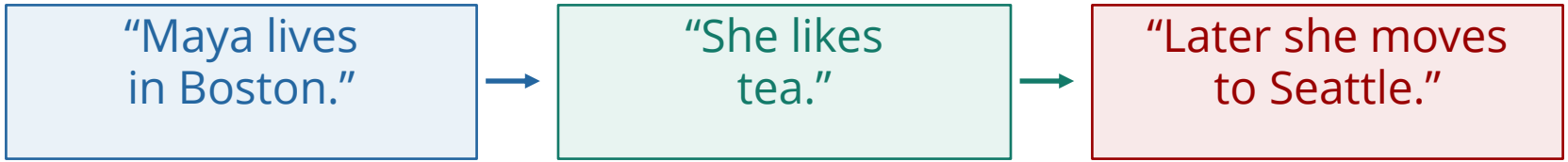
→ Change RNN architecture

Long Short Term Memory (LSTM)

Memory with Learned Gates

Where Does Maya Live Now?

A continuing story: track changing facts across context.



Query prompt: “Where does Maya live now?” → Seattle

Teaching analogy: not literal neuron slots; no measured activations.

LSTM must learn when to retain and when to update memory.

Standard LSTM: PyTorch documentation. Teaching analogy.

Standard RNN: One State for Everything

A standard RNN uses a single recurrent hidden state h_t:

Remember city (earlier context)	Remember tea (local detail)	Predict next (output score)

A single vector mixes distant context and immediate detail.
Intervening words can dilute or displace earlier facts.

A single hidden state mixes memory and immediate output.

Standard LSTM: PyTorch documentation. Teaching analogy.

Step 1: First Fact Sets the City Memory

Step 1: Process the opening sentence of the story.

Evidence 1:
“Maya lives in Boston.”

Memory sketch (analogy)

Boston [active]	Seattle [not seen]

The first sentence adds Boston to the memory sketch.

Initial evidence establishes the current context.

Standard LSTM: PyTorch documentation. Teaching analogy.

Step 2: Unrelated Context Arrives

Step 2: Unrelated preference information arrives.

Evidence 2:
“She likes tea.”

Memory sketch (analogy)

Boston [keep]	Seattle [not seen]

Tea is a preference, not a relocation. Boston remains true!
The model needs to keep older facts when new topics arrive.

New input should not erase unrelated existing memory.

Standard LSTM: PyTorch documentation. Teaching analogy.

A Learned Dimmer: How Much to Retain

A gate acts as a learned dimmer on existing memory:

City Retain Slider: HIGH



Unrelated tea arrives.
Slider near 1 retains old city.

Memory sketch (analogy)

Boston [keep]	Seattle [not seen]

Values near 1 keep most of the old cell content.

A gate acts like a continuous dimmer between 0 and 1.

Standard LSTM: PyTorch documentation. Teaching analogy.

Why Is It Called a Forget Gate?

Terminology contrast: retention multiplier vs forget gate

Retention scale: near 1 keeps most; near 0 keeps little



Historical Name: FORGET GATE Standard literature term	Actual Math: RETENTION FACTOR Multiplies past cell state c_{t-1}

Gates softly modulate features; values near 0 retain little info.

High forget gate: retain more. Low: retain less.

Standard LSTM: PyTorch documentation. Teaching analogy.

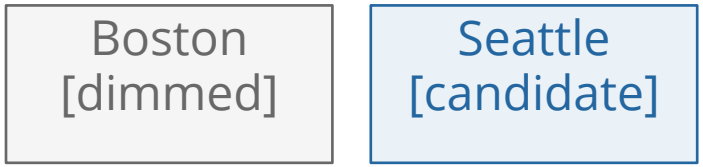
Step 3: A Relocation Changes the Fact

Step 3: A relocation overrides the previous city fact.

Evidence 3:
“Later she moves to Seattle.”

“moves to” signals a change:
Old Boston fact is outdated.

Memory sketch (analogy)



The retain slider for Boston turns low to reduce outdated info.

When facts change, the retain dimmer turns low.

Standard LSTM: PyTorch documentation. Teaching analogy.

Two Separate Decisions: Retain and Write

Two controls operate at the same step.

1. FORGET GATE: LOW



Action on existing memory:
Retain little of old Boston

2. INPUT GATE: HIGH



Action on candidate content:
Write new candidate Seattle

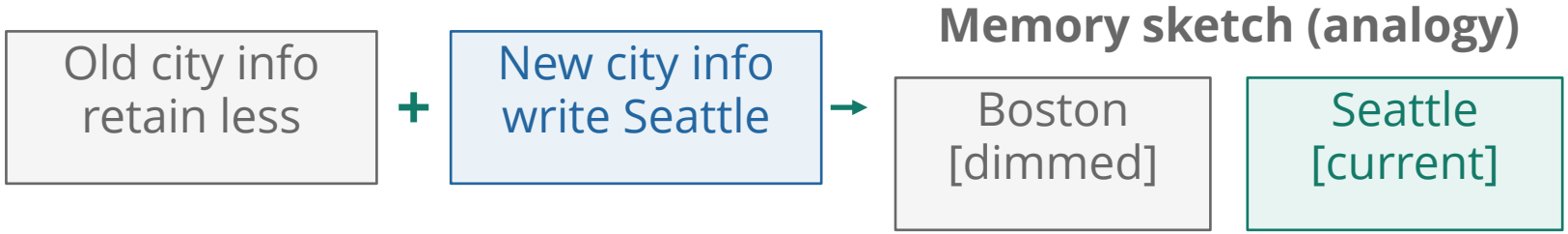
Reducing old memory and writing new candidate content are separate.

Retaining old memory and writing new memory are separate.

Standard LSTM: PyTorch documentation. Teaching analogy.

Updating Memory: Retained Plus Added

Cell update: retained past joins new candidate content:



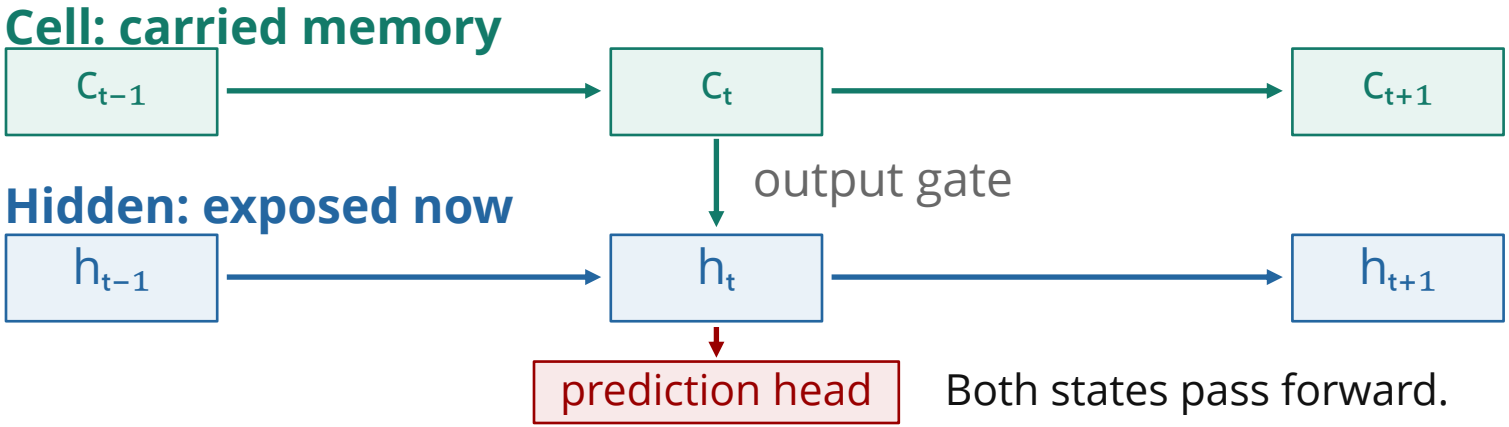
The old-memory path and new-content path meet here.

Updated cell memory combines retained past and new info.

Standard LSTM: PyTorch documentation. Teaching analogy.

Two Lanes: Cell State and Hidden State

Both states move forward as the next token arrives:

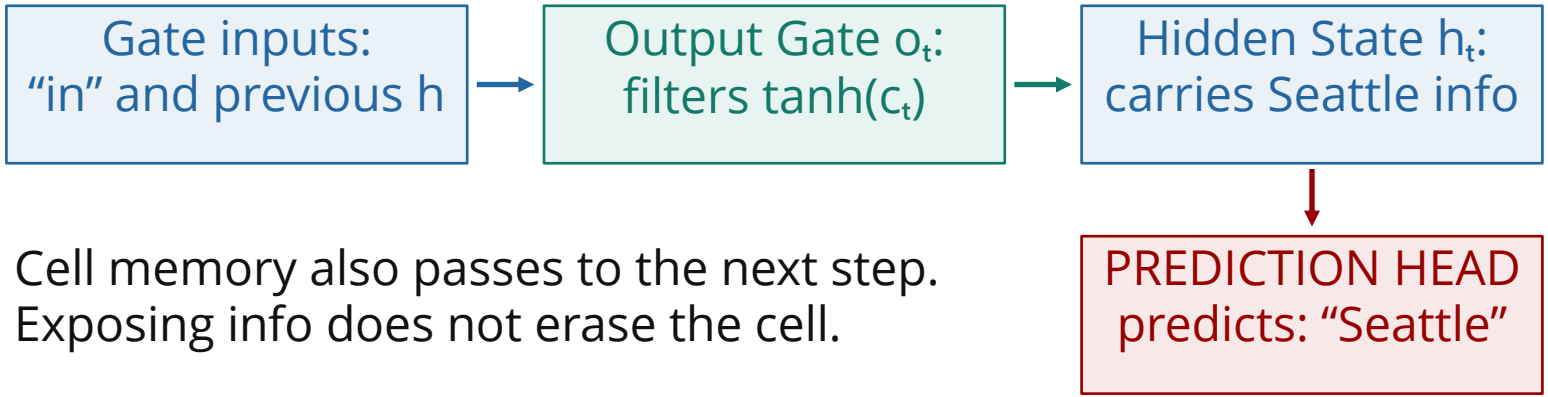


Cell state carries memory; hidden state is exposed now.

Standard LSTM: PyTorch documentation. Teaching analogy.

Output Gate: Expose What Is Needed Now

Model processes prompt tokens: "...Maya now lives in"



Cell memory also passes to the next step.
Exposing info does not erase the cell.

Exposing memory for an output does not delete the cell.

Standard LSTM: PyTorch documentation. Teaching analogy.

Conceptual Analogy vs Distributed Vectors

Teaching analogy vs real distributed representations:

Teaching Analogy	Real Neural Vectors
Named city slots	Dense vectors in $\mathbb{R}^d$

City names are labels for our sketch, not actual neurons.

Gates are vectors in $[0, 1]^d$: different dimensions dim independently.

Gates operate dimension-by-dimension on real vectors.

Standard LSTM: PyTorch documentation. Teaching analogy.

Where Do Gate Values Come From?

Current x_t and previous h_{t-1} feed separate gate networks:



At the same layer, weights are shared across time.
Backpropagation through time trains each weight matrix.

Shared trained weights compute context-dependent gates.

Standard LSTM: PyTorch documentation. Teaching analogy.

Recap: Forget, Input, and Output Roles

Three distinct gate roles work together at each step:



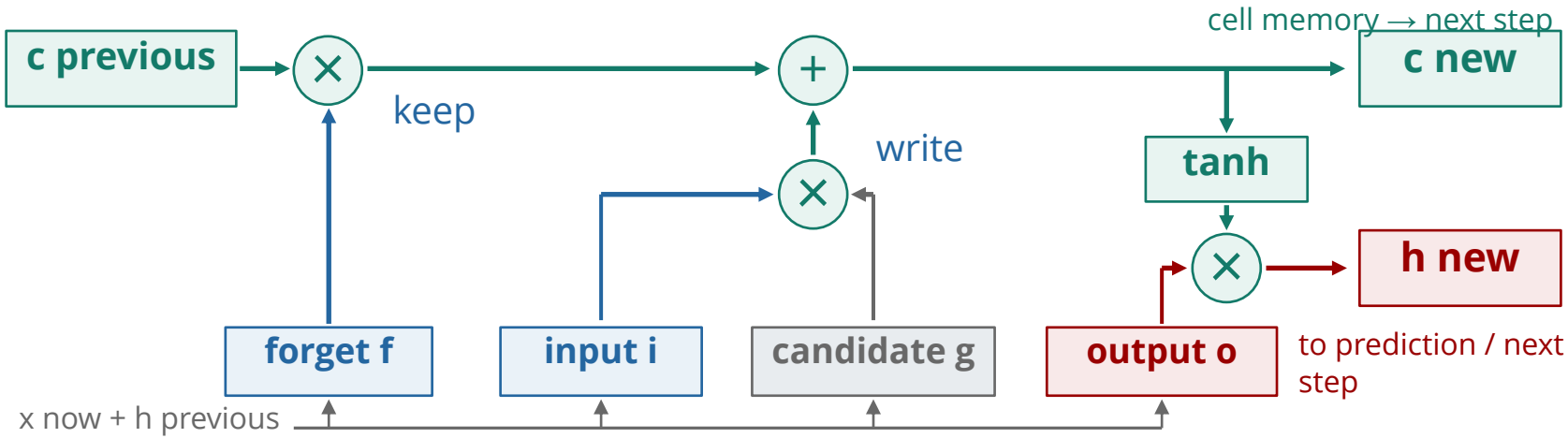
Retain past + write new candidate → updated cell → expose to hidden.

Three learned dimmers control retain, write, and expose.

Standard LSTM: PyTorch documentation. Teaching analogy.

The Complete LSTM Cell

One time step: keep, write, and expose information.



The three gates work together inside one repeated LSTM cell.

Standard LSTM cell; PyTorch LSTM equations. Editable course diagram.

The Limitation: A Compact State Bottleneck

A fixed-size state pair must compress an arbitrary history:

Long Token Sequence:
Hundreds of tokens

→

Fixed State Pair:
(c_t, h_t) fixed size

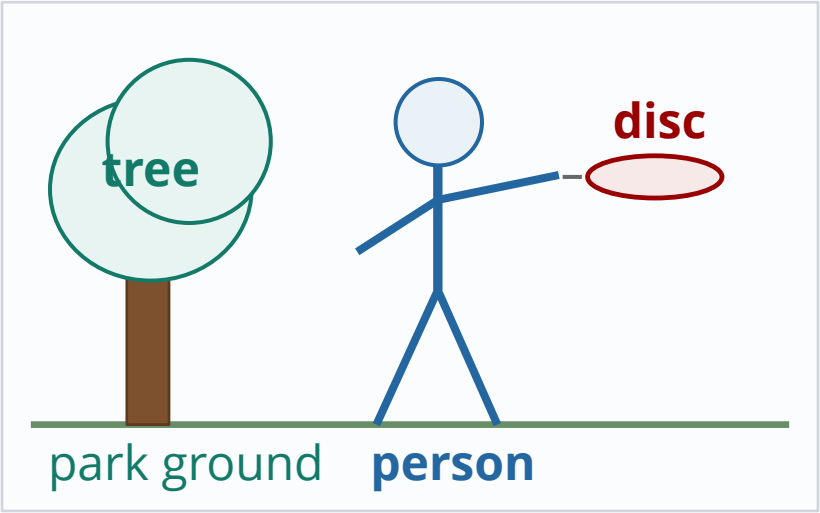
Can we keep separate vectors and read them when needed?
Next: describe an image, then return to text generation.

A fixed-size state must compress the entire past.

Standard LSTM: PyTorch documentation; Vaswani et al. (2017).

Image Captioning: Generate a Description

INPUT: one image



OUTPUT: a generated description

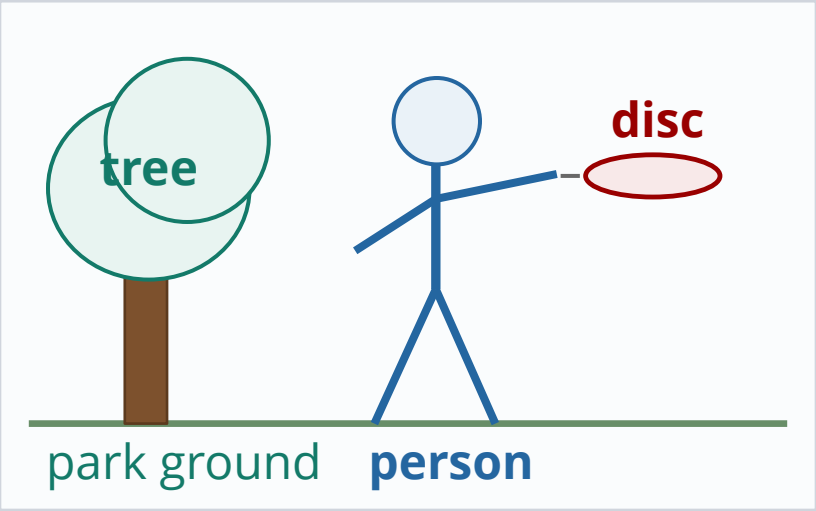
A person throws a frisbee

Generate one word at a time.
Which region helps next?

Each next word can draw on different image regions.

Image captioning example; attention mechanism: Xu et al. (2015).

Image Captioning: One Image Summary



One Image Summary

Prefix: “A”

pooled feature vector:



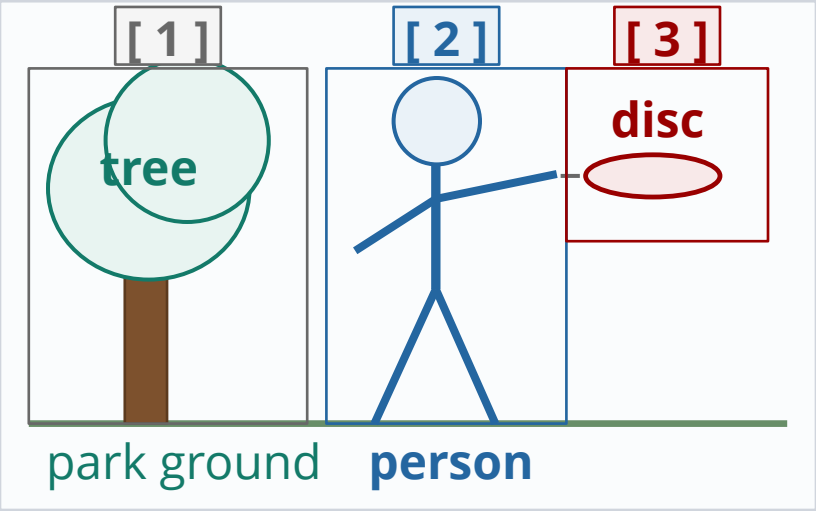
decoder state (c, h)

continuation: “person”

One pooled summary forces the entire scene into one vector.

Original teaching scene; illustrative emphasis. Xu et al. (2015).

Separate Features for Each Region



Separate Region Features

Separate vector for each region:

[1] park & tree features

[2] person features

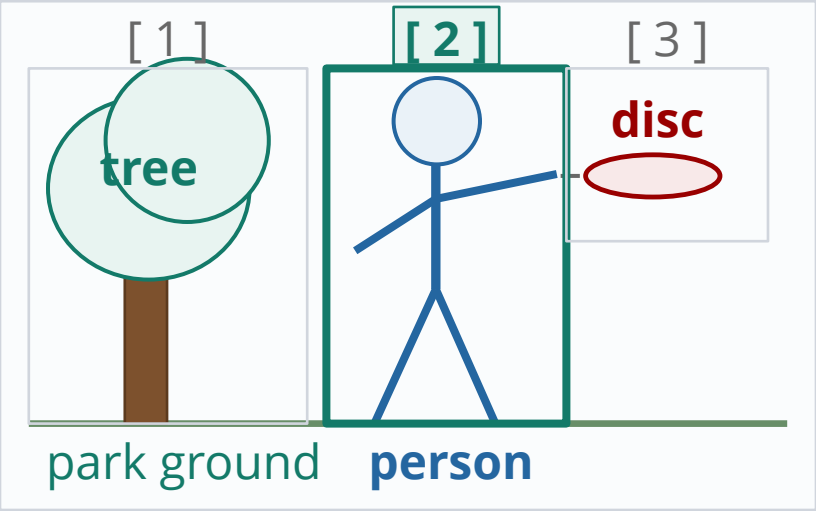
[3] disc features

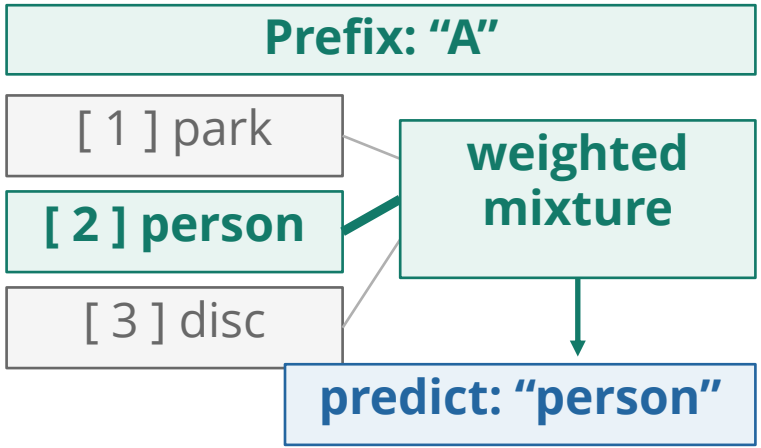
One vector per region.

Separate region vectors avoid forcing all parts together.

Original teaching scene; illustrative emphasis. Xu et al. (2015).

Prefix 'A': Query Relevant Regions



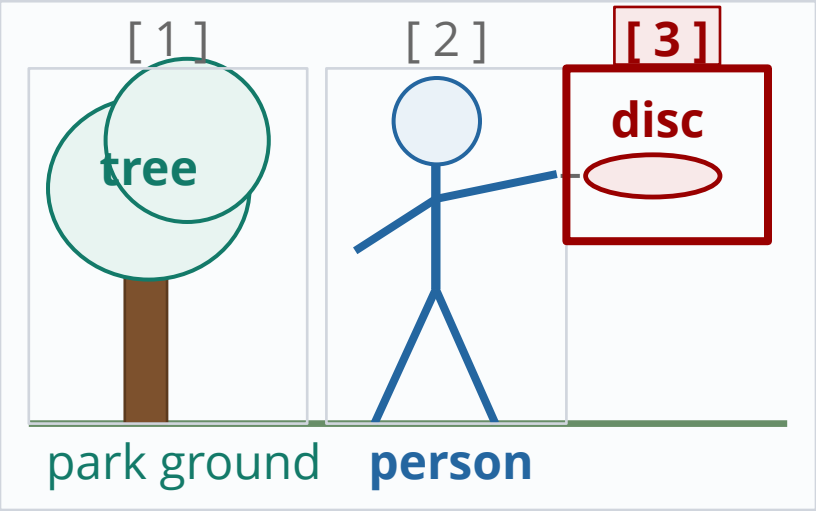


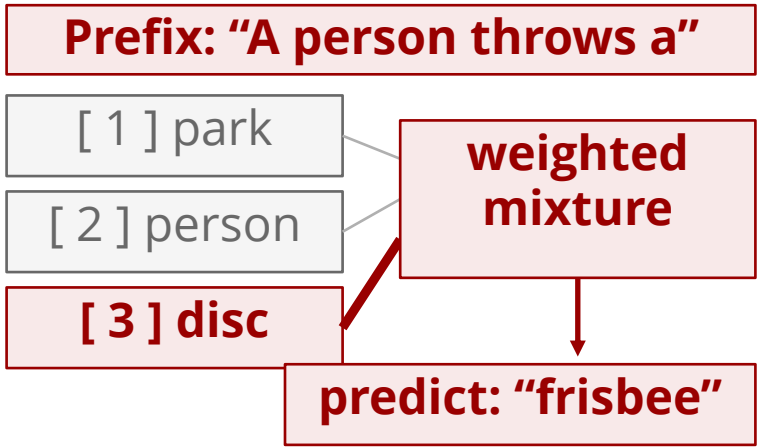
Person contributes most here.

The prefix queries evidence, weighting the person region.

Original teaching scene; illustrative emphasis. Xu et al. (2015).

Later Prefix: Query Shifts to Disc





Same regions, different weights.

A new prefix computes new scores from the same evidence.

Original teaching scene; illustrative emphasis. Xu et al. (2015).

From Image Regions to Self-Attention

Attention over an Image

Context from a separate image:

region 1: park & tree

region 2: person

region 3: disc

Read the image regions.

Self-Attention over Text

Context from the sequence itself:

position 1: “The”

position 2: “dog”

position 3: “ran”

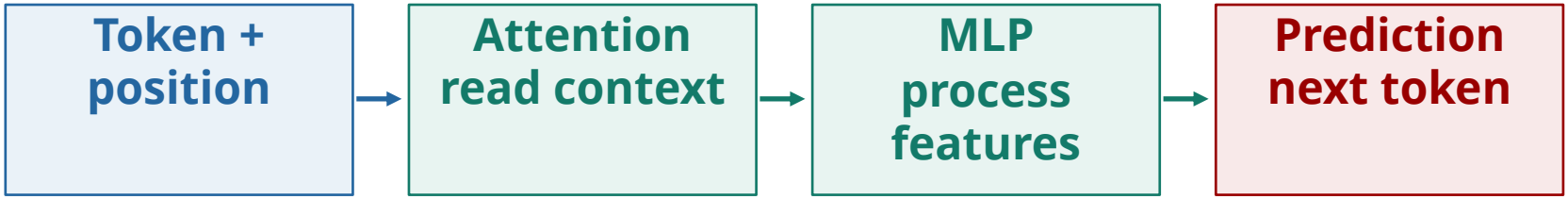
Read itself and earlier tokens.

In self-attention, the sequence supplies position and context.

Original teaching scene; illustrative emphasis. Xu et al. (2015).

Now Build a Text-Generating Transformer

Task: continue “The dog chased the ball. It …”



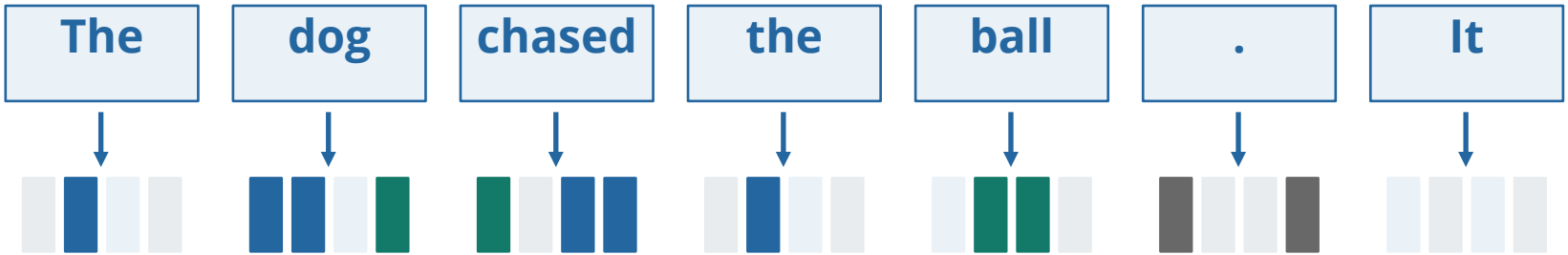
Repeat attention + MLP in several layers.

Follow one path from input words to the next-token prediction.

Vaswani et al. (2017). Course schematic.

Words Enter as Feature Vectors

Start with one learned vector per token.



Simplified token vectors; colored bars are illustrative, not measured.

Each token begins as a learned vector of feature values.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Word Order Changes Sentence Meaning

Same words, different order, different meaning.



Word vectors alone do not specify their order.

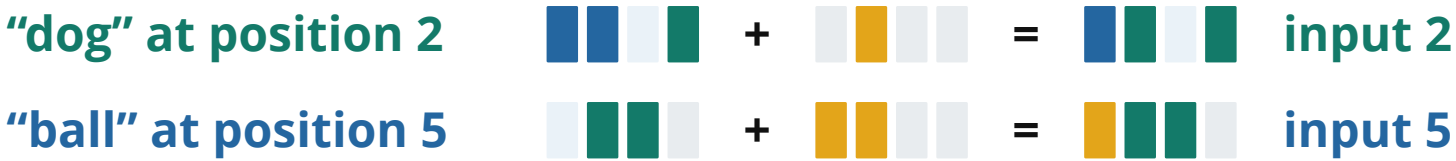
Identical words in different orders mean different things.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Add a Position Vector to Each Token

Give each token a signal for its position.

The	dog	chased	the	ball	.	It
p1	p2	p3	p4	p5	p6	p7



Here: word vector + position vector = input vector

Position values preserve word order across the sequence.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

One Token Asks About Its Context

At “It”, read the available context.



“It” could refer to the dog or the ball.

All seven positions, including “It”, can supply information.

A token needs context from other tokens to resolve its meaning.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Step 1: Compute Attention Weights

At “It”: how much should each source contribute?



Illustrative weights for one head; they add to 1.

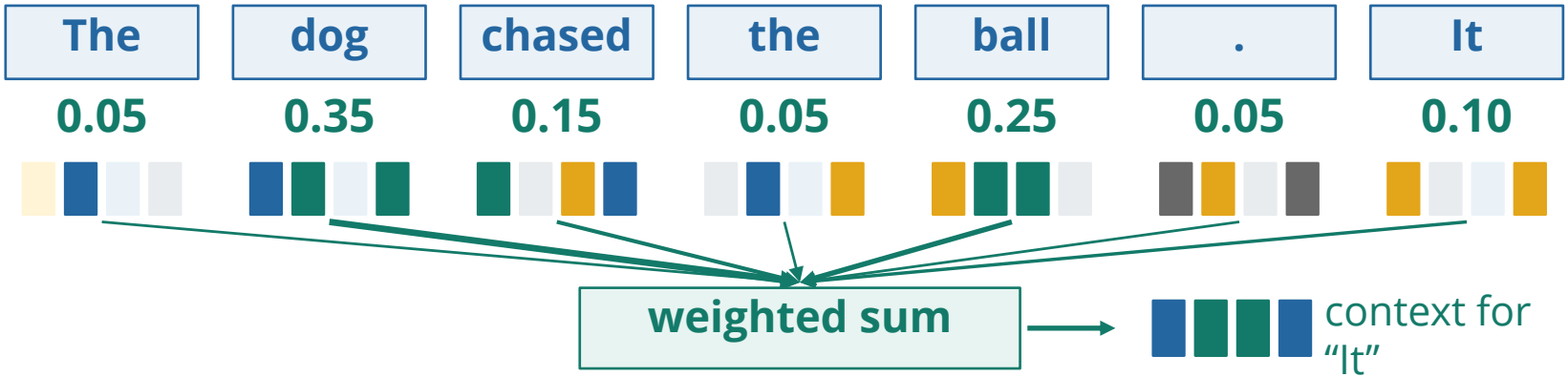
A different token or context produces a different set of weights.

Attention weights are computed from the current input.

Vaswani et al. (2017). Course schematic.

Step 2: Mix the Source Information

At “It”: use those weights to combine source vectors.



Illustrative values and weights; Block 9 shows the projections.

The weights control how much each value vector contributes.

Vaswani et al. (2017). Course schematic.

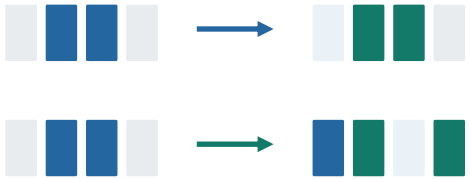
USC Viterbi School of Engineering

Context Changes the Meaning of a Word

Same starting vector; different context.

The ball rolled. It → ball context

The dog ran. It → dog context



The context changes what information reaches “It”.

Our earlier dog-and-ball sentence remains ambiguous.

Different sentence contexts produce different token vectors.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Why Future Tokens Must Be Hidden

Hide the word we are trying to predict.



CAUSAL MASK: future words hidden

Seeing “chased” would reveal the answer.

A position cannot see future tokens it is trying to predict.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Causal Attention Across Two Positions

Later positions can read a longer prefix.

At “dog”: read the first 2 positions.

The	dog	chased	the	ball	.	It

At “ball”: read the first 5 positions.

The	dog	chased	the	ball	.	It

Colored: allowed. Gray: future positions, hidden here.

Each token may attend to itself and earlier tokens only.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Order, Visibility, and Contribution Are Different

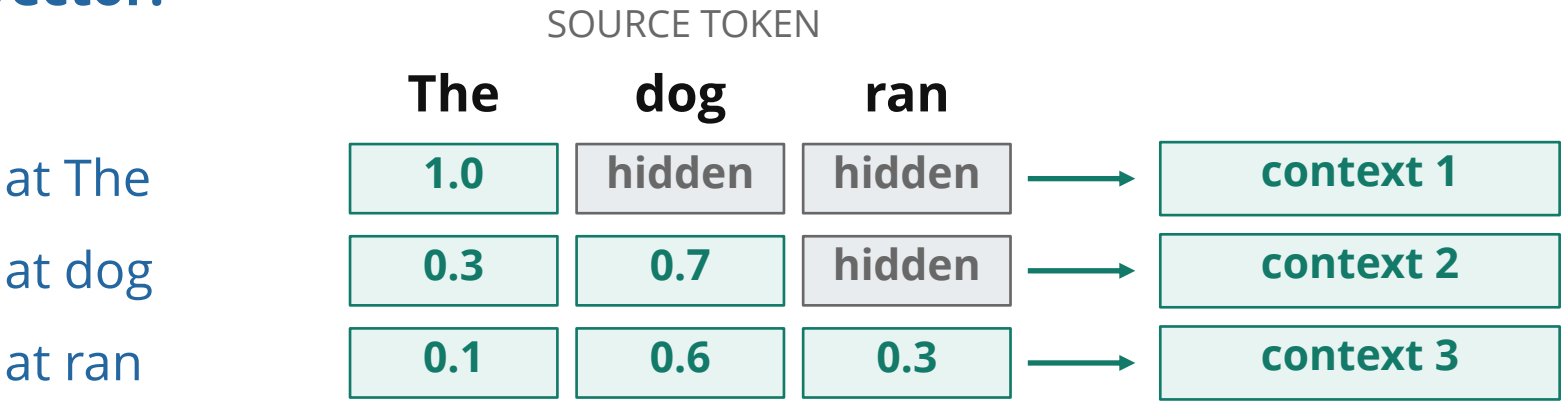
POSITION	CAUSAL MASK	ATTENTION WEIGHTS
dog at position 2	**At dog: The + dog**	**At It: dog gets 0.35**
Where is this token?	Which tokens are visible?	How much does each contribute?

Order, access, and contribution are three separate choices.

Vaswani et al. (2017). Course schematic.

Each Position Computes Its Own Mixture

Each query position gets its own weights and context vector.



Illustrative weights. Each row uses the same layer input.

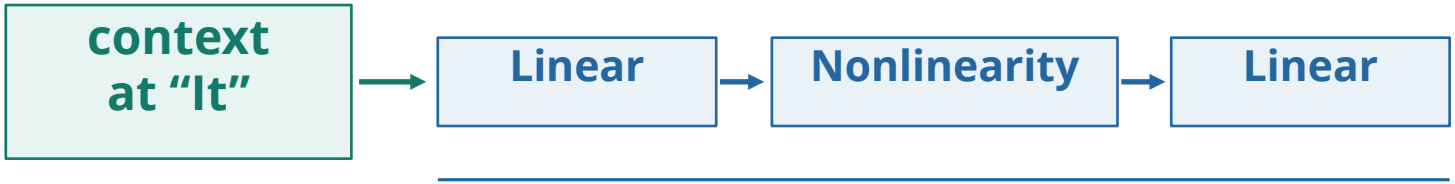
Shared parameters compute a different mixture at each position.

Vaswani et al. (2017). Course schematic.

USC Viterbi School of Engineering

Why Add an MLP After Attention?

After gathering context, learn how to combine its features.



A familiar feedforward network (MLP), applied to one position.

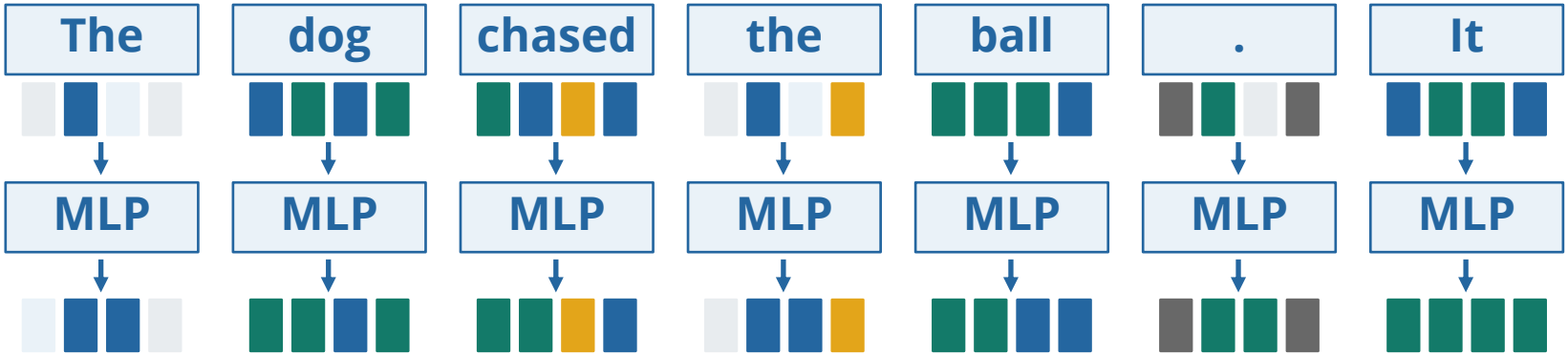
Its output is another feature vector for “It”.

Attention gathers context; the MLP learns new feature combinations.

Vaswani et al. (2017). Course schematic.

Same MLP, Different Token Vectors

Reuse this MLP at every position in the layer.



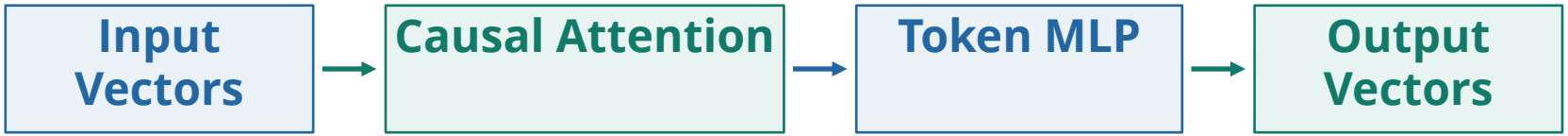
Same MLP weights at every position within this layer.

A shared MLP transforms each contextualized vector separately.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Inside One Transformer Block

Put attention and the MLP together.



Attention: exchange information across positions.
MLP: transform the features within each position.

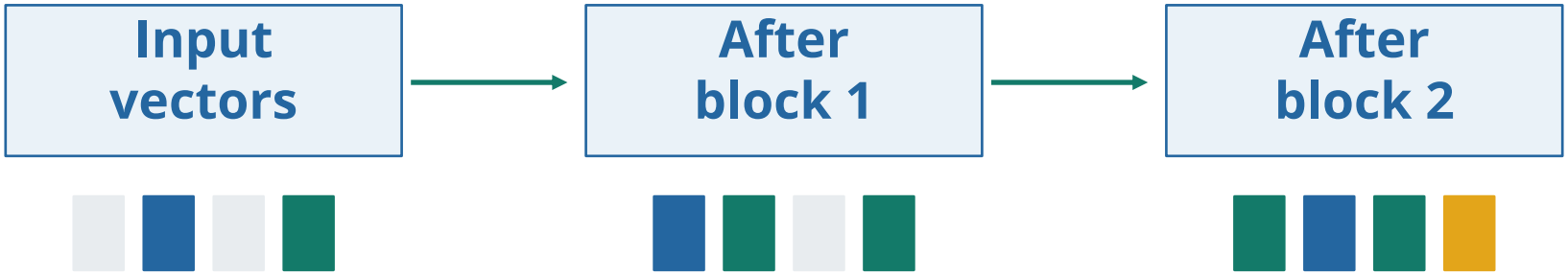
Residual connections and LayerNorm omitted for clarity.

Attention shares information; the MLP refines each vector.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Why Use More Than One Block?

Later blocks work with features already processed by earlier blocks.



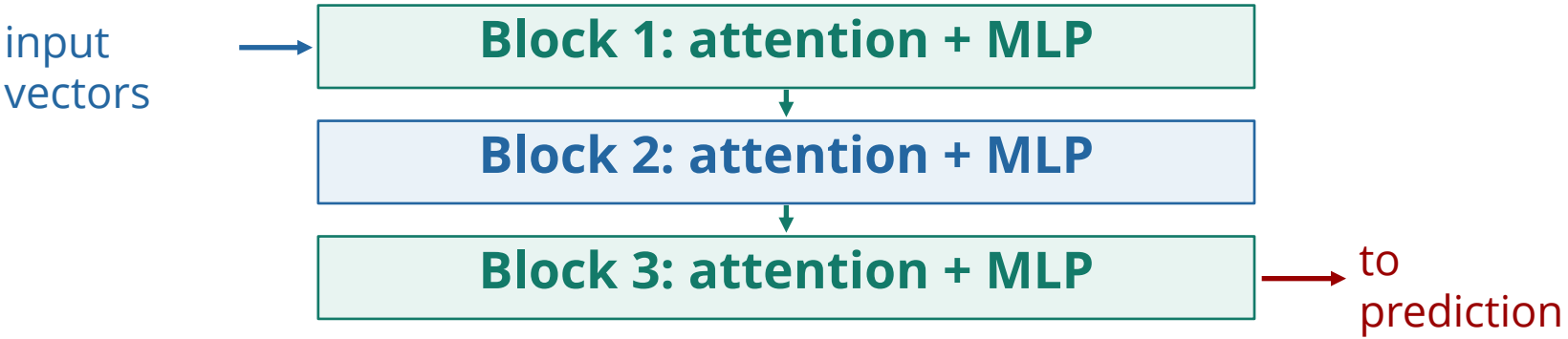
One attention layer can already read the whole allowed prefix.

More blocks add learned processing steps, with extra computation.

Vaswani et al. (2017). Course schematic.

Stack Blocks to Build a Deeper Network

Each block has its own attention and MLP weights.



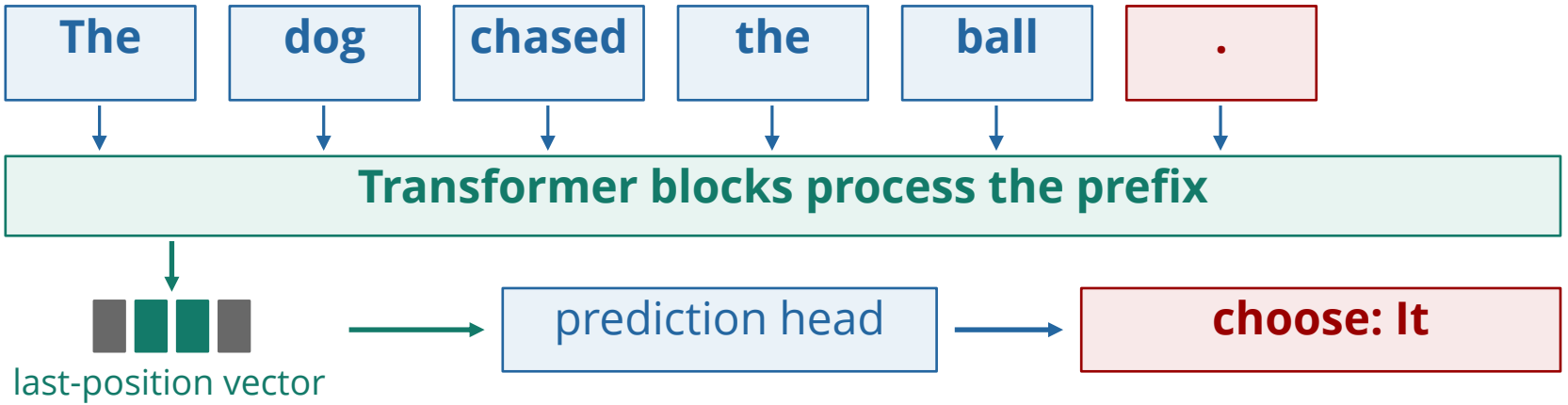
Same architecture. Different weights. One jointly trained model.

Depth counts blocks; generation length counts tokens.

Vaswani et al. (2017). Course schematic.

Generation Step 1: Predict the Next Word

Read the prefix, then choose one more token.

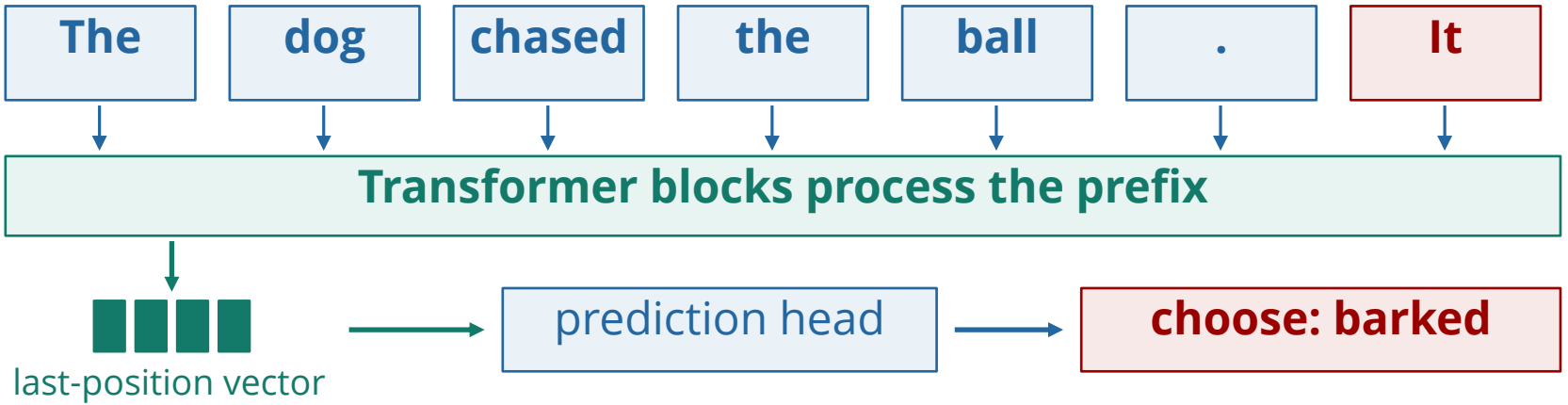


The final transformed vector produces scores for the next word.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Generation Step 2: Append and Predict Again

Read the prefix, then choose one more token.

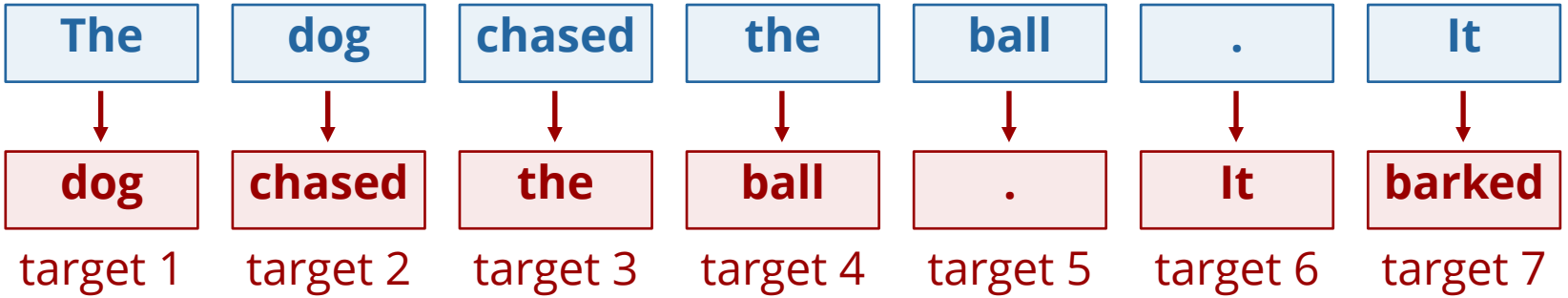


Append the new token to the sequence and repeat the process.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Training Scores All Positions in Parallel

During training, the full example is already known.



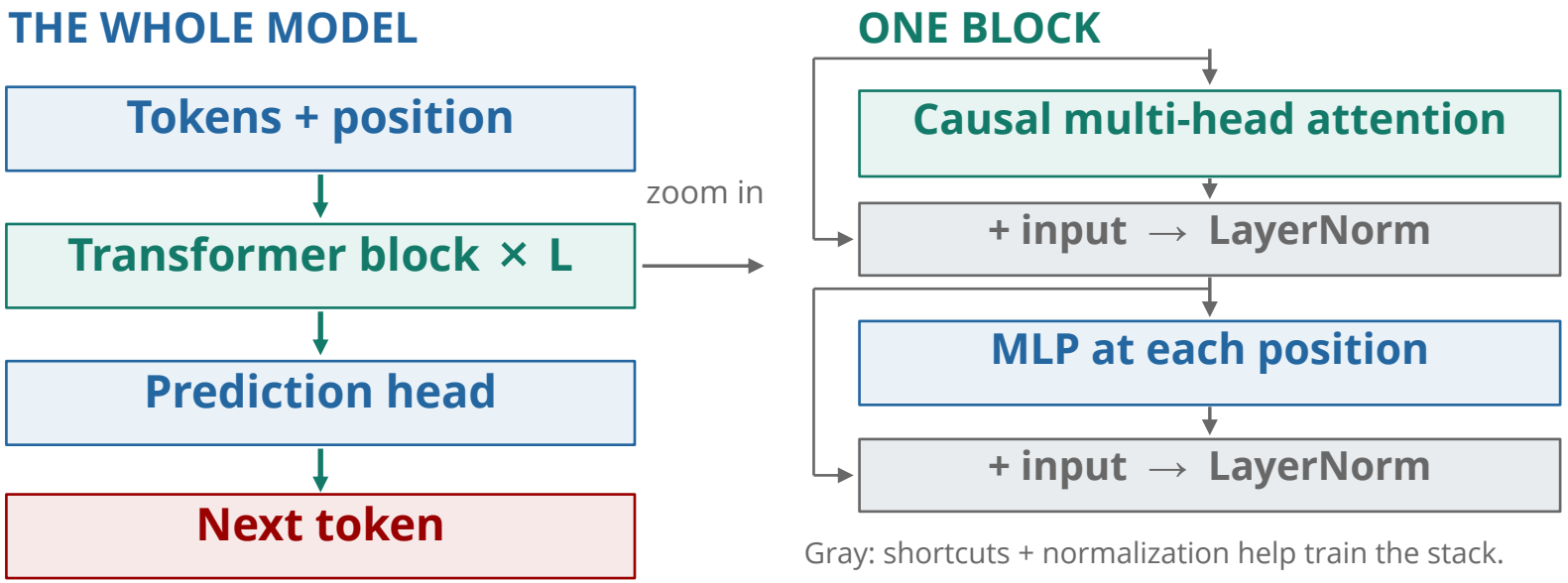
Score every next-token target, with future tokens hidden.

During generation, the next chosen token is not yet known.

Training evaluates all sequence positions in a single pass.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

The Complete Text-Generating Transformer



Gray: shortcuts + normalization help train the stack.

Generate → append the new token → run the model again.

Vaswani et al. (2017). Course schematic.

The Transformer Picture and Next Steps

Four pieces to remember from this introduction.

Position	Attention	MLP	Prediction
Track word order	Mix context	Transform each vector	Score the next token

“Self”: the current position and context share a sequence.

Block 9: how the model learns which information to read.

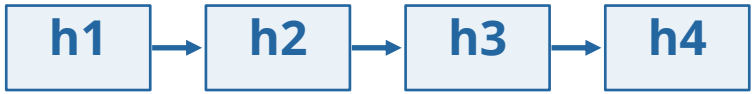
Today: the information flow. Block 9: how scores are computed.

Vaswani et al. (2017); Radford et al. (2019). Teaching analogy.

Why Transformers Scale: Efficient Training

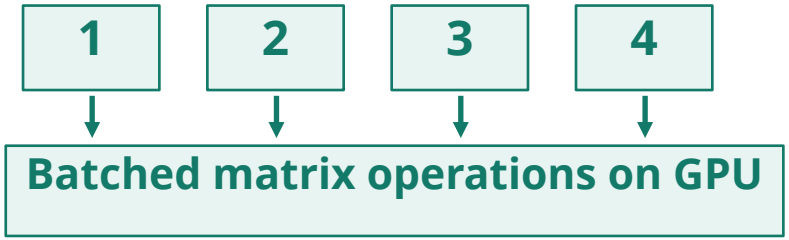
Training text is known: process many positions together.

RNN: sequential state updates



Each step waits for the previous state.

Transformer: parallel positions



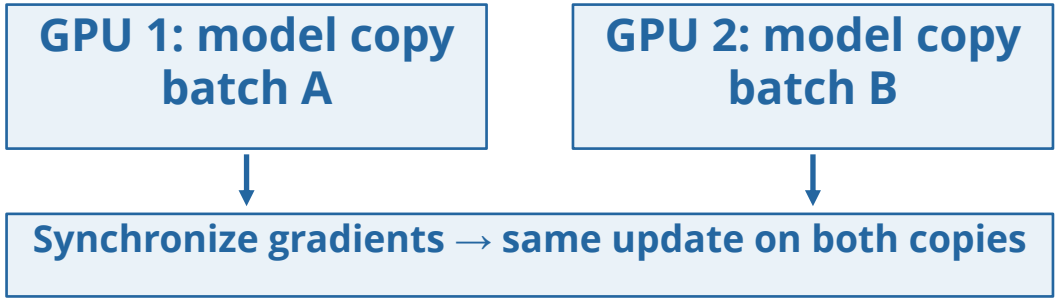
Parallel positions + adjustable width/depth support large training runs.

Train many positions together; generate one new token at a time.

Vaswani et al. (2017). Course schematic.

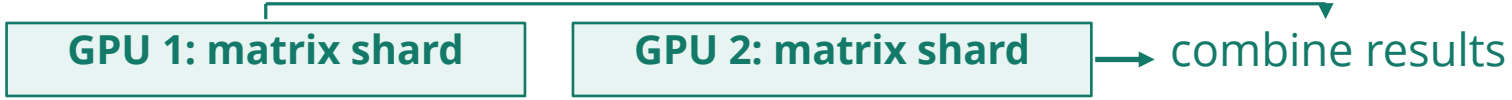
Scaling Further: Train Across Multiple GPUs

MORE DATA: copy the model, split the examples



Data parallelism

BIGGER MODEL: split large computations across GPUs



Communication is required. These training methods also apply to other architectures.

Sources: PyTorch DDP tutorial; Shoeybi et al. (2019), Megatron-LM.

USC Viterbi School of Engineering

RNN applications

- Image Captioning
- Image Captioning with Attention
- Question Answering
- Visual Question Answering
- Speech Recognition
- Action Recognition in Videos
- Text Parsing
- Machine Translation

USC Viterbi School of Engineering

Image Captioning



I look better than Tom Cruise.

Explain Images with Multimodal Recurrent Neural Networks, Mao et al.
Deep Visual-Semantic Alignments for Generating Image Descriptions, Karpathy and Fei-Fei
Show and Tell: A Neural Image Caption Generator, Vinyals et al.
Long-term Recurrent Convolutional Networks for Visual Recognition and Description, Donahue et al.
Learning a Recurrent Visual Representation for Image Caption Generation, Chen and Zitnick

Image Captioning

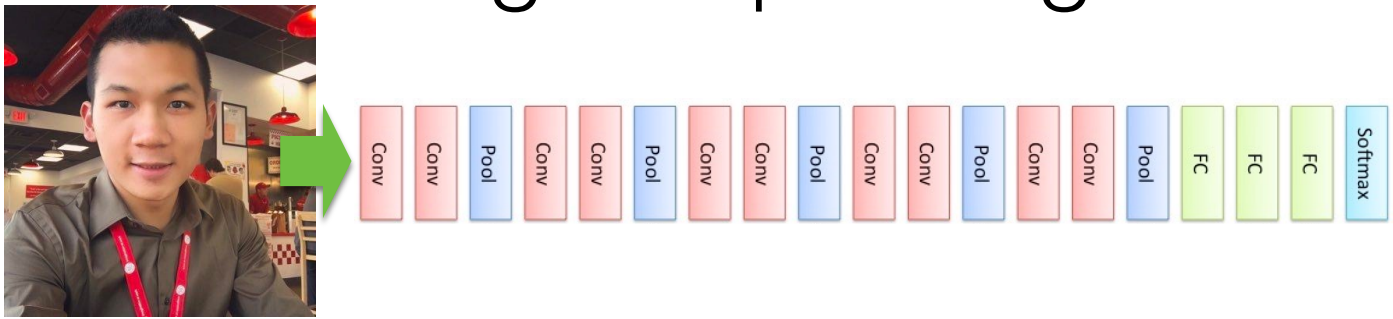


Image Captioning

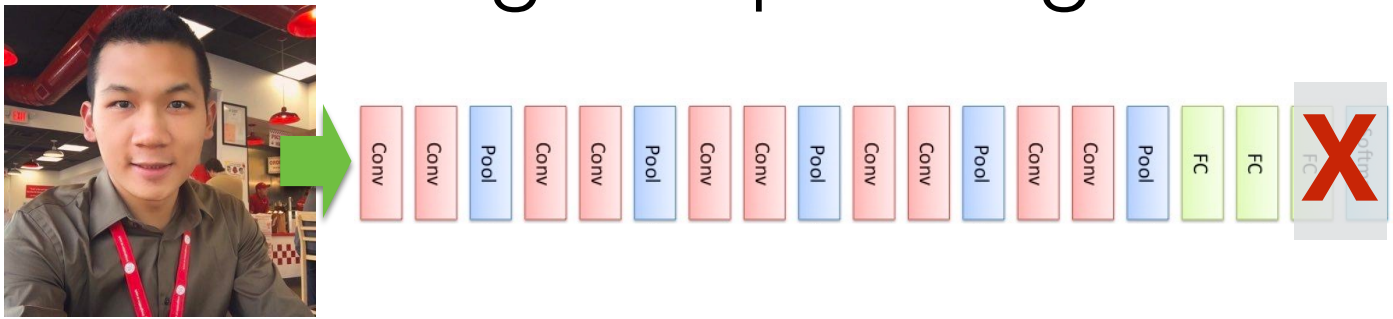
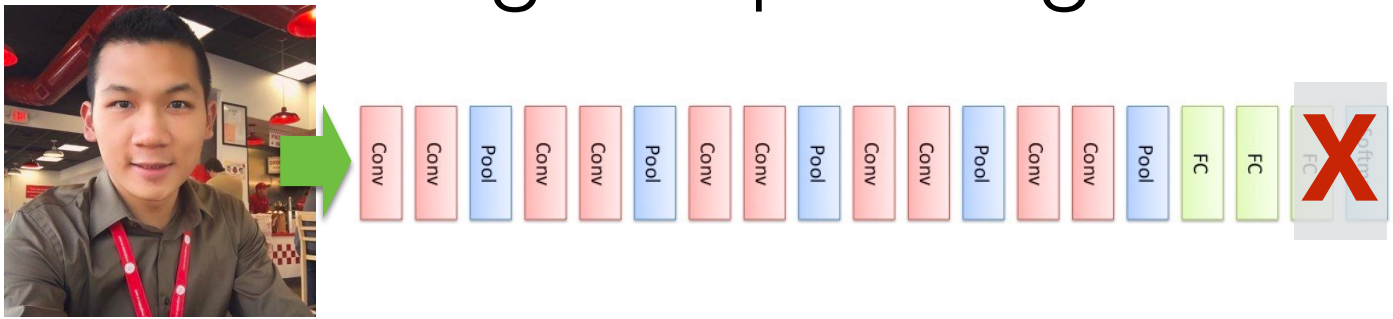


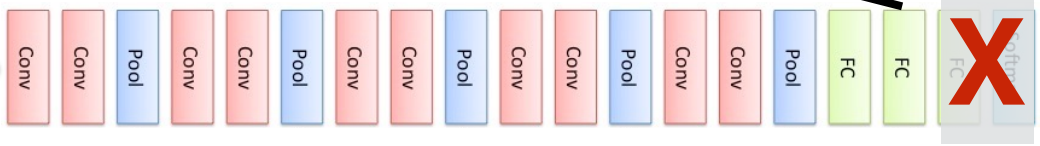
Image Captioning



USC Viterbi School of Engineering

Image Captioning





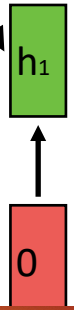
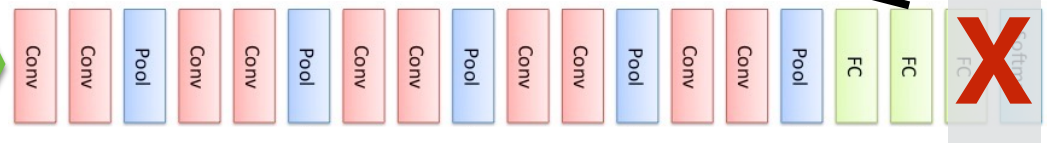


Image Captioning



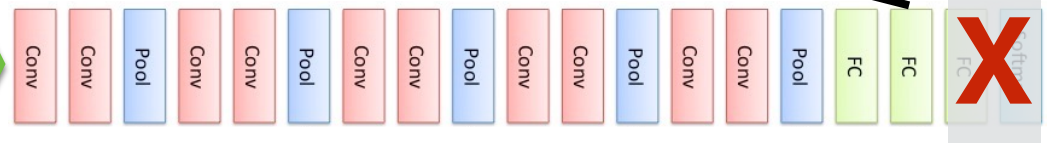


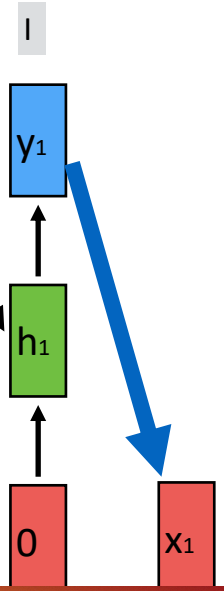


USC Viterbi School of Engineering

Image Captioning



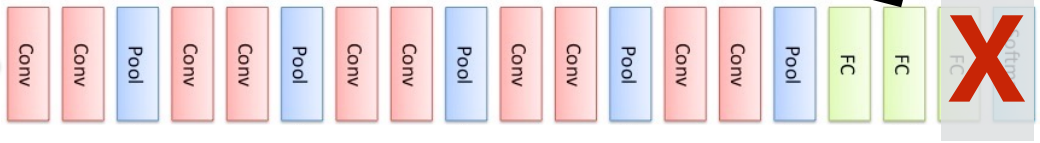


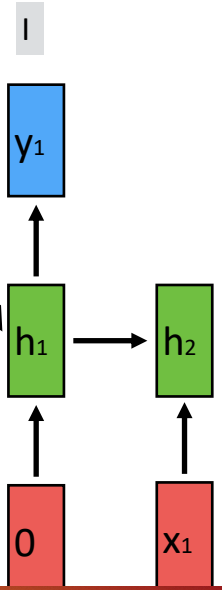


USC Viterbi School of Engineering

Image Captioning



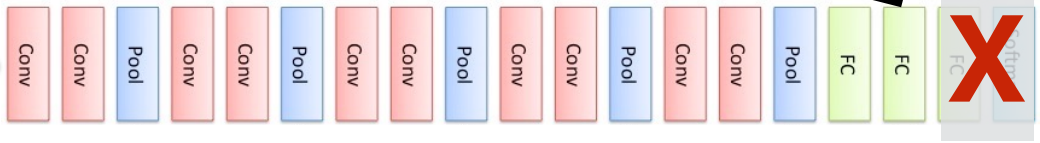


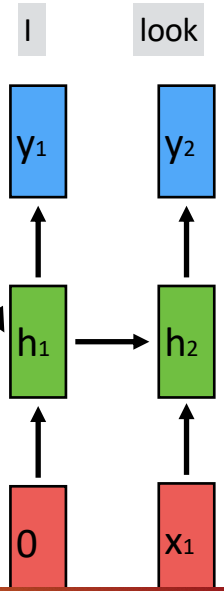


USC Viterbi School of Engineering

Image Captioning



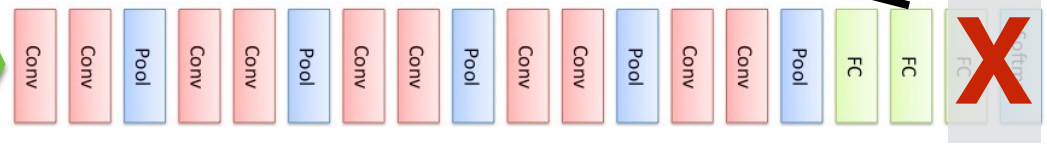


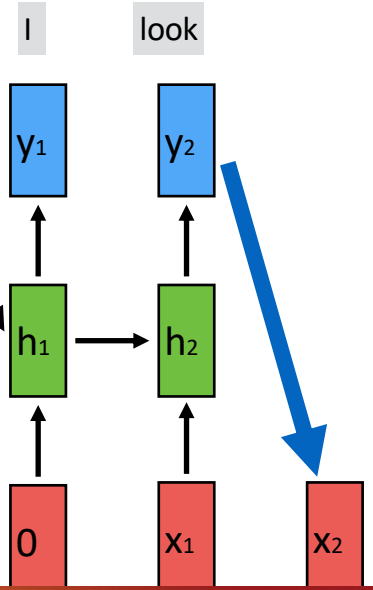


USC Viterbi School of Engineering

Image Captioning



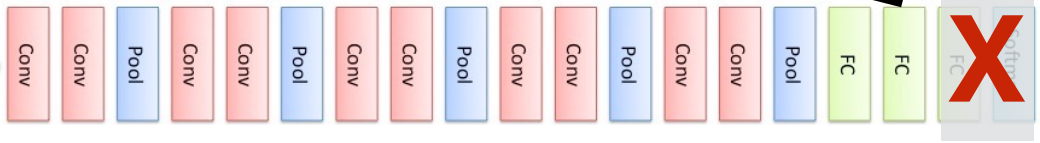


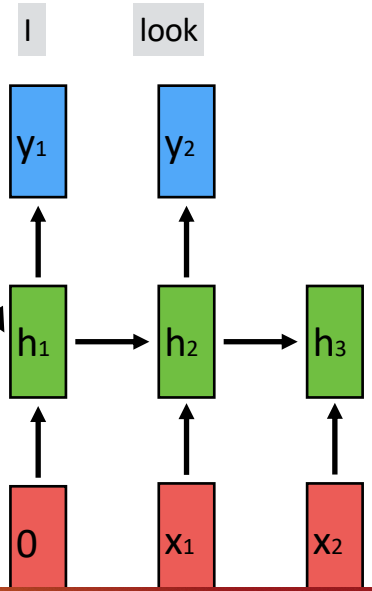


USC Viterbi School of Engineering

Image Captioning



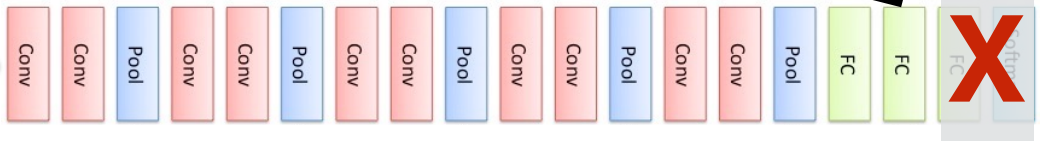


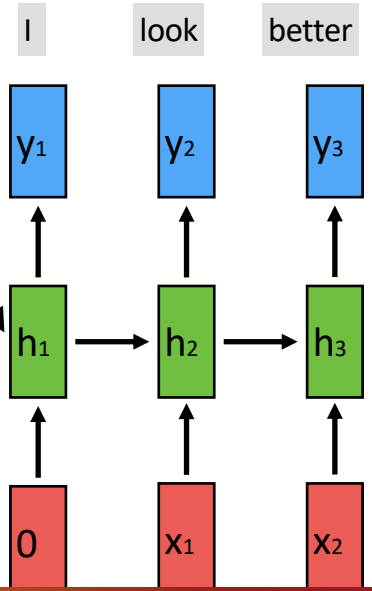


USC Viterbi School of Engineering

Image Captioning



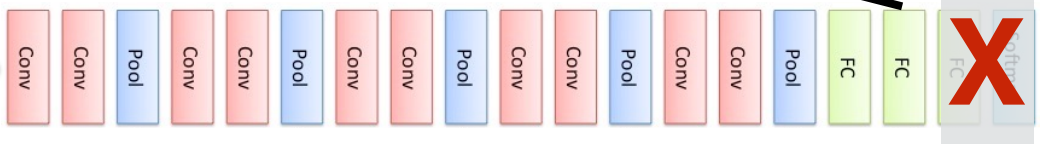


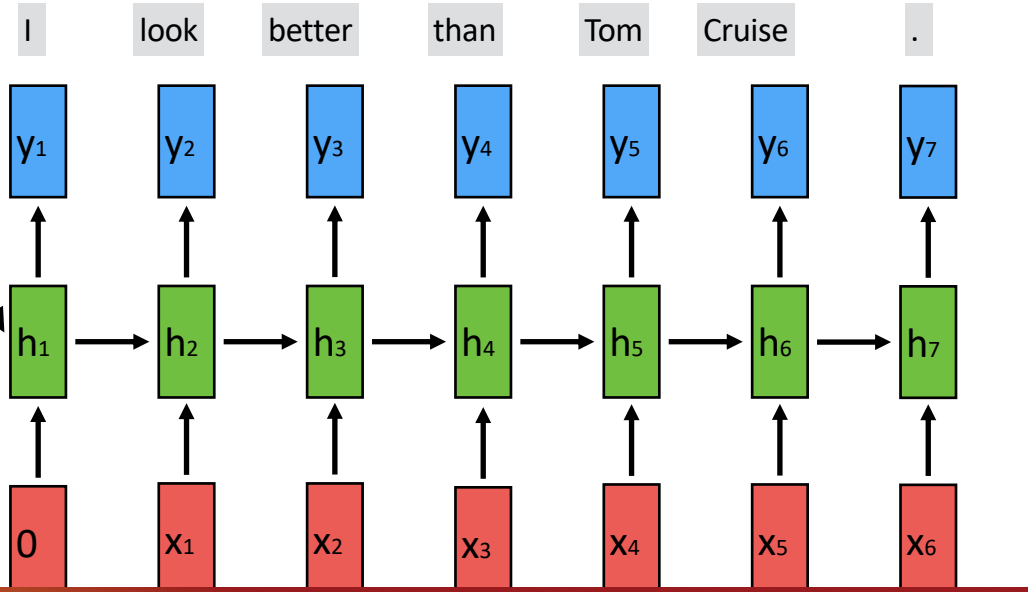


USC Viterbi School of Engineering

Image Captioning

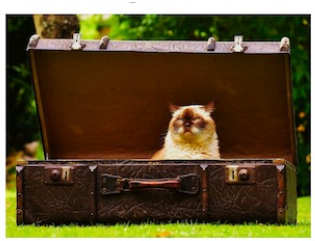




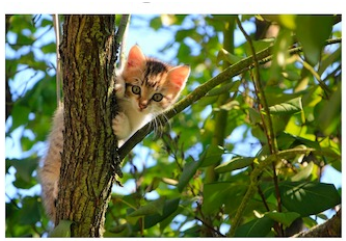


USC Viterbi School of Engineering

Image Captioning: Examples



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



A white teddy bear sitting in the grass



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field



A man riding a dirt bike on a dirt track

USC Viterbi School of Engineering

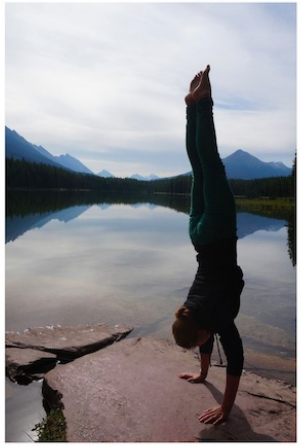
Image Captioning: Failures



A woman is holding a cat in her hand



A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch



A man in a baseball uniform throwing a ball

Question Answering

Mary moved to the bathroom.

John went to the hallway.

Where is Mary? bathroom

Example: bAbI task 1, Weston et al. (2015), CC BY 3.0

Next: encode text and question, then predict an answer word

Question Answering

Text

Mary moved to the bathroom.
John went to the hallway.

Question

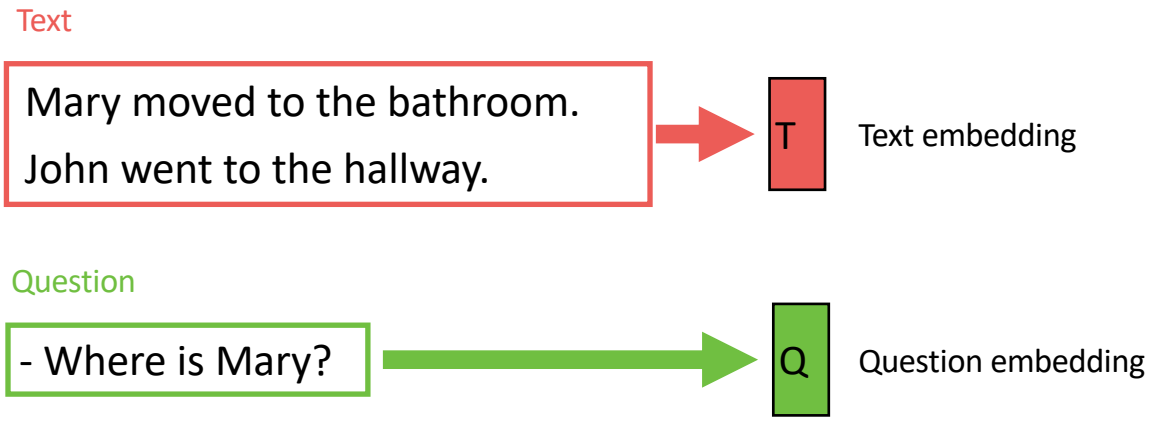
Where is Mary?

Answer

bathroom

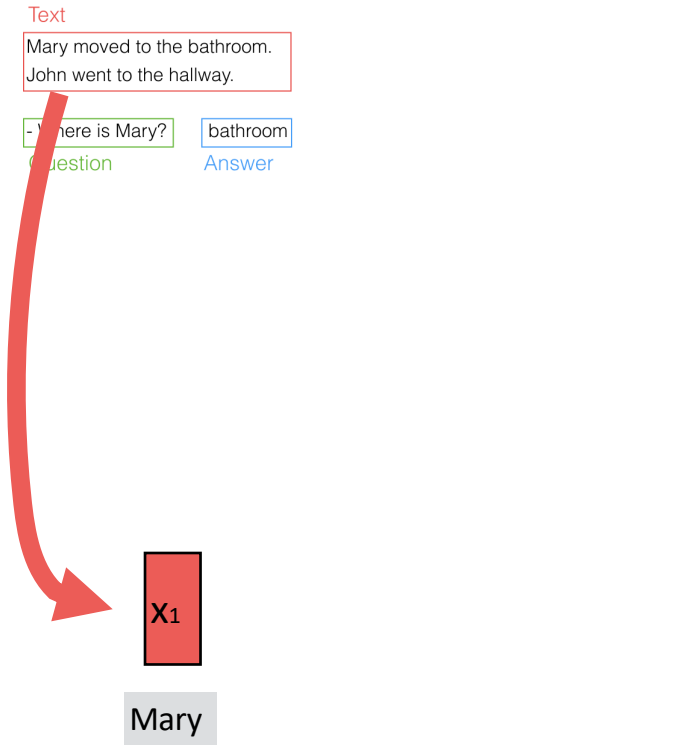
Example: bAbI task 1, Weston et al. (2015), CC BY 3.0
Next: encode text and question, then predict an answer word

Question Answering

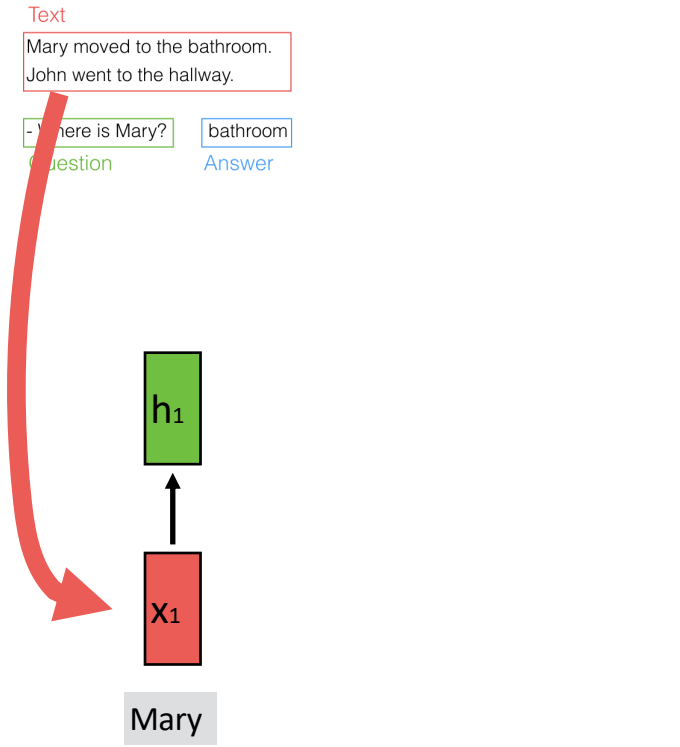


USC Viterbi School of Engineering

Question Answering

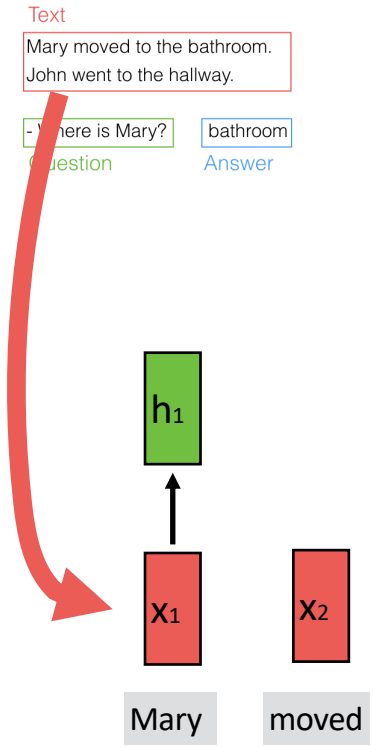


Question Answering



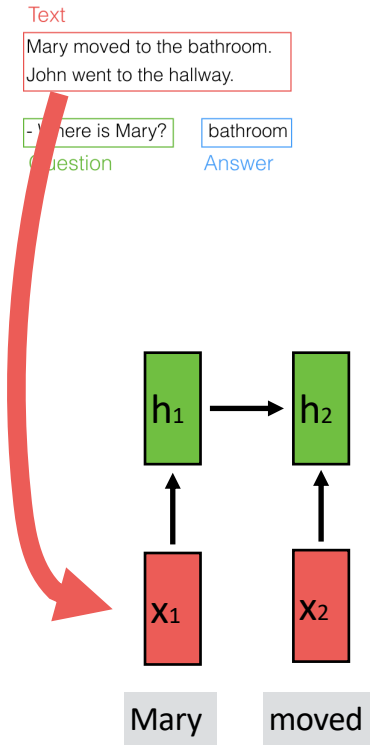
USC Viterbi School of Engineering

Question Answering



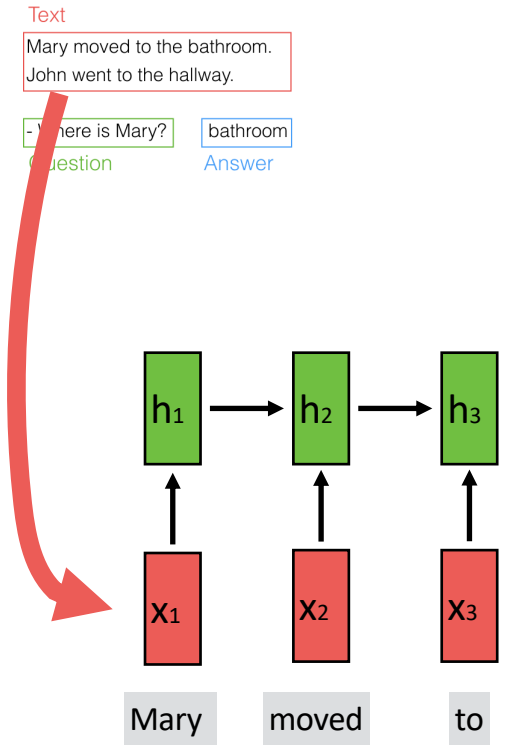
USC Viterbi School of Engineering

Question Answering



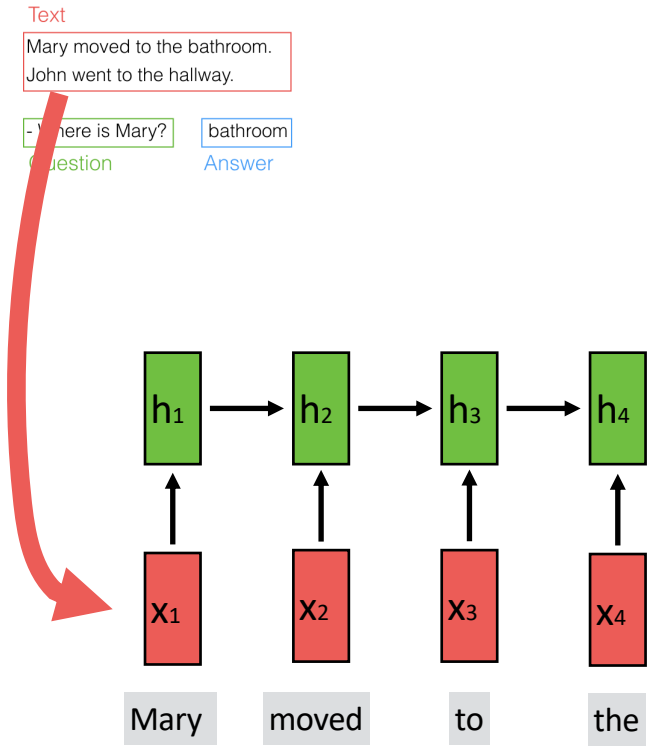
USC Viterbi School of Engineering

Question Answering



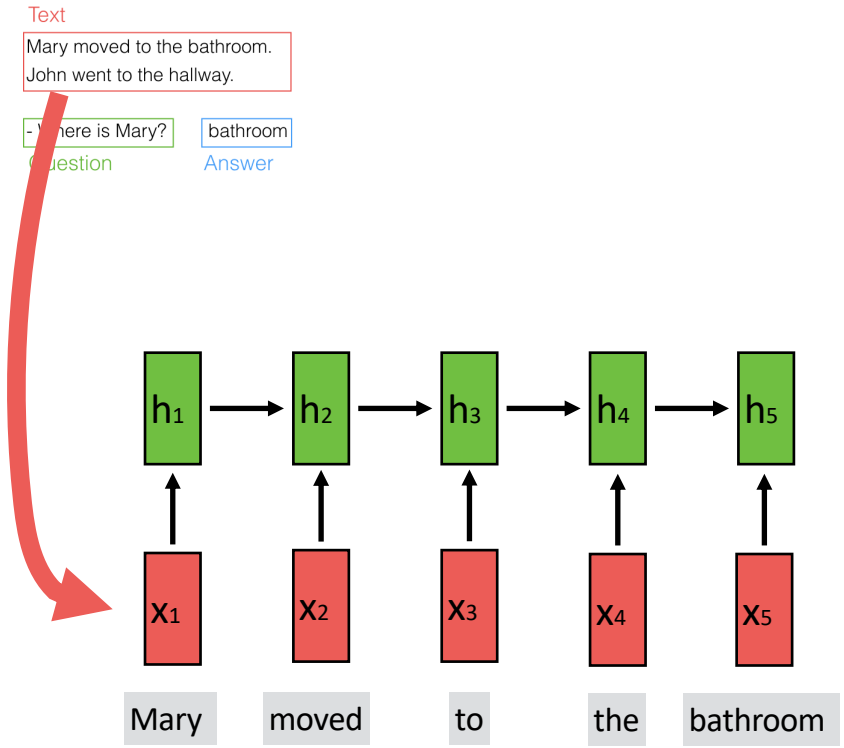
USC Viterbi School of Engineering

Question Answering

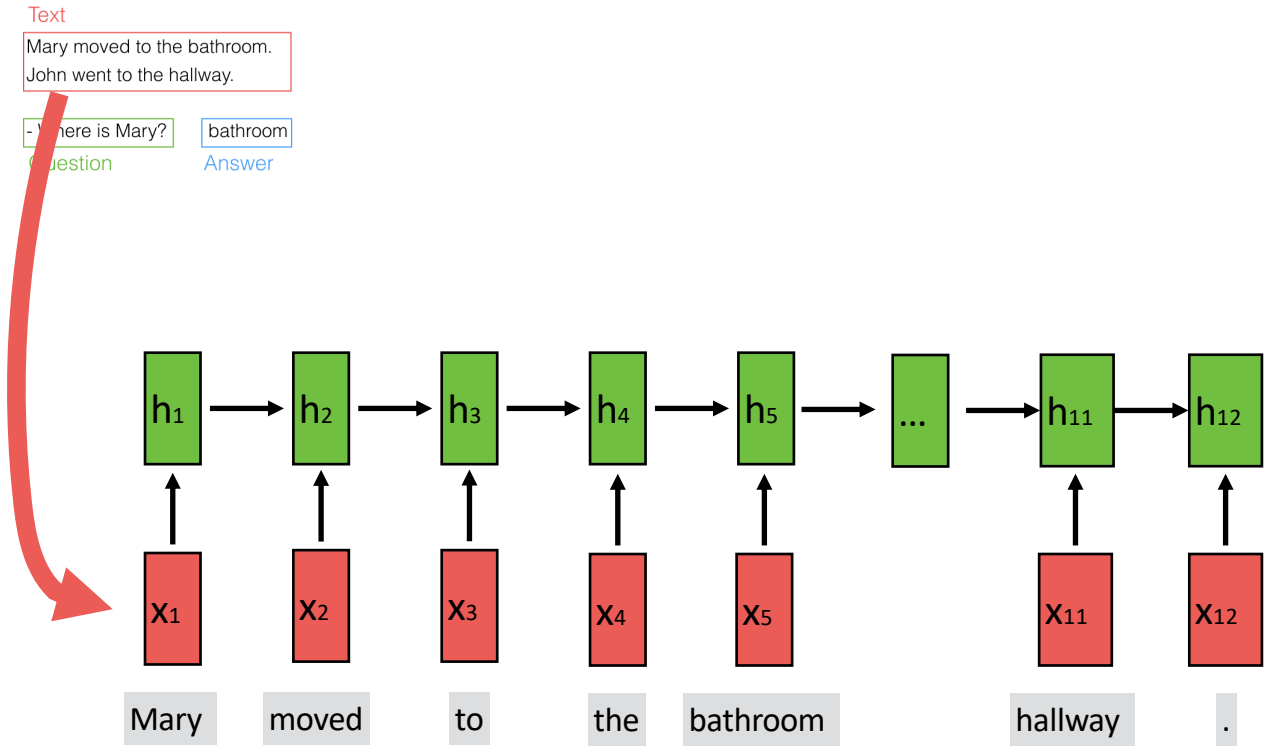


USC Viterbi School of Engineering

Question Answering

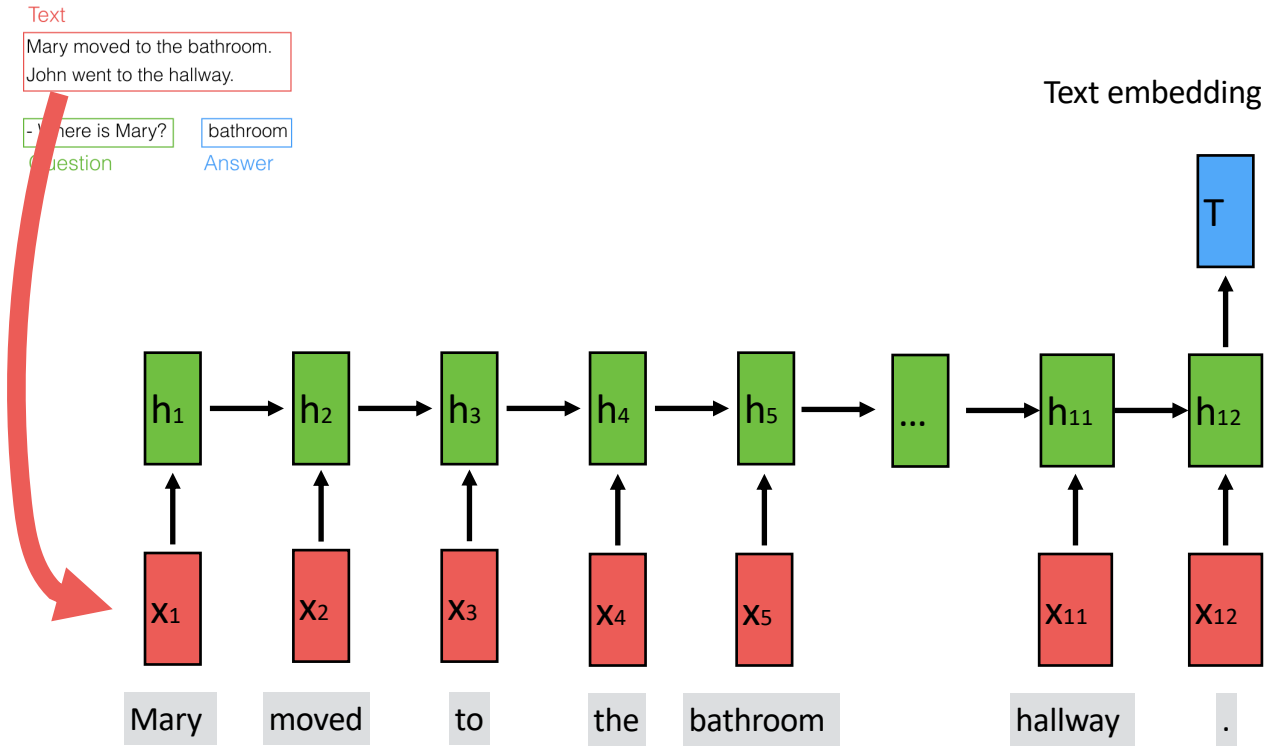


Question Answering



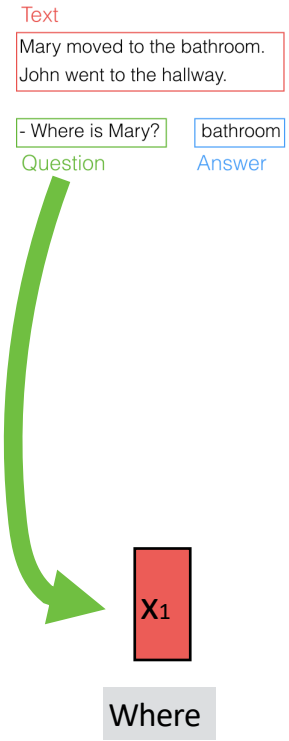
USC Viterbi School of Engineering

Question Answering



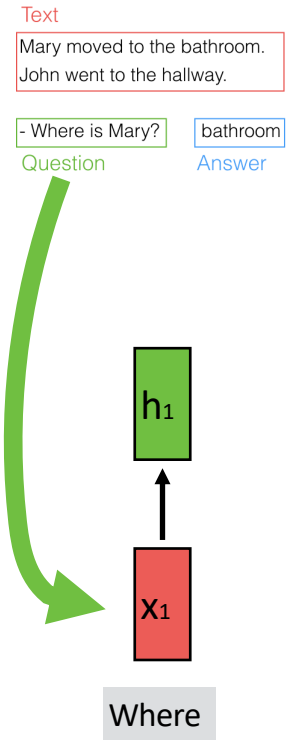
USC Viterbi School of Engineering

Question Answering



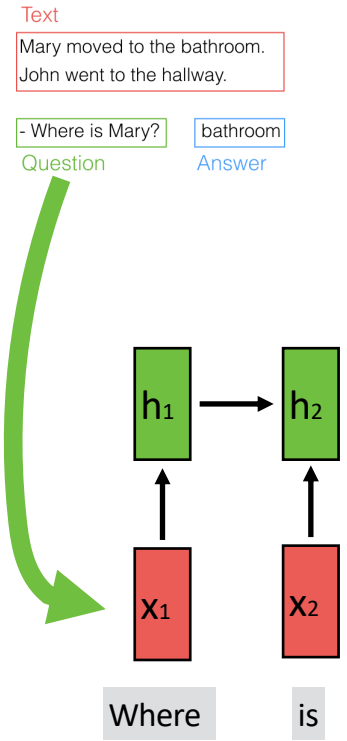
USC Viterbi School of Engineering

Question Answering



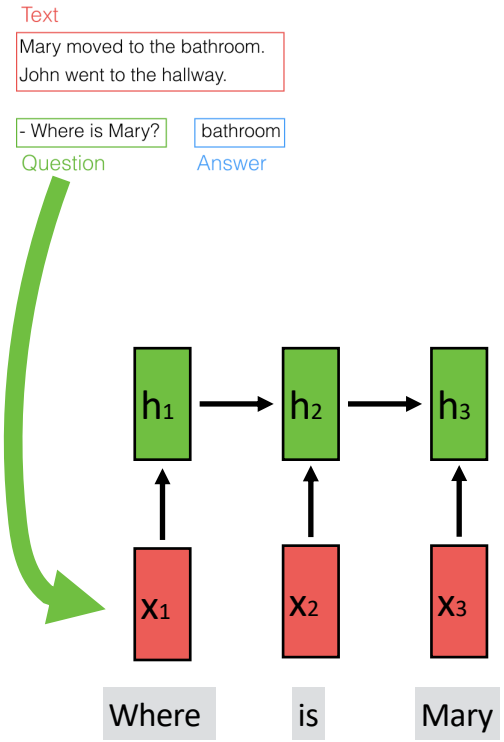
USC Viterbi School of Engineering

Question Answering



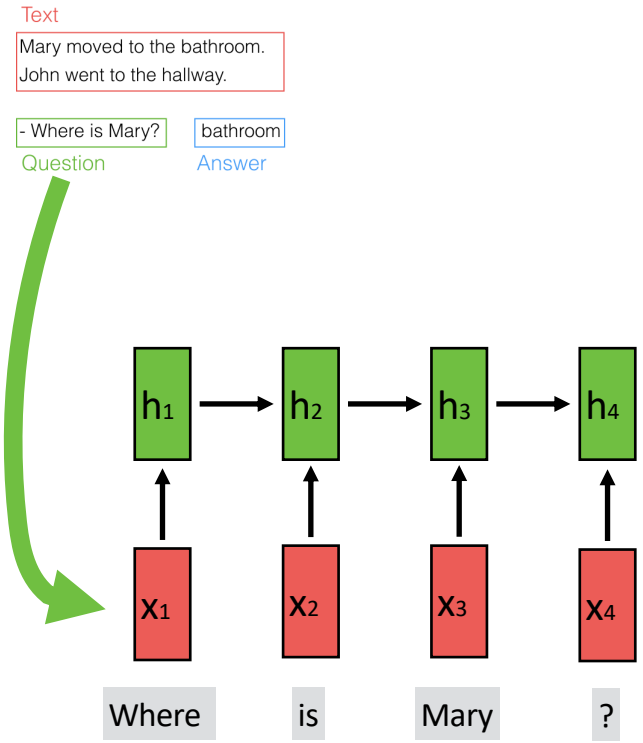
USC Viterbi School of Engineering

Question Answering



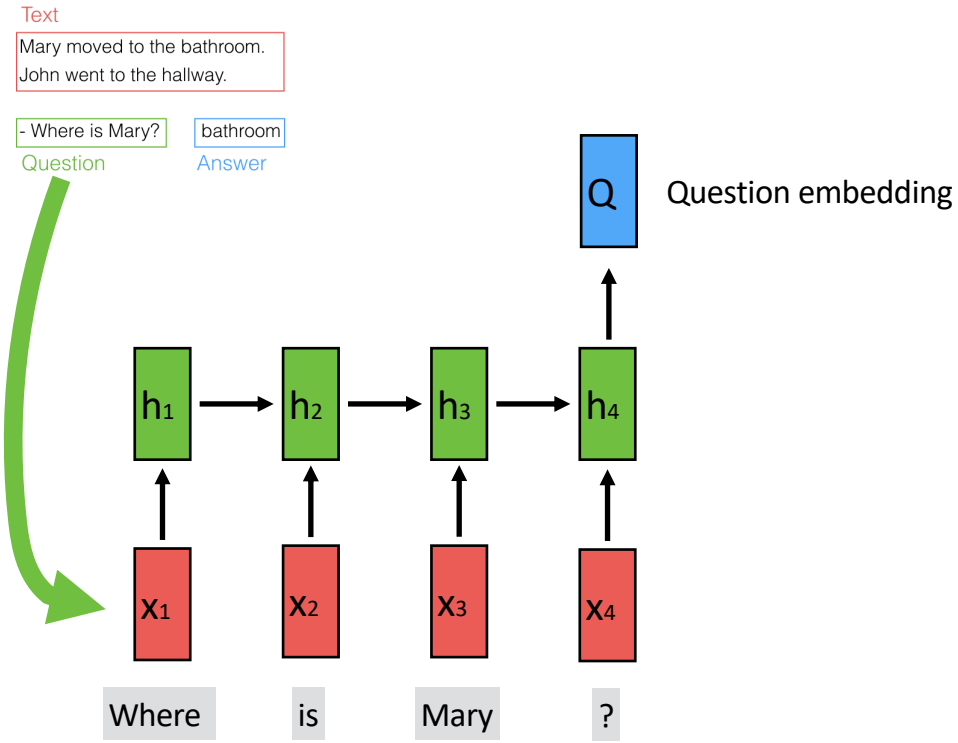
USC Viterbi School of Engineering

Question Answering



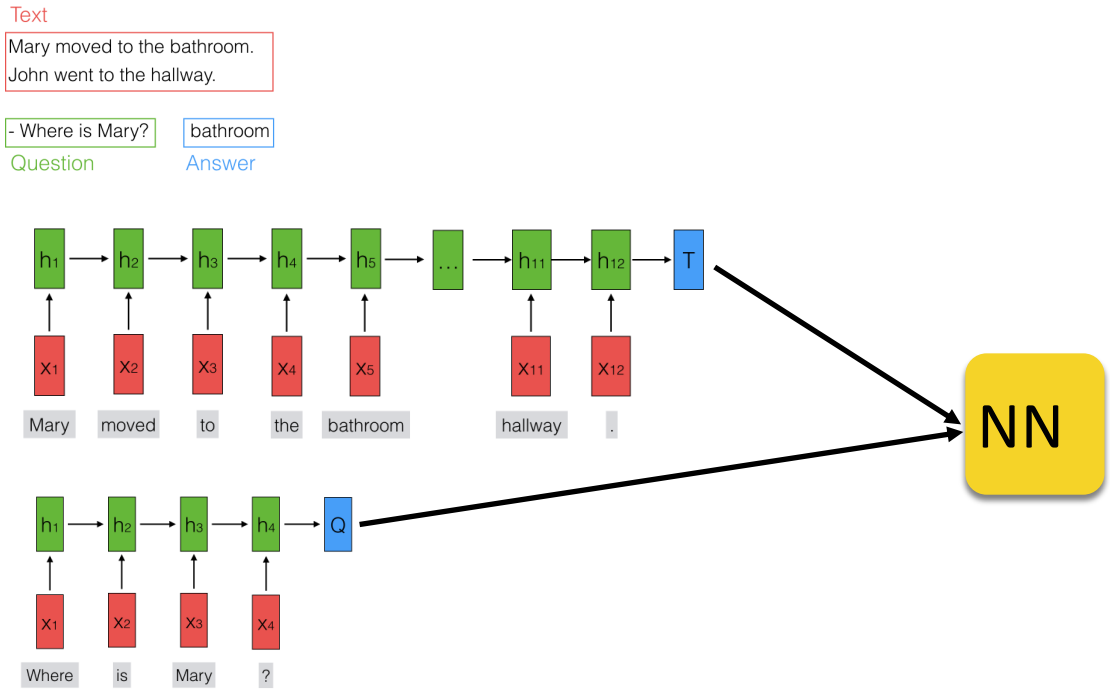
USC Viterbi School of Engineering

Question Answering



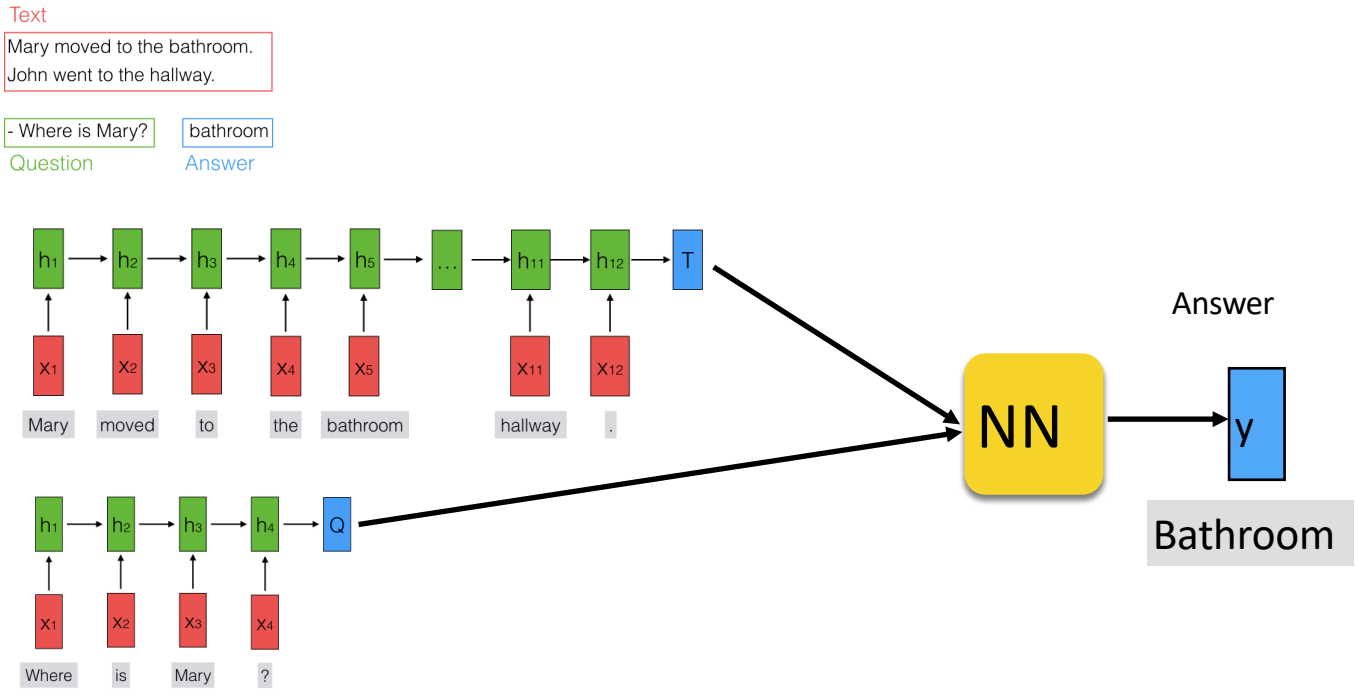
USC Viterbi School of Engineering

Question Answering



USC Viterbi School of Engineering

Question Answering



USC Viterbi School of Engineering

Read the Evidence, Then Select a Span

PASSAGE

The observatory opened in 1935.
Its telescope was upgraded in 1987.

Question: When did the observatory open?

Selected answer span: 1935

The answer must match the relation asked about in the question.

Course schematic.

Visual Question Answering



What color are her eyes?
What is the mustache made of?



How many slices of pizza are there?
Is this a vegetarian pizza?



Is this person expecting company?
What is just under the tree?

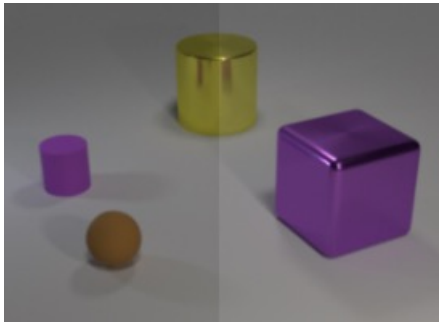


Does it appear to be rainy?
Does this person have 20/20 vision?

VQA: Visual Question Answering, Agrawal et al
A simple neural network module for relational reasoning, Santoro et al

Visual Question Answering

Image



Question

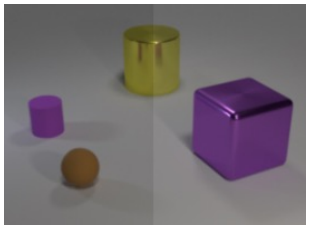
What is the size of the brown sphere?

Answer

Small

Visual Question Answering

Image

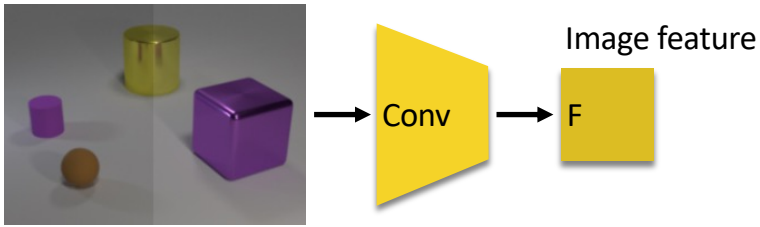


Question

What is the size of the brown sphere?

Visual Question Answering

Image

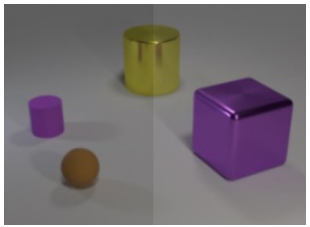


Question

What is the size of the brown sphere?

Visual Question Answering

Image



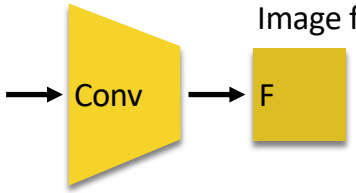
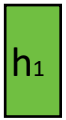


Image feature

Question

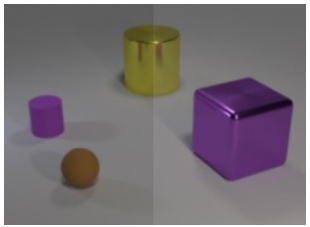
What is the size of the brown sphere?



What

Visual Question Answering

Image



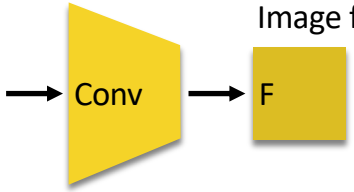
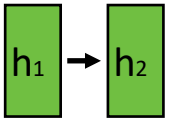


Image feature

Question

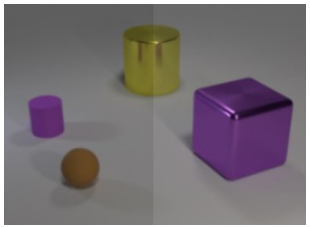
What is the size of the brown sphere?



What is

Visual Question Answering

Image



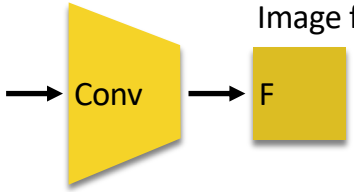
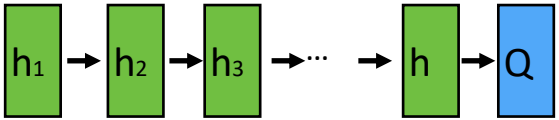


Image feature

Question

What is the size of the brown sphere?

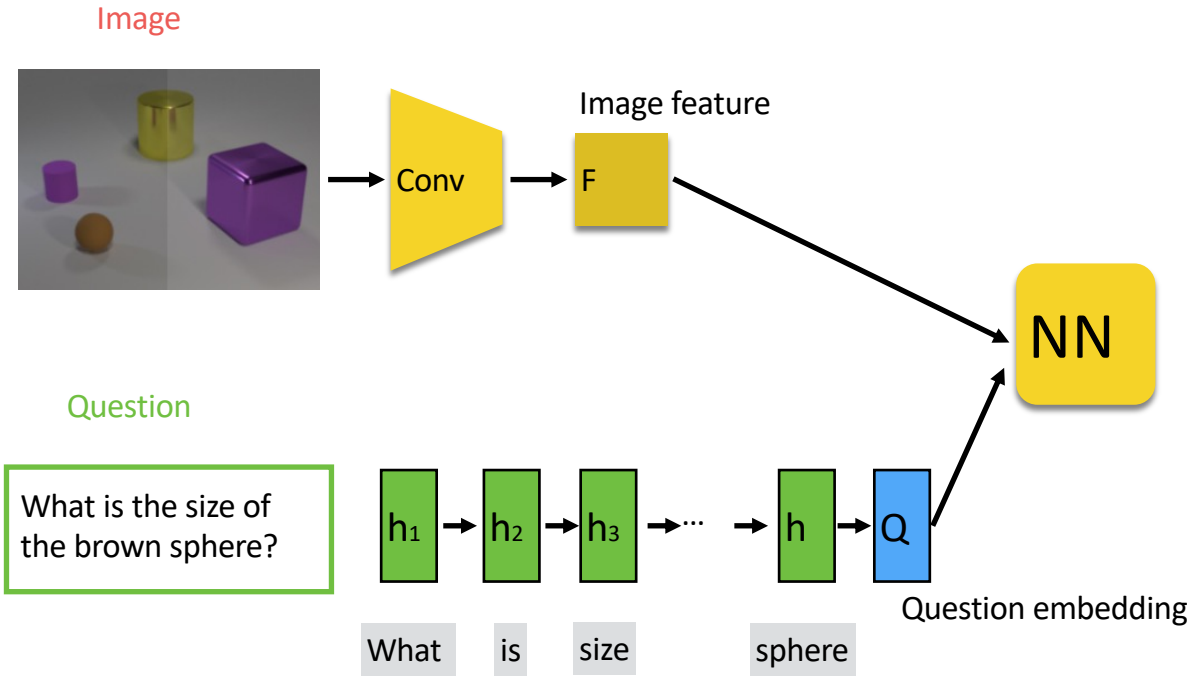


Question embedding

What is size sphere

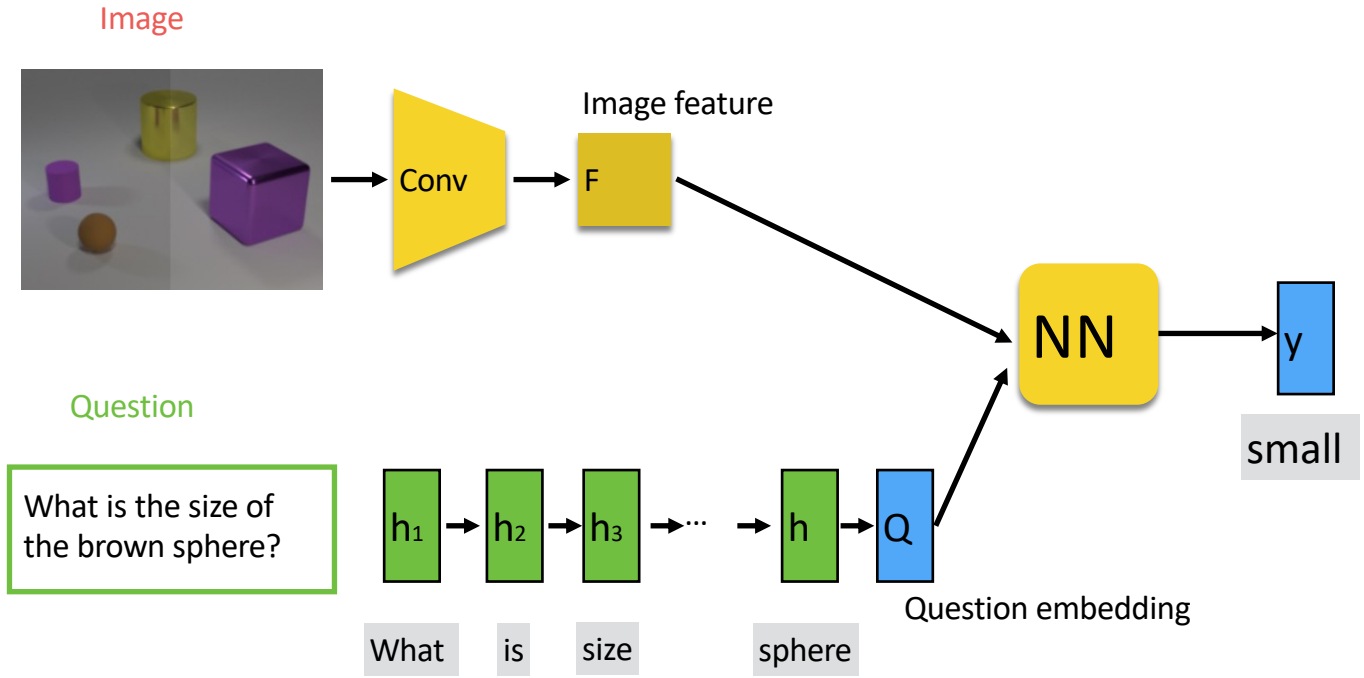
USC Viterbi School of Engineering

Visual Question Answering



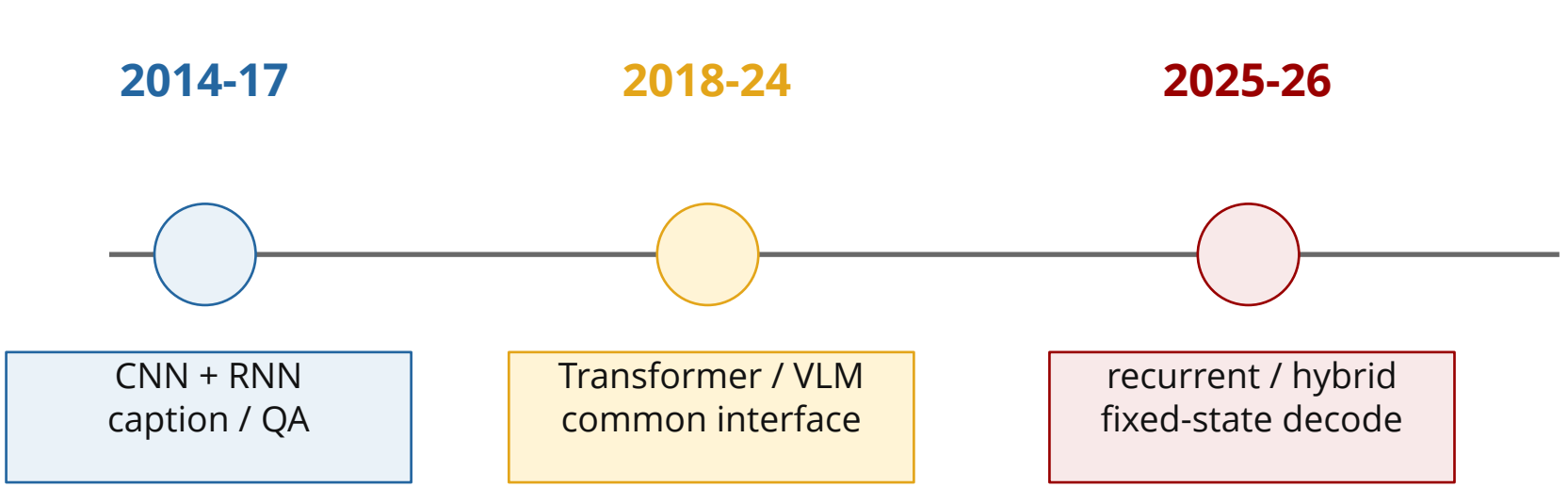
USC Viterbi School of Engineering

Visual Question Answering



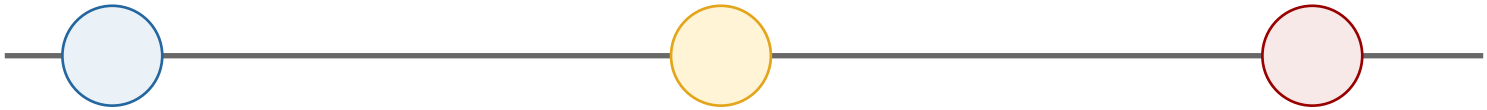
USC Viterbi School of Engineering

From 2014 RNNs to 2026 Systems



Keep the mechanisms; update the deployment framing.

What Survives Across Generations



STATE

what persists across steps

SHARED RULE

same parameters at many positions

CONDITIONING

which evidence changes output

INTERFACE

how one module feeds another

Architecture names change faster than these design questions.

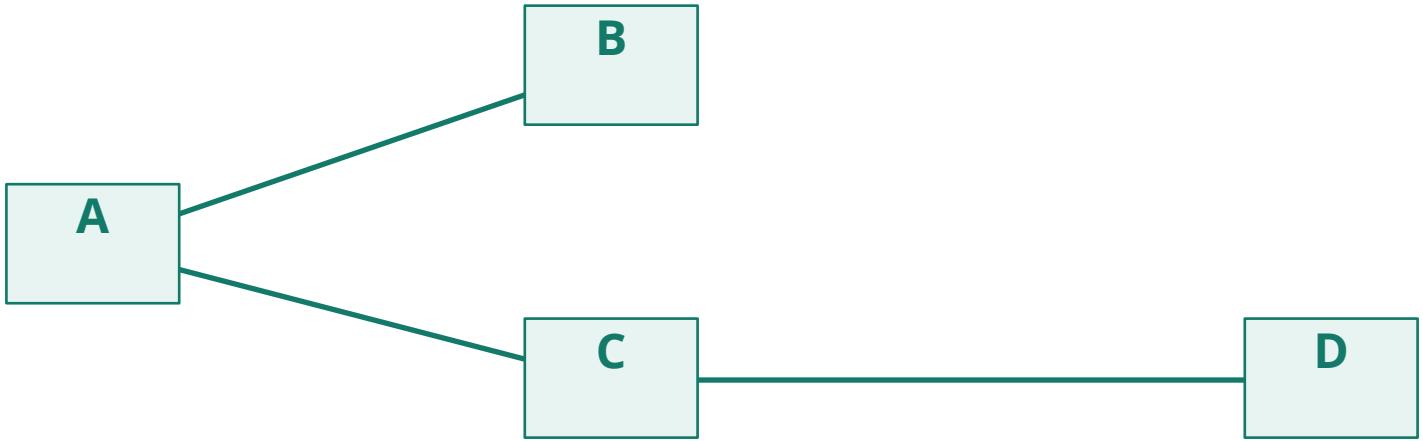
Sequence: Follow the Given Order



A recurrent step consumes the next item in order.

Course schematic.

Graph: Follow the Given Edges



A graph layer reads neighbors from the previous layer.

Course schematic.

Reference: LSTM Details

Equations, gradient paths, and worked values

Optional material for a deeper implementation study.

The classroom introduction focuses on memory and gates.

Course schematic.

Long Short Term Memory (LSTM)

- Learn to retain information across time
  - Additive cell updates provide a controllable gradient path

Hochreiter and Schmidhuber, “Long Short Term Memory”, Neural Computation 1997
Colah's blog: http://colah.github.io/posts/2015-08-Understanding-LSTMs/

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

e.g., $g = tanh(W_{gh}h_{t-1} + W_{gx}x_t)$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

Long Short Term Memory (LSTM)

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

$$W = \begin{bmatrix} W_i \\ W_f \\ W_o \\ W_g \end{bmatrix}$$

Long Short Term Memory (LSTM)

LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

$$W = \begin{bmatrix} W_i \\ W_f \\ W_o \\ W_g \end{bmatrix}$$

$$i = \sigma(W_i \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

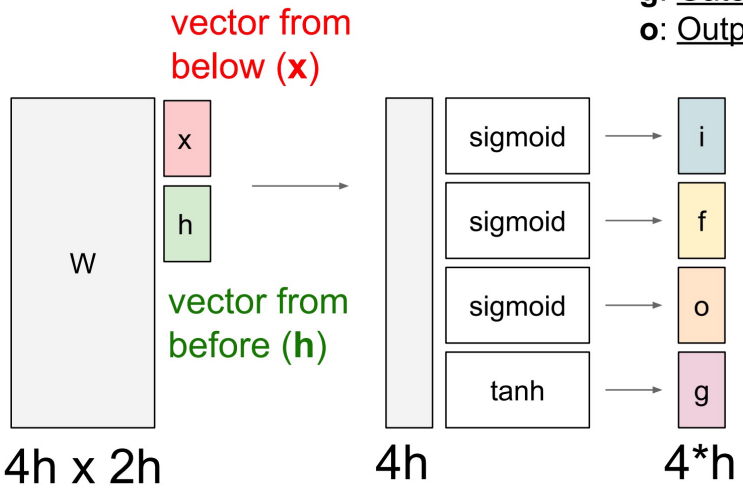
$$f = \sigma(W_f \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

$$o = \sigma(W_o \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

$$g = tanh(W_g \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix})$$

Long Short Term Memory (LSTM)

f: Forget gate, Whether to erase cell
i: Input gate, whether to write to cell
g: Gate gate (?), How much to write to cell
o: Output gate, How much to reveal cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

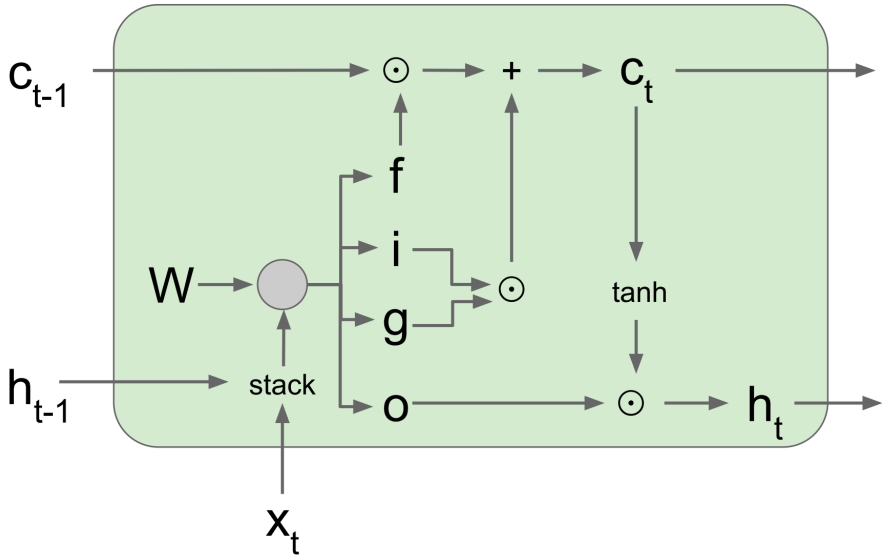
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

g: candidate content (tanh); i controls how much is written.

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

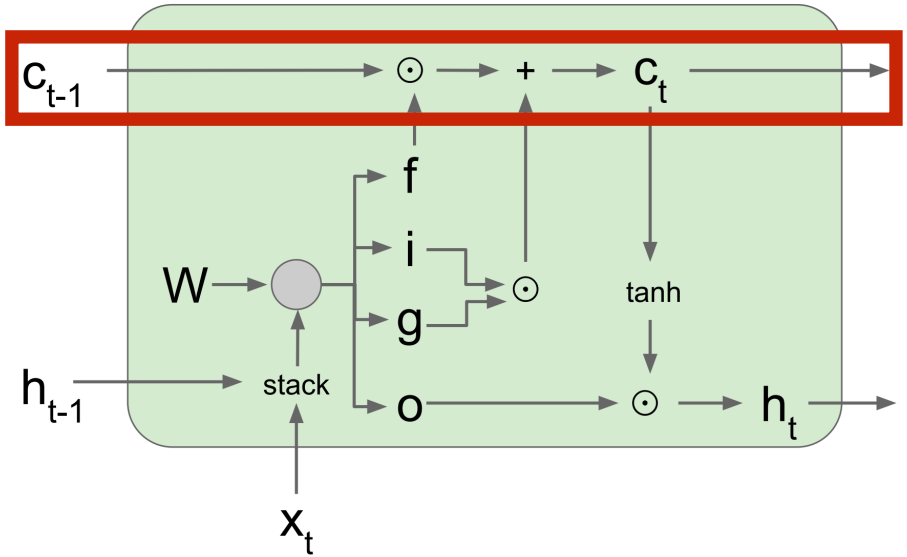
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

Long Short Term Memory (LSTM)



- Cell state: information to pass to the next state

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

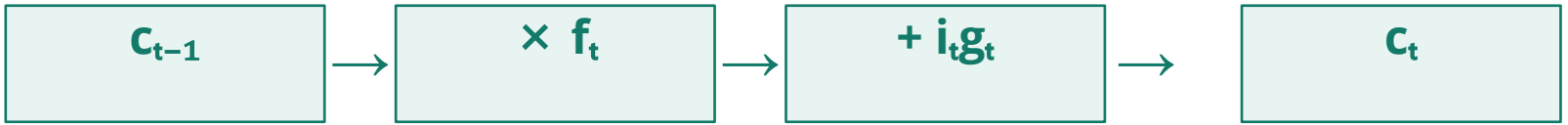
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

The Cell State Has an Additive Path

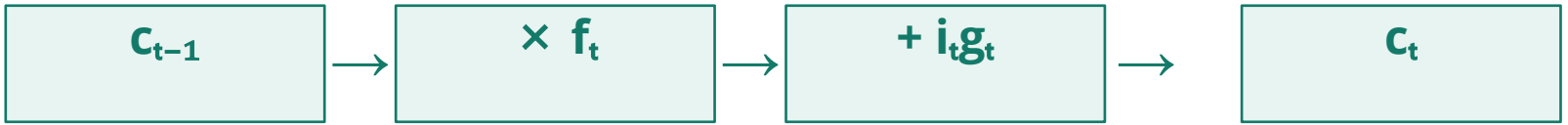


$$c_t = f_t\, c_{t-1} + i_t\, g_t$$

The additive route avoids a repeated recurrent weight matrix.

Standard LSTM equations; see PyTorch LSTM documentation. Scalar path shown.

Forget Gates Control That Path

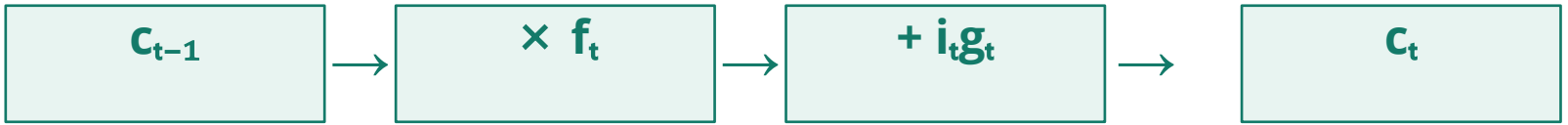


Direct-path multiplier: $f_t \times f_{t-1} \times \ldots$

This path preserves a gradient when the forget gates stay near 1.

Standard LSTM equations; see PyTorch LSTM documentation. Scalar path shown.

Long Memory Is a Learned Behavior



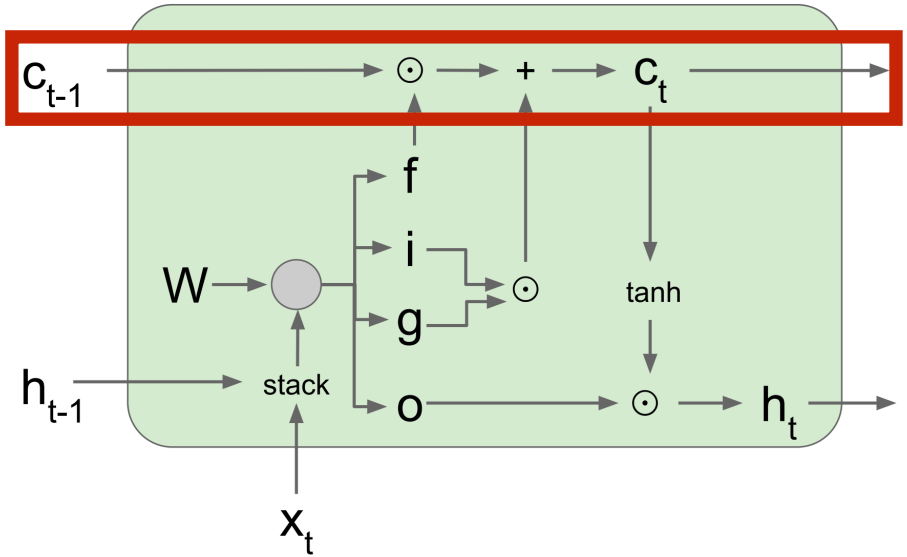
$$10 \text{ steps with } f = 0.9: \quad 0.9^{10} \approx 0.349$$

Direct cell path only; gates also depend on the recurrent state.

LSTM improves the gradient route; retention still depends on gates.

Standard LSTM equations; see PyTorch LSTM documentation. Scalar path shown.

Long Short Term Memory (LSTM)



- Cell state: information to pass to the next state

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

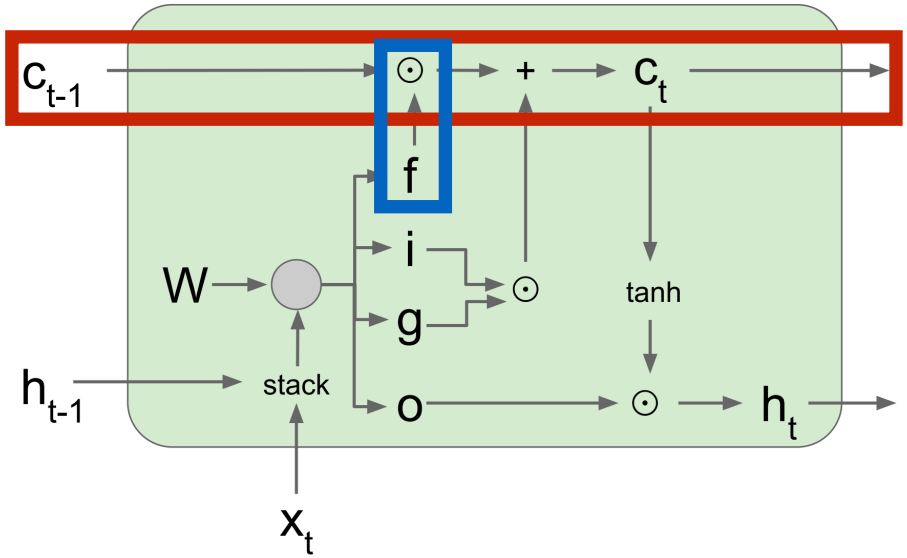
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

Long Short Term Memory (LSTM)



- Cell state: information to pass to the next state
- Forget gate: how much information to pass (sigmoid)

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

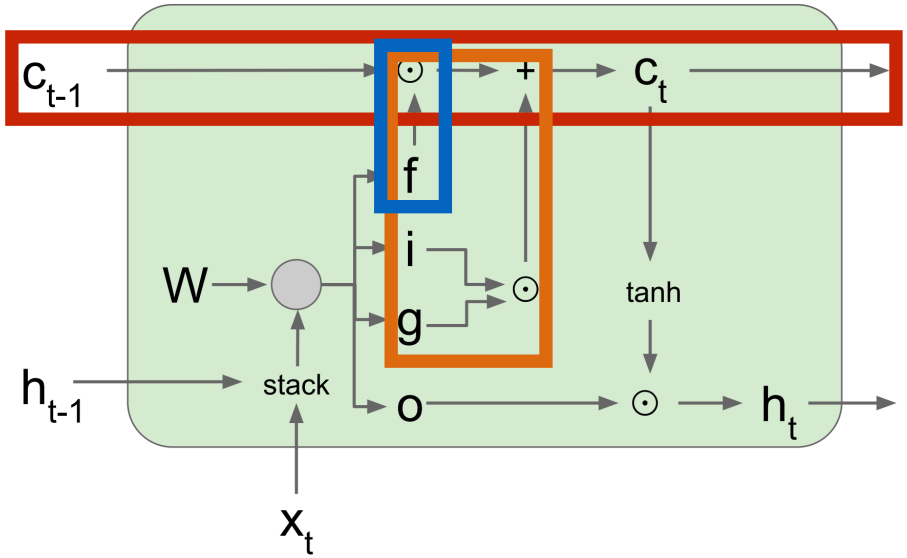
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

Long Short Term Memory (LSTM)



- Cell state: information to pass to the next state
- Forget gate: how much information to pass (sigmoid)
- **Input gate:** how much information to add (sigmoid)
- **Candidate update:** content to add (tanh)

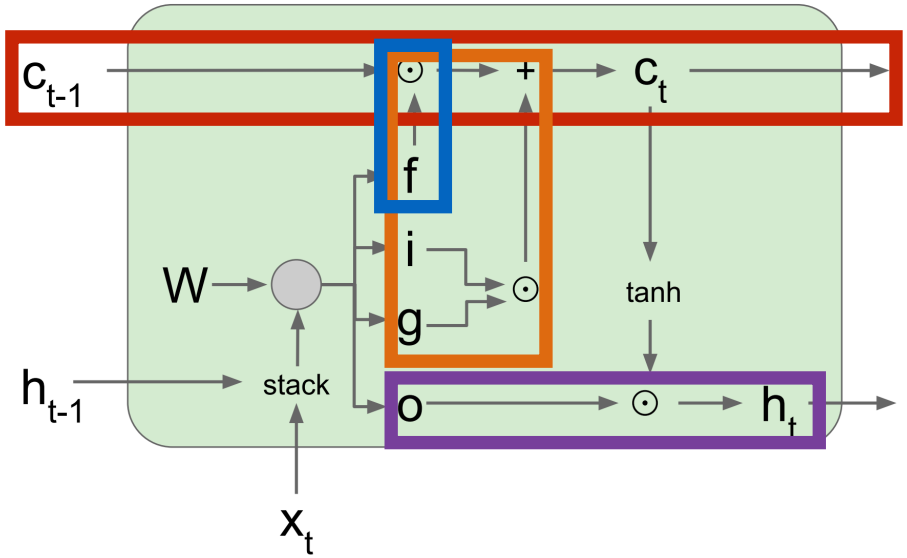
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Long Short Term Memory (LSTM)



- Cell state: information to pass to the next state
- Forget gate: how much information to pass (sigmoid)
- **Input gate:** how much information to add (sigmoid)
- **Candidate update:** content to add (tanh)
- **Output gate:** what we are going to output (sigmoid)

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

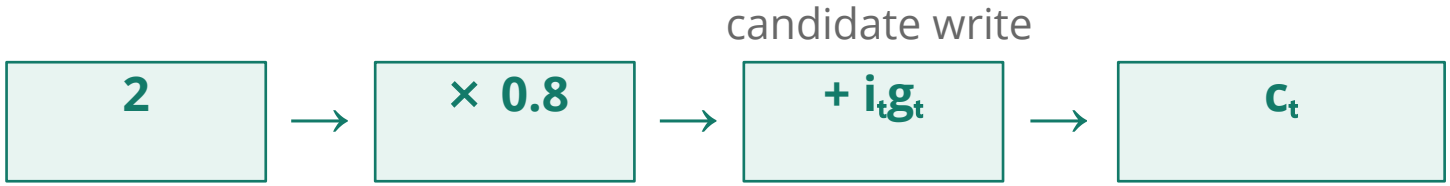
$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

USC Viterbi School of Engineering

Read the Gate Values

$c_{t-1} = 2 \quad f_t = 0.8 \quad i_t = 0.5 \quad g_t = 0.4 \quad o_t = 0.75$



Sigmoid gates: retain, write, expose.
The tanh candidate supplies the content to write.

The gates decide how the state changes.

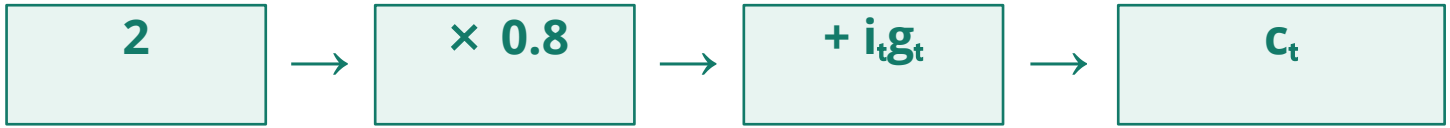
Illustrative gate outputs; standard LSTM equations.

USC Viterbi School of Engineering

Retain Part of the Old Cell

$c_{t-1} = 2 \quad f_t = 0.8 \quad i_t = 0.5 \quad g_t = 0.4 \quad o_t = 0.75$

candidate write



Retained value $= 0.8 \times 2 = 1.6$

The forget gate scales the existing cell value.

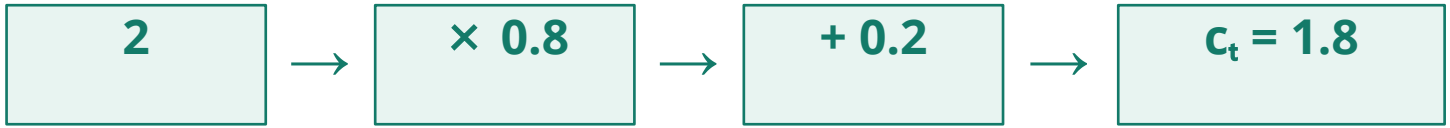
Illustrative gate outputs; standard LSTM equations.

USC Viterbi School of Engineering

Add the New Candidate

$c_{t-1} = 2 \quad f_t = 0.8 \quad i_t = 0.5 \quad g_t = 0.4 \quad o_t = 0.75$

candidate write



Written value $= 0.5 \times 0.4 = 0.2$

Updated cell $= 1.6 + 0.2 = 1.8$

Old information and new information meet by addition.

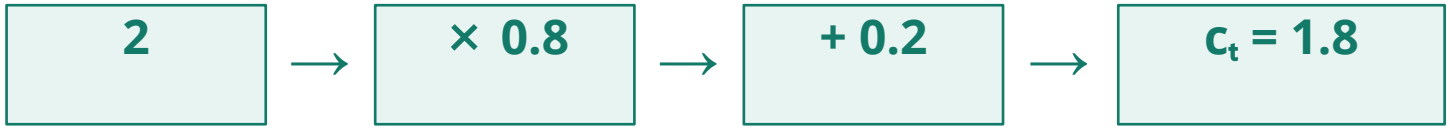
Illustrative gate outputs; standard LSTM equations.

USC Viterbi School of Engineering

Expose Part of the Updated Cell

$c_{t-1} = 2 \quad f_t = 0.8 \quad i_t = 0.5 \quad g_t = 0.4 \quad o_t = 0.75$

candidate write



$$h_t = o_t \tanh(c_t) = 0.75 \times \tanh(1.8) \approx 0.710$$

The cell state and the exposed hidden state have different roles.

Illustrative gate outputs; standard LSTM equations.

Other variants

GRU [*Learning phrase representations using rnn encoder-decoder for statistical machine translation*, Cho et al. 2014]

$$
\begin{aligned}
r_t &= \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\
z_t &= \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\
\tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t
\end{aligned}
$$

[*LSTM: A Search Space Odyssey*, Greff et al., 2015]

[*An Empirical Exploration of Recurrent Network Architectures*, Jozefowicz et al., 2015]

MUT1:

$$
\begin{aligned}
z &= \mathrm{sigm}(W_{\mathrm{xz}}x_t + b_{\mathrm{z}}) \\
r &= \mathrm{sigm}(W_{\mathrm{xr}}x_t + W_{\mathrm{hr}}h_t + b_{\mathrm{r}}) \\
h_{t+1} &= \tanh(W_{\mathrm{hh}}(r \odot h_t) + \tanh(x_t) + b_{\mathrm{h}}) \odot z \\
&+ h_t \odot (1 - z)
\end{aligned}
$$

MUT2:

$$
\begin{aligned}
z &= \mathrm{sigm}(W_{\mathrm{xz}}x_t + W_{\mathrm{hz}}h_t + b_{\mathrm{z}}) \\
r &= \mathrm{sigm}(x_t + W_{\mathrm{hr}}h_t + b_{\mathrm{r}}) \\
h_{t+1} &= \tanh(W_{\mathrm{hh}}(r \odot h_t) + W_{xh}x_t + b_{\mathrm{h}}) \odot z \\
&+ h_t \odot (1 - z)
\end{aligned}
$$

MUT3:

$$
\begin{aligned}
z &= \mathrm{sigm}(W_{\mathrm{xz}}x_t + W_{\mathrm{hz}}\tanh(h_t) + b_{\mathrm{z}}) \\
r &= \mathrm{sigm}(W_{\mathrm{xr}}x_t + W_{\mathrm{hr}}h_t + b_{\mathrm{r}}) \\
h_{t+1} &= \tanh(W_{\mathrm{hh}}(r \odot h_t) + W_{xh}x_t + b_{\mathrm{h}}) \odot z \\
&+ h_t \odot (1 - z)
\end{aligned}
$$

Reference: Cache and Architecture Choices

KV cache and modern recurrent models

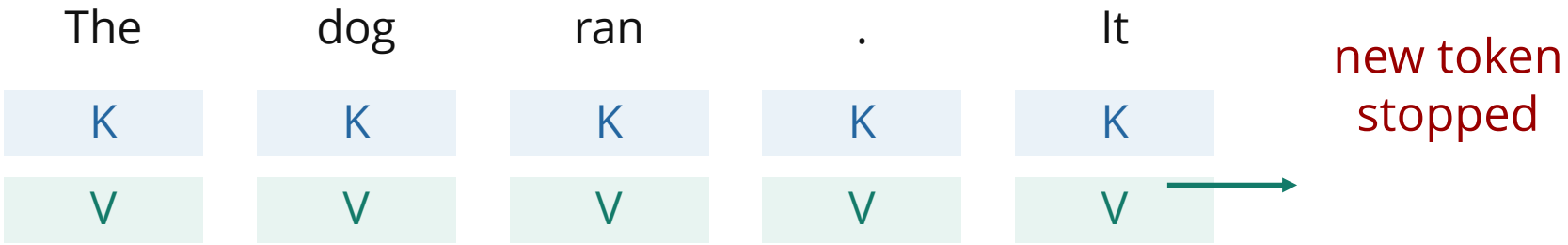
Return after learning how attention reads context.

Optional extensions to the introductory examples.

Course schematic.

KV Cache Reuses Earlier Work

Each layer stores two vectors for each processed token:

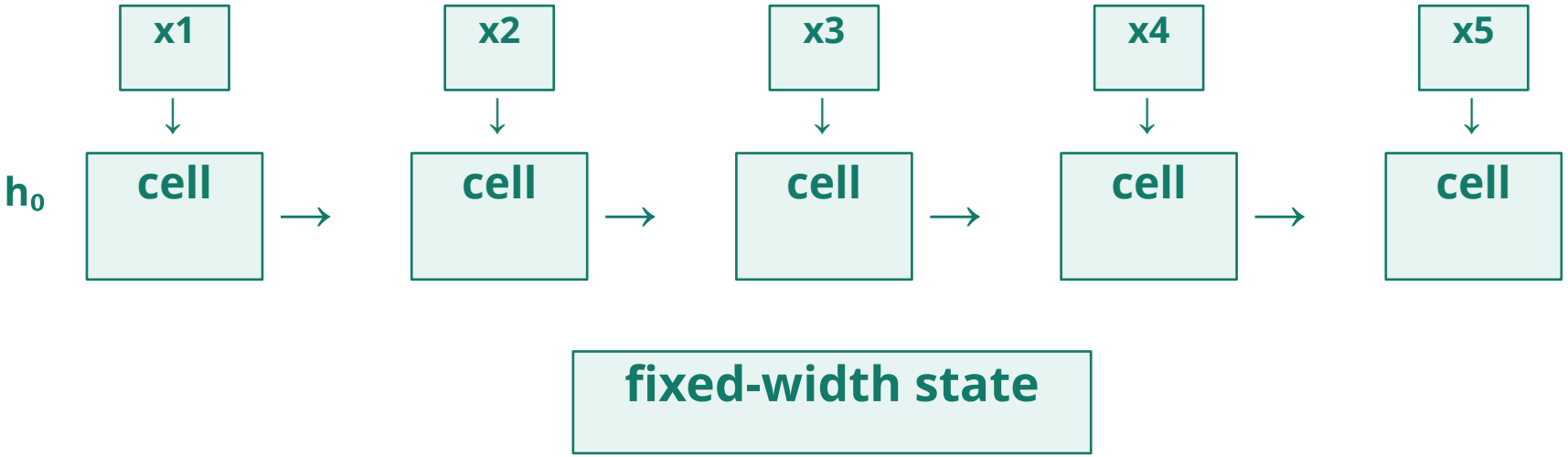


K (key): helps score relevance. V (value): information to mix.

Reuse past K/V; compute and append K/V for the new position.

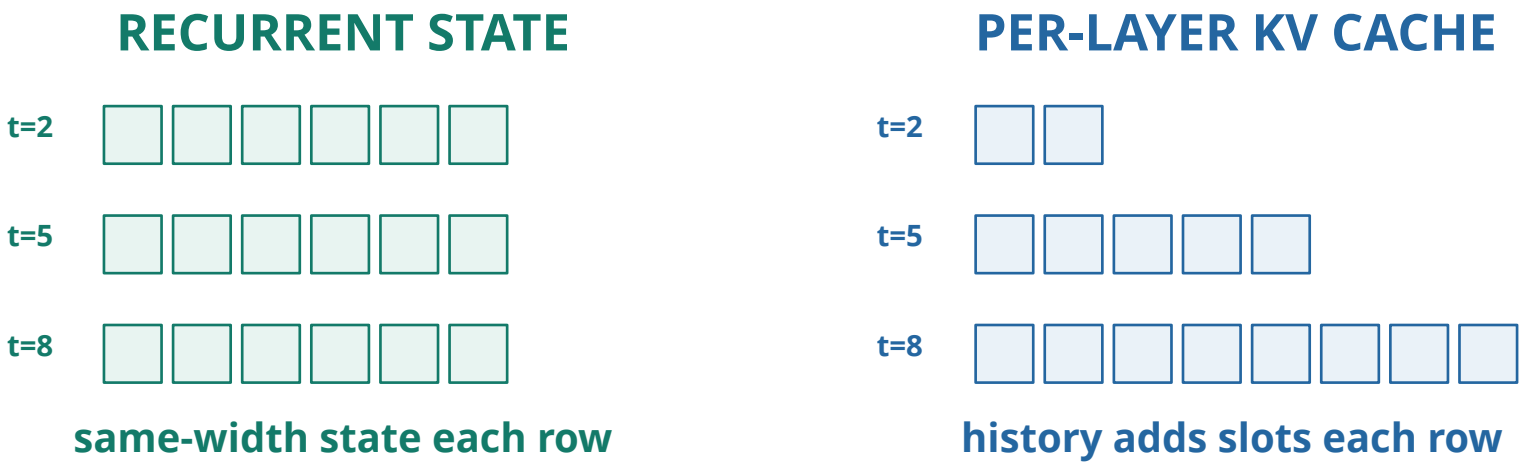
Hugging Face Transformers: How caching works. Simplified per-layer cache.

One State Flows Through Time



Length grows; the carried state width stays fixed.

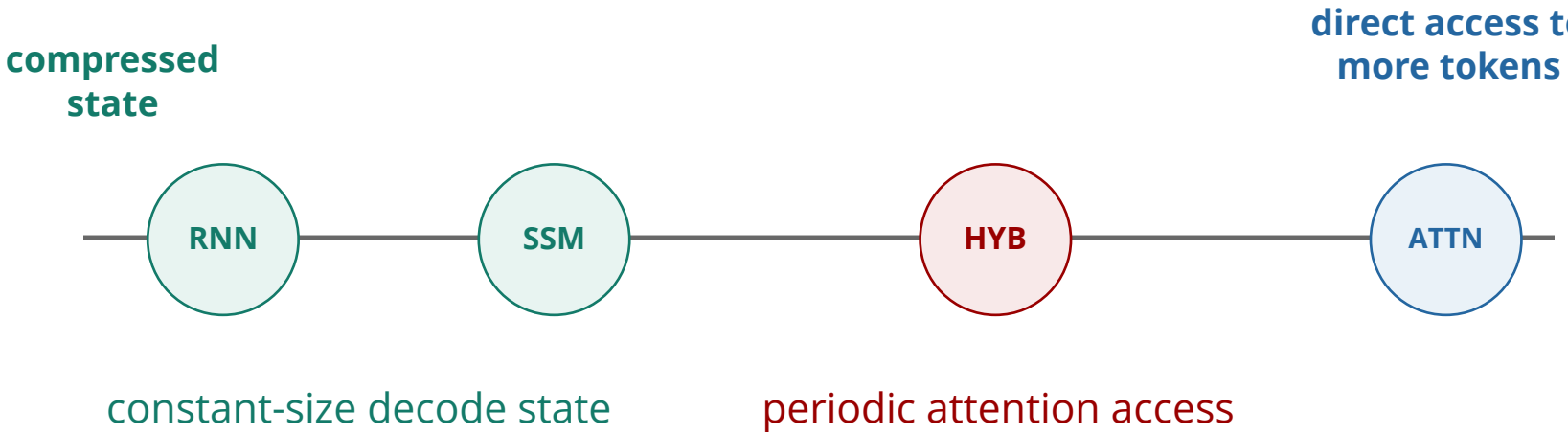
Fixed State vs. Growing History



Memory scaling differs; model quality is a separate question.

The Trade: Compress or Re-read

3 / 3



Efficiency comes from what the model stores and can revisit.

Modern Recurrence: Two Design Paths

STRONGER RECURRENCE

richer state updates
without full history

xLSTM scaling study

Mamba-3: complex + MIMO

HYBRID LAYERS

fixed-state layers plus
periodic direct access

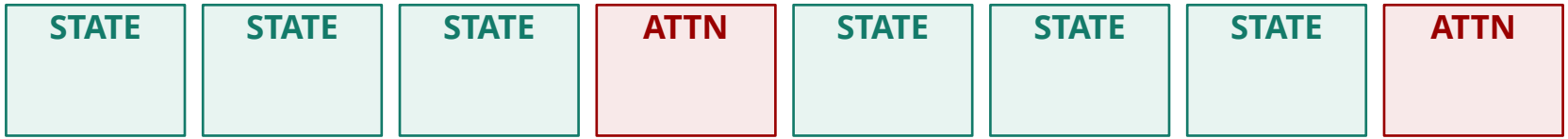
Qwen3-Next: 3:1 mix

Kimi Linear: KDA + MLA

Mechanisms are documented; benchmark claims remain source-specific.

Periodic Attention Restores Direct Access

8-layer example along depth →

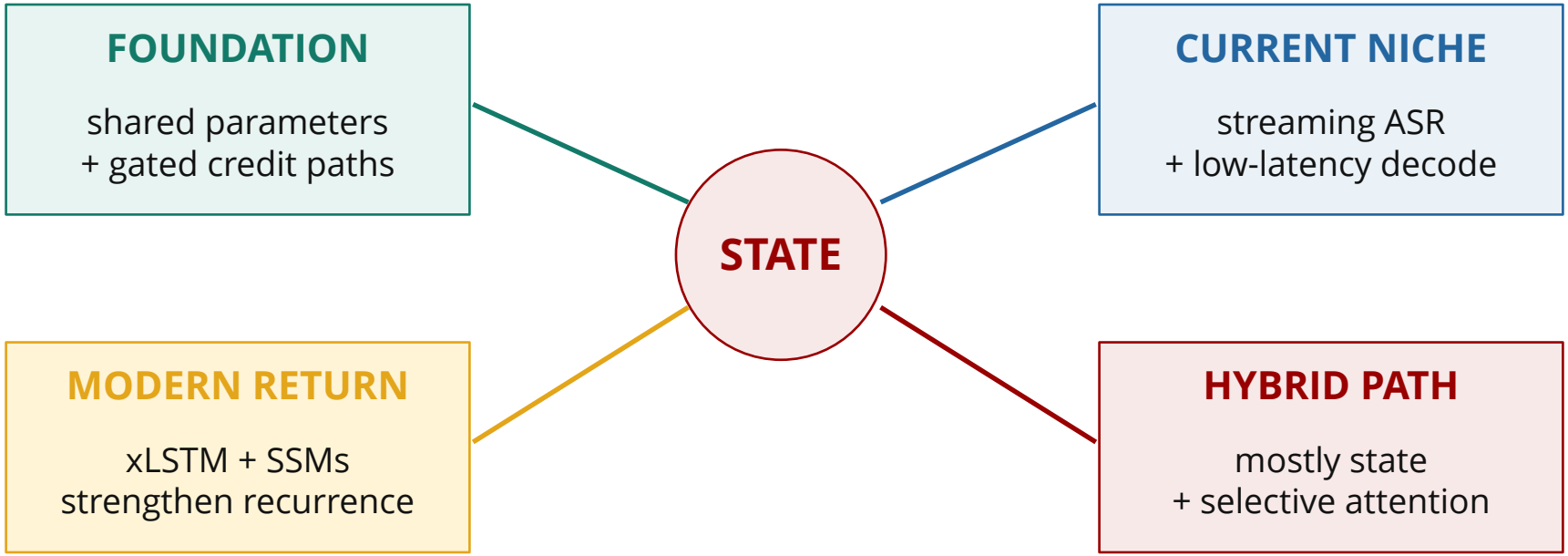


bounded recurrent memory

periodic token lookup

Hybrids spend direct-access capacity selectively.

RNN and LSTM in 2026



Ask what state is stored, what can be revisited, and at what cost.

Optional: Bidirectional Context

Task: read the sentence and interpret the word “bank”.



Both directions help when the full input is already available.

Bidirectional RNN/LSTM concept; see PyTorch LSTM documentation.